

III B.tech – I Semester
COMPUTER NETWORKS LAB
(GR24A3094)

Department of Computer Science and Engineering

GOKARAJU RANGARAJU
INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Autonomous)

Gokaraju Rangaraju Institute of Engineering and Technology (Autonomous)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION AND MISSION OF THE INSTITUTE

To be among the best of the institutions for engineers and technologists with attitudes, skills and knowledge and to become an epicentre of creative solutions.

To achieve and impart quality education with an emphasis on practical skills and social relevance.

VISION AND MISSION OF THE DEPARTMENT

To be a center of global excellence and to emerge as a valuable resource for industry and society.

1) To produce qualified and competent computer professionals with international standards.

2) To foster innovative and application-oriented research capabilities of young minds for the progress of society.
3) To inculcate strong ethical values and professional behaviour so as to adapt to the emerging changes in the field of computing technology.

PROGRAM EDUCATIONAL OBJECTIVES

PEO 1: Graduates will be prepared for a successful career in Computer Science discipline and related skills for industry to meet the needs of the nation and leading industries and also to excel in postgraduate programs.

PEO 2: Graduates will continue to learn and apply the acquired knowledge to solve engineering problems and appreciation of the arts, humanities and social sciences.

PEO 3: Graduates will have good and broad scientific and engineering knowledgebase so as to comprehend, analyze, design and create novel products and solutions for real-time applications.

PEO 4: Graduates will understand professional and ethical responsibility, develop leadership, utilize membership opportunities, develop effective communication abilities, teamwork skills, multidisciplinary approach and life-long learning required for a successful professional career.

PROGRAM OUTCOMES (POs)

PO1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization for the solution of complex engineering problems.

PO2. **Problem analysis:** Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. **Design/Development of Solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public

health and safety, and cultural, societal, and environmental considerations.

PO4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal, and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and the need for sustainable development.

PO8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with the society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions

PO11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES(PSOs)

PSO1: **Emerging Technologies**: Apply the concepts of Computer Science and Engineering to learn the emerging technologies and to develop inventive solutions.

PSO2: **Industry oriented technical skills**: To foster industrial focused technical skills in Computer science and Engineering through value added courses and, to create a futuristic equipped professional.

Gokaraju Rangaraju Institute of Engineering and Technology

Course Code: GR24A3094 L/T/P/C:0/0/3/1.5

Course Outcomes:

1. Implement the Data link layer framing methods and protocols.
2. Illustrate routing, security and congestion control algorithms,
3. Analyze how network and Internetwork devices are configured using Packet Tracer.
4. Demonstrate Virtual LANs, Network Address Translation and routing protocols.
5. Demonstrate how to connect and monitor IOT devices

TASK-1

Implement the following Data Link Layer framing methods

a) Bit stuffing b) Character-stuffing c) Character count.

TASK -2

Implement the following Data Link layer protocols

a) Simplex protocol b) Stop and Wait protocol c) Sliding Window protocol

TASK -3

Design a program to implement the following:

a) Shortest Path routing protocol b) Distance Vector routing protocol c) Token Bucket algorithm

TASK -4

Develop a program to implement the following:

a) DES algorithm b) RSA algorithm

Task-5

a). Configure network devices, such as hubs and switches within a network topology using Packet Tracer software.

b). Construct a single LAN and understand the concepts and operation of ARP.

Task-6

a). Configure and implementation of a Switch within a Network using Packet Tracer.

b). Configure and deploy a router to enable inter-network communication using Packet Tracer.

Task-7

a). Configure network topology and implement static routing using Packet Tracer Software.

b). Configure network topology and implement dynamic routing protocol such as RIP, OSPF etc. using Packet Tracer

Task-8

a). Configure network topology to implement VLANs with using Packet Tracer software.

b). Learn and implement basic commands of Computer network like PING, traceroute, nslookup etc.

Task-9
a). Configure and examine Network Address Translation (NAT)
b). Configure PAT and interpret NAT table entries for multiple internal hosts.

Task-10
a) Configure DHCP Server in the Network using packet tracer software.
b) Configure a remote login using SSH and Telnet.

Task-11
a) Establishing a Web Server Connection Using the PC's Web Browser
b) Install Wireshark and view
i). Network Traffic. ii). Wired and wireless NIC information. iii). Examine Ethernet Frames

Task-12
a). Adding IoT devices to Smart Homes using Packet Tracer
b). Connect and Monitor IoT Devices using Packet Tracer

TEXT BOOKS:

1. Packet Tracer Network Simulator, Jesin A, O'REILLY, Packt Publishing
2. Introduction to Networks Companion Guide (CCNAv7), Cisco Press, Cisco Networking Academy

REFERENCE BOOKS:

1. Routing and switching Essentials Companion guide, Cisco Press, Cisco Networking Academy.
2. CISCO PACKET TRACER LABS: Best practice of configuring or troubleshooting Network Kindle Edition, Mulayam Singh.

S.No	Task No	Name of the Task	Page No.
1	Task 1a	Implement the following Data Link Layer framing methods a)Bit Stuffing	1
2	Task 1b	Implement the following Data Link Layer framing methods b)Character Stuffing	3
3	Task 1c	Implement the following Data Link Layer framing methods c)Character count	5
4	Task 2a	Implement the data link layer framing method Simplex protocol	7
5	Task 2b	Implement the data link layer framing method Sliding window protocol(GobackN)	9
6	Task 2c	Implement the data link layer framing method Sliding window protocol(selective repeat)	11
7	Task 3a	Design a program to implement the following: Shortest Path routing protocol	14
8	Task 3b	Design a program to implement the following: Distance Vector routing protocol	16
9	Task 3c	Design a program to implement the following: Token Bucket algorithm.	18
10	Task 4a	Develop a program to implement the following: DES algorithm	20
11	Task 4b	Develop a program to implement the following: RSA algorithm	28
12	Task 5a	Configure network devices, such as hubs and switches within a network topology usingPacket Tracer software.	30
13	Task 5b	Construct a single LAN and understand the concepts and operation of ARP.	31

S.No	Task No	Name of the Task	Page No.
14	Task 6a	Configure and implementation of a Switch within a Network using Packet Tracer.	34
15	Task 6b	Learn and implement basic commands of Computer network like PING, traceroute,nslookup etc.	38
16	Task 7a	Configure and implementation of a Router within a Network using Packet Tracer.	43
17	Task 7b	Configure and examine Network Address Translation (NAT)	50
18	Task 8a	Configure network topology to implement VLANs with using Packet Tracer software	55
19	Task 8b	Configure network topology and implement static routing using Packet Tracer Software.	62
20	Task 9	Configure network topology and implement dynamic routing protocol such as RIP, OSPF using Packet Tracer.	65
21	Task 10a	Configure DHCP Server in the Network using packet tracer software.	71
22	Task 10b	Configure a remote login using SSH and Telnet.	74
23	Task 11a	Establishing a Web Server Connection Using the PC's Web Browser	76
24	Task 11b	View wired and wireless NIC information.	83
25	Task 11c	Install Wireshark and view i). Network Traffic. ii).Examine Ethernet Frames	85
26	Task 12	a) Adding IoT devices to Smart Homes using Packet Tracer. b) Connect and Monitor IoT Devices using Packet Tracer.	86

Task 1: Implement the following Data Link Layer framing methods.

1a). Bit Stuffing

Aim: Implement the data link layer framing method.

Description:

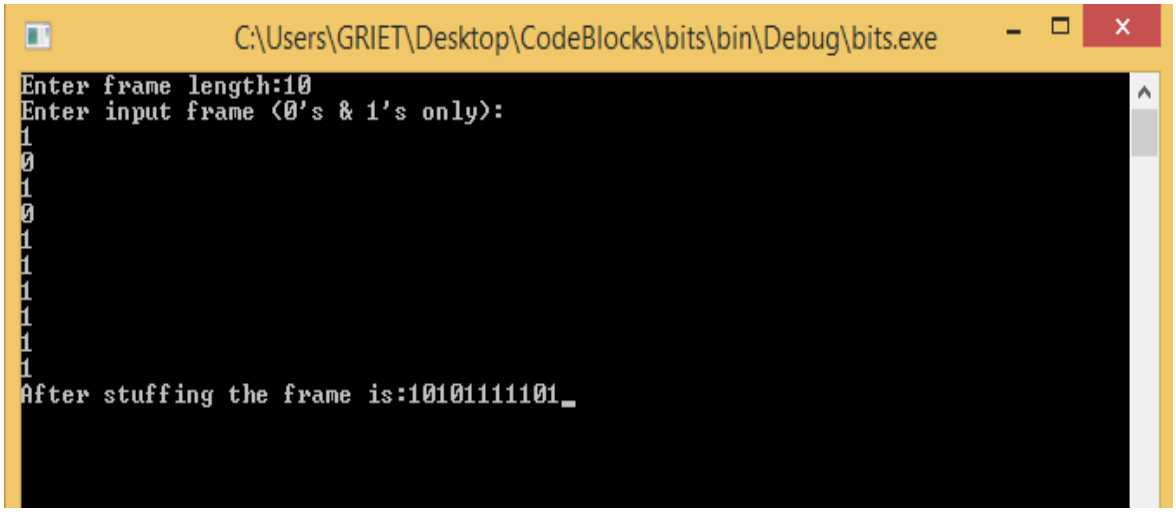
The new technique allows data frames to contain an arbitrary number if bits and allows character codes with an arbitrary no of bits per character. Each frame begins and ends with special bit pattern, 01111110, called a flag byte. Whenever the sender' s data link layer encounters five consecutive ones in the data, it automatically stuffs a 0 bit into the out going bit stream.

Program:

```
#include <stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
int a[20],b[30],i,j,k,count,n;
printf("Enter frame length:");
scanf("%d",&n);
printf("Enter input frame (0's & 1's only):"); for(i=0;i<n;i++)
scanf("%d",&a[i]);
 i=0;
count=1; j=0;
while(i<n)
{
if(a[i]==1)
{
b[j]=a[i];
for(k=i+1;a[k]==1 && k<n &&count<5;k++)
{
j++;
b[j]=a[k]; count++; if(count==5)
{
j++; b[j]=0;
}
i=k;
}
}
else
```

```
{
b[j]=a[i];
} i++; j++;
}
printf("After stuffing the frame is:");
for(i=0;i<j;i++)
printf("%d",b[i]);
getch();
}
```

Output:



1 b) Character Stuffing

Aim: Implement the data link layer framing method character stuffing

Description:

The framing method gets around the problem of resynchronization after an error by having each frame start with the ASCII character sequence DLE STX and the sequence DLE ETX. If the destination ever losses the track of the frame boundaries all it has to do is look for DLE STX or DLE ETX characters to figure out. The data link layer on the receiving g end removes the D LE before the data are given to the network layer. This technique is called character stuffing.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include<string.h>
#include<process.h>
void main()
{
int i=0,j=0,n,pos;char a[20],b[50],ch;
printf("enter string\n");
scanf("%s",&a);
n=strlen(a);
printf("enter position\n");
scanf("%d",&pos);
if(pos>n)
{
printf("invalid position, Enter again :");
scanf("%d",&pos);
}
printf("enter the character\n");
ch=getche();
b[0]='d';
b[1]='l';
b[2]='e';
b[3]='s';
b[4]='t';
b[5]='x'; j=6;
while(i<n)
{
if(i==pos-1)
{
b[j]='d';
b[j+1]='l';
b[j+2]='e';
b[j+3]=ch; b[j+4]='d';
```

```
b[j+5]='l';
b[j+6]='e';
j=j+7;
}
if(a[i]=='d' && a[i+1]=='l' && a[i+2]=='e')
{

b[j]='d';
b[j+1]='l';
b[j+2]='e';
j=j+3;
}
b[j]=a[i]; i++;
j++;
}
b[j]='d';
b[j+1]='l';
b[j+2]='e';
b[j+3]='e';
b[j+4]='t';
b[j+5]='x';
b[j+6]='\0';
printf("\nframe after stuffing:\n");
printf("%s",b);
getch();
}
```

Output:



1.c) Character Count

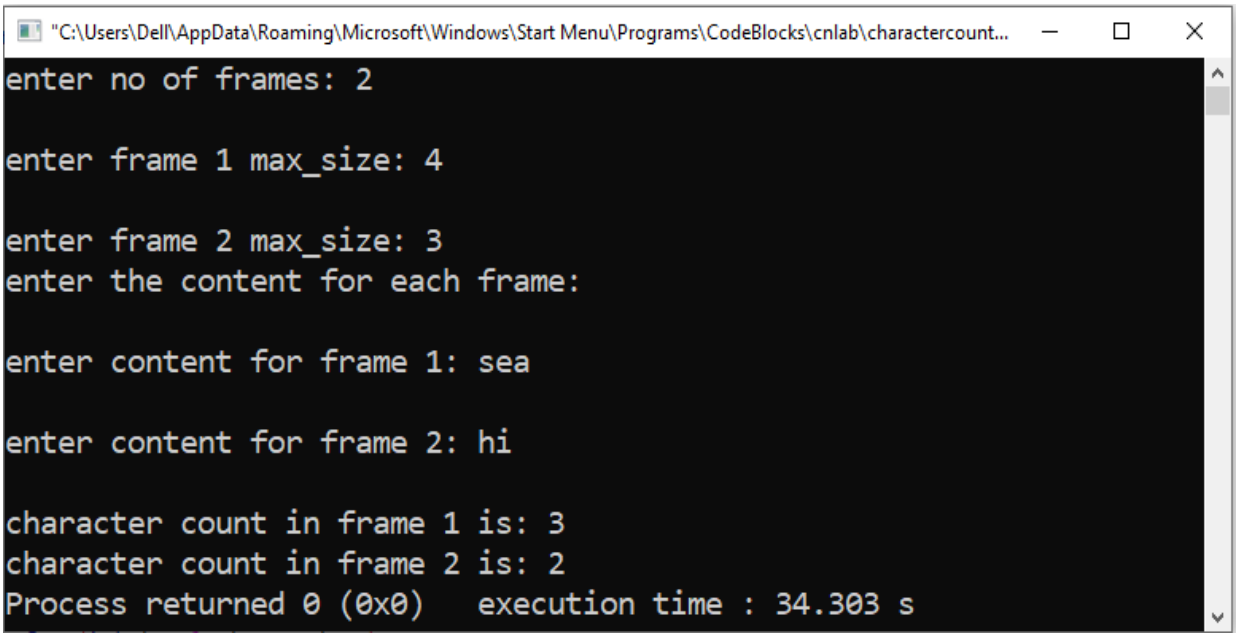
AIM: Implement the data link layer framing method.

Description:
The framing method to count the number of characters entered in each frame

Program:

```
#include <stdio.h>
#include <string.h>
int main() {
int n;
printf("enter no of frames: ");
scanf("%d", &n);
int frames[n];
for (int i = 0; i < n; i ++)
{
printf("\nenter frame %d max_size: ", i+1);
scanf("%d", &frames[i]);
}
char string [n][50];
printf("enter the content for each frame: \n");
for (int i = 0; i < n; i ++){
   printf("\nenter content for frame %d: ", i+1);
   scanf("%s", &string[i]);
   while (strlen(string[i]) > frames[i]){
      printf("entered string exceeding the maxsize!!\n enter again: ");
      scanf("%s", &string[i]); }
}
for (int i = 0; i < n; i ++)
{
   int cnt = 0;
   for (int j = 0; j < strlen(string[i]); j++)
   {
      if (string[i][j] != ' ')
      cnt++;
   }
   printf("\ncharacter count in frame %d is: %d", i+1, cnt);
}
   return 0;
}
```

Output:


"C:\Users\Dell\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\CodeBlocks\cnlab\charactercount...
enter no of frames: 2
enter frame 1 max_size: 4
enter frame 2 max_size: 3
enter the content for each frame:
enter content for frame 1: sea
enter content for frame 2: hi
character count in frame 1 is: 3
character count in frame 2 is: 2
Process returned 0 (0x0)   execution time : 34.303 s

Task-2

2.a) Implement the following Data Link layer protocols

Simplex protocol (stop and wait protocol)

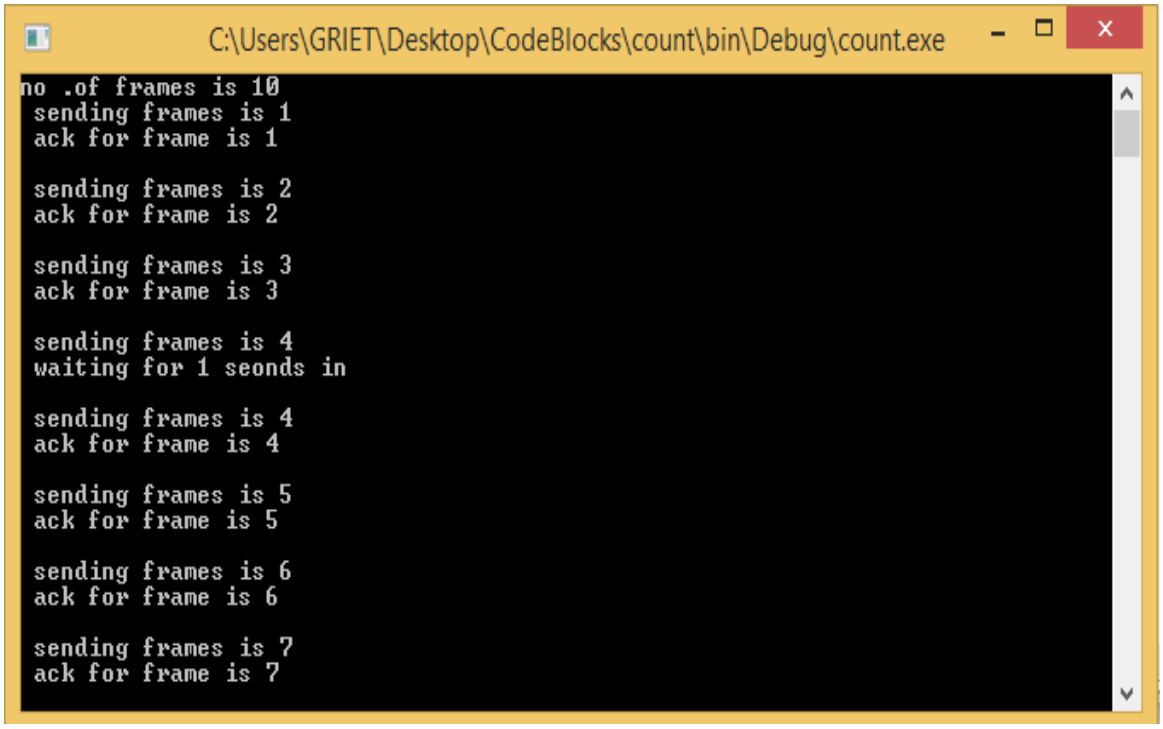
Aim: Implement the data link layer framing protocols

Description: Stop and wait protocol is a simple and reliable protocol for flow control. It is a simple data transmission method where a sender transmits one data packet at a time and then waits for an acknowledgment (ACK) from the receiver before sending the next packet

Program:

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
   int i,j,n,x,x1=10,x2;
   n=10;
   i=1;j=1;
   printf("no .of frames is %d",n);
   getch();
   while(n>0)
   {
    printf("\n sending frames is %d",i);
    x=rand()%10;
    if(x%10==0)
    {
       for(x2=1;x2<2;x2++)
       {
          printf("\n waiting for %d seonds in \n ",x2);
          sleep(x2);
        }
       printf("\n sending frames is %d ",i);
       x=rand()%10;
    }
    printf("\n ack for frame is %d \n ",j);
    n=n-1;
    i++;
    j++;
   }
   printf("\n end of stop and wait protocol \n");
   getch();
}
```

Output:


C:\Users\GRIET\Desktop\CodeBlocks\count\bin\Debug\count.exe
no .of frames is 10
sending frames is 1
ack for frame is 1
sending frames is 2
ack for frame is 2
sending frames is 3
ack for frame is 3
sending frames is 4
waiting for 1 seonds in
sending frames is 4
ack for frame is 4
sending frames is 5
ack for frame is 5
sending frames is 6
ack for frame is 6
sending frames is 7
ack for frame is 7

2.b) Sliding Window protocol (Go Back N)

Aim: Implement the data link layer framing method.

Description: Go-Back-N is a data link layer protocol that uses a sliding window method. In this, if any frame is corrupted or lost, all subsequent frames have to be sent again.
The size of the sender window is N in this protocol. For example, Go-Back-8, the size of the sender window, will be 8. The receiver window size is always 1.
If the receiver receives a corrupted frame, it cancels it. The receiver does not accept a corrupted frame. When the timer expires, the sender sends the correct frame again.

Program:

```
#include <stdio.h>

int main() {
int total_frames, windowsize, sent = 0, ack = 0, i;
int frame_start = 0; // Start of the window

printf("Enter the total number of frames to be sent: ");
scanf("%d", &total_frames);

printf("Enter the window size: ");
scanf("%d", &windowsize);

if (windowsize <= 0 || total_frames <= 0 || windowsize > total_frames) {
   printf("Invalid window size or number of frames.\n");
   return 1;
}
while (sent < total_frames) {
   // Transmit frames within the window
   for (i = 0; i < windowsize && (frame_start + i) < total_frames; i++) {
      printf("Frame %d has been transmitted.\n", frame_start + i);
   }

   // Prompt for the last acknowledgment received
   printf("\nPlease enter the last Acknowledgement received (0 to %d): ", frame_start + i - 1);
   scanf("%d", &ack);

   if (ack < frame_start || ack > frame_start + i - 1) {
      if (ack >= total_frames) {
         printf("All frames have been transmitted and acknowledged.\n");
         break; // Exit the loop if the acknowledgment is greater than or equal to the total number
                  of frames
      }
      printf("Invalid acknowledgment. Please enter a valid acknowledgment within the
      transmitted window.\n");
```

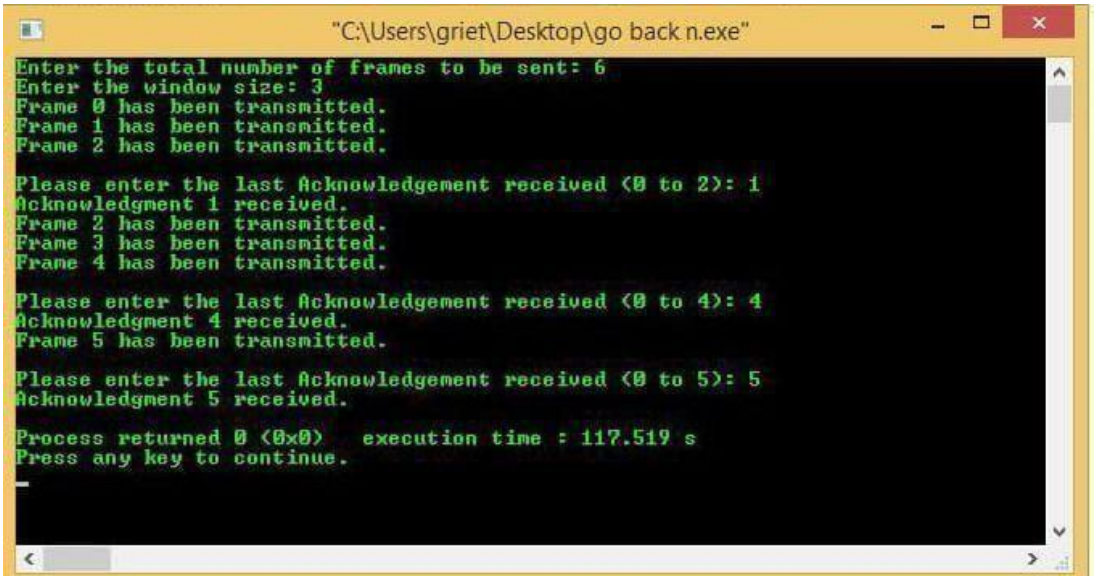
```
        } else {
            printf("Acknowledgment  %d received.\n",  ack);

            // Move the window to start after the last acknowledged frame
            frame_start = ack + 1;

            // Update the sent variable to ensure frames are transmitted correctly
            sent = frame_start;
        }
    }

    return 0;
}
```

output:



2.c) Sliding window protocol (selective repeat)

Aim: Implement the data link layer framing method.

Description: Selective Repeat ARQ is also known as the Selective Repeat Automatic Repeat Request. It is a data link layer protocol that uses a sliding window method. The Go-back-N ARQ protocol works well if it has fewer errors. But if there is a lot of error in the frame, lots of bandwidth loss in sending the frames again. So, we use the Selective Repeat ARQ protocol. In this protocol, the size of the sender window is always equal to the size of the receiver window.

Program:

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_FRAMES 10

void transmit_frames(int start, int end) {
    for (int i = start; i < end; i++) {
        printf("Frame %d has been transmitted.\n", i);
    }
}

int main() {
    int total_frames, window_size, i;
    bool ack_received[MAX_FRAMES] = {false};
    int sent_start = 0;
    int ack;

    // Input for total frames and window size
    printf("Enter the total number of frames to be sent (max %d): ", MAX_FRAMES);
    scanf("%d", &total_frames);

    printf("Enter the window size: ");
    scanf("%d", &window_size);

    if (window_size <= 0 || total_frames <= 0 || window_size > total_frames) {
        printf("Invalid window size or number of frames.\n");
        return 1;
    }

    while (sent_start < total_frames) {
        // Determine the end of the current window
        int end = sent_start + window_size;
        if (end > total_frames) end = total_frames;

        transmit_frames(sent_start, end);
```

```
        // Prompt for acknowledgments
        printf("\nPlease enter the acknowledgments received (enter -1 to finish):\n");
        for (i = sent_start; i < end; i++) {
            printf("Enter acknowledgment for frame %d: ", i);
            scanf("%d", &ack);
            if (ack == -1) {
                printf("No acknowledgment for frame %d. Will retransmit.\n", i);
            } else if (ack >= sent_start && ack < end) {
                ack_received[ack] = true;
                printf("Acknowledgment %d received.\n", ack);
            } else {
                printf("Invalid acknowledgment. It should be within the transmitted window.\n");
            }
        }

        // Retransmit only the frames that need it
        printf("\nRetransmitting unacknowledged frames...\n");
        for (i = sent_start; i < end; i++) {
            if (!ack_received[i]) {
                printf("Retransmitting frame %d.\n", i);
                transmit_frames(i, i + 1);
            }
        }

        // Move the window forward

        while (sent_start < total_frames && ack_received[sent_start]) {
            sent_start++;
        }

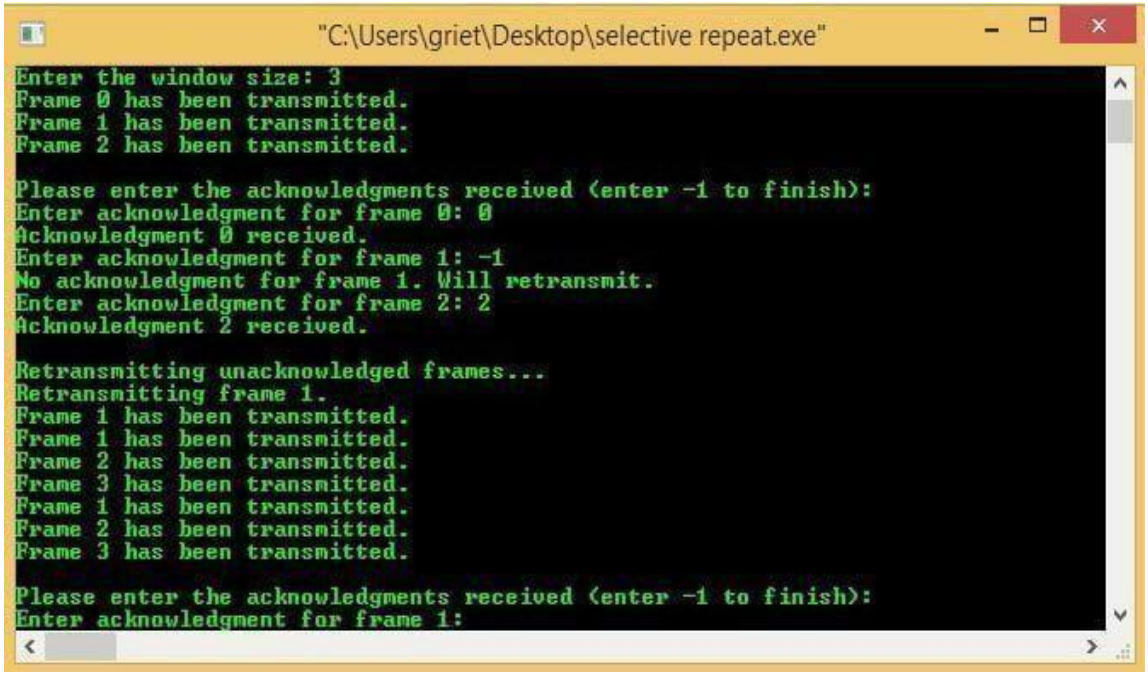
        // Adjust window end to include new frames that come after the last acknowledged frame
        int new_end = sent_start + window_size;
        if (new_end > total_frames) new_end = total_frames;

        // Transmit new frames if there are any
        if (sent_start < total_frames && new_end > sent_start) {
            transmit_frames(sent_start, new_end);
        }
    }
    printf("All frames have been transmitted and acknowledged.\n");

    return 0

}
```

output:


"C:\Users\griet\Desktop\selective repeat.exe"
Enter the window size: 3
Frame 0 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.
Please enter the acknowledgments received (enter -1 to finish):
Enter acknowledgment for frame 0: 0
Acknowledgment 0 received.
Enter acknowledgment for frame 1: -1
No acknowledgment for frame 1. Will retransmit.
Enter acknowledgment for frame 2: 2
Acknowledgment 2 received.
Retransmitting unacknowledged frames...
Retransmitting frame 1.
Frame 1 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.
Frame 3 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.
Frame 3 has been transmitted.
Please enter the acknowledgments received (enter -1 to finish):
Enter acknowledgment for frame 1:

Task 3

3a) Develop a program to implement Shortest Path routing protocol

Aim: Implement the shortest path routing protocol.

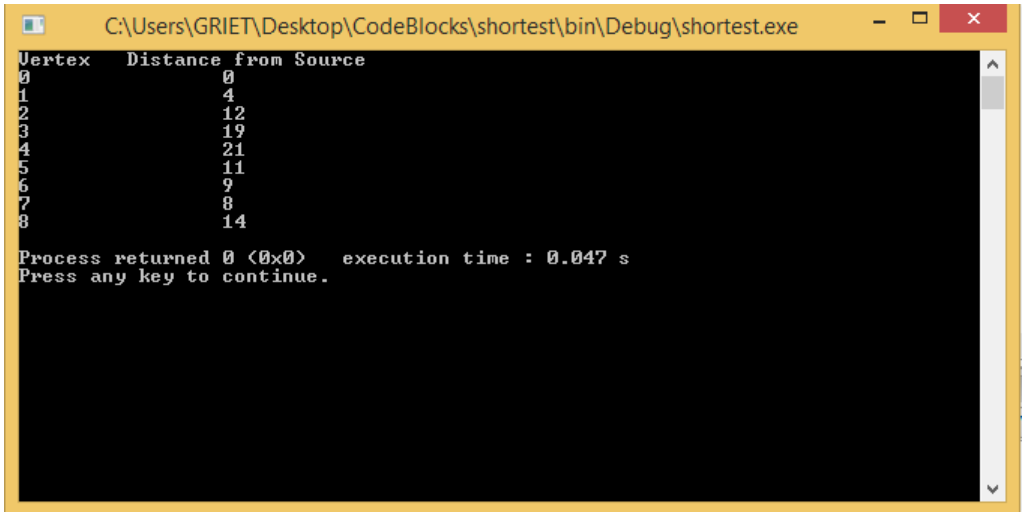
Description: Dijkstra's algorithm is a computer algorithm that finds the shortest path between a starting node and all other nodes in a graph. It's used to solve single-source shortest path problems.

Program:

```
#include  <stdio.h>
#include <stdlib.h>
#define V 9
int minDistance(int dist[], int sptSet[])
{
   int min = INT_MAX, min_index;
   int v;
   for (v = 0; v < V; v++)
      if (sptSet[v] == 0 && dist[v] <= min)
         min = dist[v], min_index = v;
   return min_index;
}
void printSolution(int dist[], int n) {
   printf("Vertex Distance from Source\n");
   int i;
   for (i = 0; i < V; i++)
      printf("%d \t\t %d\n", i, dist[i]);
}
void dijkstra(int graph[V][V], int src) {
   int dist[V];
   int sptSet[V];
   int i, count, v;
   for (i = 0; i < V; i++)
      dist[i] = INT_MAX, sptSet[i] = 0;
   dist[src] = 0;
   for (count = 0; count < V - 1; count++) {
      int u = minDistance(dist, sptSet);
      sptSet[u] = 1;
      for (v = 0; v < V; v++)
         if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX && dist[u]
               + graph[u][v] < dist[v])
            dist[v] = dist[u] + graph[u][v];
   }
   printSolution(dist, V);
}
int main() {
```

```
    int graph[V][V] =  {{0, 4, 0, 0, 0, 0, 0, 8, 0},
                        {4, 0, 8, 0, 0, 0, 0, 11, 0},
                        {0, 8, 0, 7, 0, 4, 0, 0, 2},
                        {0, 0, 7, 0, 9, 14, 0, 0, 0},
                        {0, 0, 0, 9, 0, 10, 0, 0, 0},
                        {0, 0, 4, 0, 10, 0, 2, 0, 0},
                        {0, 0, 0, 14, 0, 2, 0, 1, 6},
                        {8, 11, 0, 0, 0, 0, 1, 0, 7},
                        {0, 0, 2, 0, 0, 0, 6, 7, 0}
                       };
    dijkstra(graph,  0);
    return 0;
}
```

Output:



3b) Write a program to implement Distance Vector routing protocol

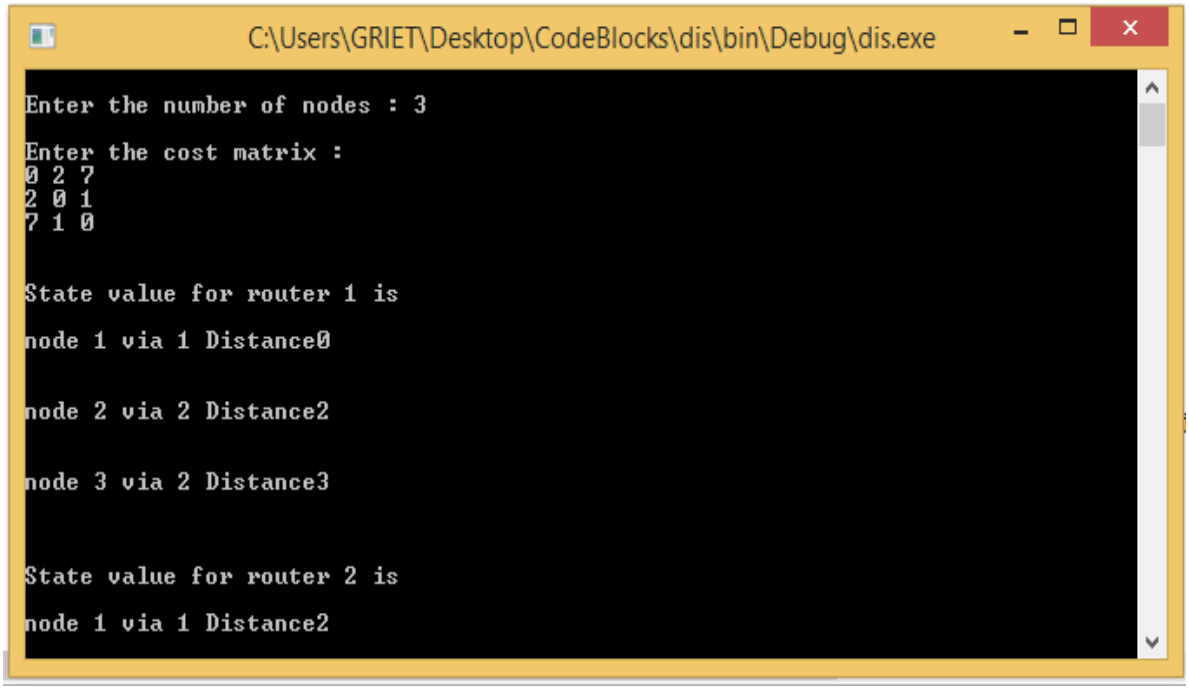
Aim: Implement distance vector routing protocol.

Description: The distance vector routing algorithm is one of the most commonly used routing algorithms. It is a distributed algorithm, meaning that it is run on each router in the network. The algorithm works by each router sending updates to its neighbours about the best path to each destination.

Program:

```
#include <stdio.h>
#include <stdlib.h>
struct node {
unsigned dist[20];
unsigned from[20];
}rt[10];
int main()
{
        int dmat[20][20],n,i,j,k,count=0;
        printf("\nEnter the number of nodes : ");
        scanf("%d",&n);
        printf("\nEnter the cost matrix :\n");
        for(i=0;i<n;i++)
                for(j=0;j<n;j++ )
                {
                        scanf("%d",&dmat[i][j]);
                        dmat[i][i]=0;
                        rt[i].dist[j]=dmat[i][j];
                        rt[i].from[j]=j; }
                         do { count=0;
                        for(i=0;i<n;i++)
                        for(j=0;j<n;j++)
                        for(k=0;k<n;k++)
                                if(rt[i].dist[j]>dmat[i][k]+rt[k].dist[j])  {
                                        rt[i].dist[j]=rt[i].dist[k]+rt[k].dist[j];
                                        rt[i].from[j]=k;
                                        count++;
                                 }
                }while(count!=0);
                for(i=0;i<n;i++) {
                        printf("\n\nState value for router %d is \n",i+1);
                        for(j=0;j<n;j++)  {
                                printf("\t\nnode %d via %d
                                Distance%d",j+1,rt[i].from[j]+1,rt[i].dist[j]);
                                printf("\n\n");
                }
  } }
```

Output:


C:\Users\GRIET\Desktop\CodeBlocks\dis\bin\Debug\dis.exe
Enter the number of nodes : 3
Enter the cost matrix :
0 2 7
2 0 1
7 1 0
State value for router 1 is
node 1 via 1 Distance0
node 2 via 2 Distance2
node 3 via 2 Distance3
State value for router 2 is
node 1 via 1 Distance2

.

1. C) Write a program to implement Token Bucket

algorithm Aim: Implement token bucket algorithm.

Description: Token bucket algorithm is one of the techniques for congestion control algorithms. When too many packets are present in the network it causes packet delay and loss of packet which degrades the performance of the system.

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int bucket_size = 10;            // The maximum size of the bucket (token capacity)
    int tokens = 0;                  // Current number of tokens in the bucket
    int token_rate = 3;              // Rate at which tokens are added (tokens per second)
    int no_of_queries = 6;           // Number of incoming packets (queries)
    int input_pkt_size[] = {4, 6, 8, 2, 1, 5}; // Incoming packet sizes
    int required_tokens = 4;         // Tokens required to process each packet
    int queue = 0;                   // Queue size for holding packets if tokens are insufficient
    int time_interval = 1;           // Time interval for adding tokens (simulated as 1 second)

    for (int i = 0; i < no_of_queries; i++)
   {
        // Add tokens at a constant rate
        tokens += token_rate * time_interval;
        printf("no of tokens added:%d",tokens);
        if (tokens > bucket_size)
       {
            tokens = bucket_size;  // Ensure tokens do not exceed bucket capacity
        }

        printf("\nTime: %d seconds\n", i+1);
        printf("Incoming packet size: %d\n", input_pkt_size[i]);

        // If not enough tokens for current packet, queue it
        if (tokens < required_tokens)
        {
            printf("Not enough tokens! Queuing packet of size %d.\n", input_pkt_size[i]);
            queue += input_pkt_size[i];  // Add to the queue
        }
        else
        {
```

```
            // Process the packet if enough tokens are available
            tokens -= required_tokens;
            printf("Processed packet of size %d. Tokens remaining: %d\n", input_pkt_size[i], tokens);

            // If there is a queue, process the queued packets when tokens become available
            while (queue > 0 && tokens >= required_tokens)
            {
                printf("Processing queued packet of size %d\n", queue);
                tokens -= required_tokens;
                queue = 0;  // Clear the queue after processing
            }
        }

        // Print current token and queue status
        printf("Tokens in bucket: %d\n", tokens);
        printf("Packets in queue: %d\n", queue);
    }

    return 0;
}
```

Output:



TASK-4

2. a) Develop a program to implement DES algorithm

Aim: Implement the Data Encryption Standard algorithm.

Description: The DES (Data Encryption Standard) algorithm is a symmetric-key block cipher created in the early 1970s by an IBM team and adopted by the National Institute of Standards and Technology (NIST). The algorithm takes the plain text in 64-bit blocks and converts them into ciphertext using 48-bit keys.

Program:

```
#include <stdio.h>
#include <stdlib.h>
int Original_key [64] = { // you can change key if required
0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0,
0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1,
1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0,
1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1
};
int Permutated_Choice1[56] = {
  57, 49, 41, 33, 25, 17, 9,
   1, 58, 50, 42, 34, 26, 18,
  10,  2, 59, 51, 43, 35, 27,
  19, 11,  3, 60, 52, 44, 36,
  63, 55, 47, 39, 31, 23, 15,
   7, 62, 54, 46, 38, 30, 22,
  14,  6, 61, 53, 45, 37, 29,
  21, 13,  5, 28, 20, 12,  4
};

int Permutated_Choice2[48] = {
  14, 17, 11, 24, 1, 5,
   3, 28, 15,  6, 21, 10,
  23, 19, 12,  4, 26,  8,
  16,  7, 27, 20, 13,  2,
  41, 52, 31, 37, 47, 55,
  30, 40, 51, 45, 33, 48,
  44, 49, 39, 56, 34, 53,
  46, 42, 50, 36, 29, 32
};

int Iintial_Permutation [64] = {
  58, 50, 42, 34, 26, 18, 10, 2,
  60, 52, 44, 36, 28, 20, 12, 4,
  62, 54, 46, 38, 30, 22, 14, 6,
  64, 56, 48, 40, 32, 24, 16, 8,
  57, 49, 41, 33, 25, 17,  9, 1,
  59, 51, 43, 35, 27, 19, 11, 3,
  61, 53, 45, 37, 29, 21, 13, 5,
```

```
  63, 55, 47, 39, 31, 23, 15, 7
};

int Final_Permutation[] =
{
  40, 8, 48, 16, 56, 24, 64, 32,
  39, 7, 47, 15, 55, 23, 63, 31,
  38, 6, 46, 14, 54, 22, 62, 30,
  37, 5, 45, 13, 53, 21, 61, 29,
  36, 4, 44, 12, 52, 20, 60, 28,
  35, 3, 43, 11, 51, 19, 59, 27,
  34, 2, 42, 10, 50, 18, 58, 26,
  33, 1, 41,  9, 49, 17, 57, 25
};
int P[] =
{
  16,  7, 20, 21,
  29, 12, 28, 17,
   1, 15, 23, 26,
   5, 18, 31, 10,
   2,  8, 24, 14,
  32, 27,  3,  9,
  19, 13, 30,  6,
  22, 11,  4, 25
};

int E[] =
{
  32,  1,  2,  3,  4,  5,
   4,  5,  6,  7,  8,  9,
   8,  9, 10, 11, 12, 13,
  12, 13, 14, 15, 16, 17,
  16, 17, 18, 19, 20, 21,
  20, 21, 22, 23, 24, 25,
  24, 25, 26, 27, 28, 29,
  28, 29, 30, 31, 32,  1
};

int S1[4][16] =
{
14,  4, 13,  1,  2, 15, 11,  8,  3, 10,  6, 12,  5,  9,  0,  7,
0, 15,  7,  4, 14,  2, 13,  1, 10,  6, 12, 11,  9,  5,  3,  8,
4,  1, 14,  8, 13,  6,  2, 11, 15, 12,  9,  7,  3, 10,  5,  0,
15, 12,  8,  2,  4,  9,  1,  7,  5, 11,   3, 14, 10,  0,  6, 13
};

int S2[4][16] =
{
15,  1,  8, 14,  6, 11,  3,  4,  9,  7,   2, 13, 12,  0,  5, 10,
3, 13,  4,  7, 15,  2,  8, 14, 12,  0,   1, 10,  6,  9, 11,  5,
```

```
0, 14,  7, 11, 10,  4, 13,   1, 5,  8, 12,  6,  9,  3,  2, 15,
13,  8, 10,  1,  3, 15,  4,   2, 11, 6,  7, 12,  0,  5, 14,  9
};

int S3[4][16] =
{
10,  0,  9, 14,  6,  3, 15,  5,  1, 13, 12,  7, 11,  4,  2,  8,
13,  7,  0,  9,  3,  4,  6, 10,  2,  8,  5, 14, 12, 11, 15,  1,
13,  6,  4,  9,  8, 15,  3,  0, 11,  1,  2, 12,  5, 10, 14,  7,
1, 10, 13,  0,  6,  9,  8,  7,  4, 15, 14,  3, 11,  5,  2, 12
};

int S4[4][16] =
{
7, 13, 14,  3,  0,  6,  9, 10,  1,  2,  8,  5, 11, 12,  4, 15,
13,  8, 11,  5,  6, 15,  0,  3,  4,  7,  2, 12,  1, 10, 14,  9,
10,  6,  9,  0, 12, 11,  7, 13, 15,  1,  3, 14,  5,  2,  8,  4,
3, 15,  0,  6, 10,  1, 13,  8,  9,  4,  5, 11, 12,  7,  2, 14
};

int S5[4][16] =
{
2, 12,  4,  1,  7, 10, 11,  6,  8,  5,  3, 15, 13,  0, 14,  9,
14, 11,  2, 12,  4,  7, 13,  1,  5,  0, 15, 10,  3,  9,  8,  6,
4,  2,  1, 11, 10, 13,  7,  8, 15,  9, 12,  5,  6,  3,  0, 14,
11,  8, 12,  7,  1, 14,  2, 13,  6, 15,  0,  9, 10,  4,  5,  3
};

int S6[4][16] =
{
12,  1, 10, 15,  9,  2,  6,  8,  0, 13,  3,  4, 14,  7,  5, 11,
10, 15,  4,  2,  7, 12,  9,  5,  6,  1, 13, 14,  0, 11,  3,  8,
9, 14, 15,  5,  2,  8, 12,  3,  7,  0,  4, 10,  1, 13, 11,  6,
4,  3,  2, 12,  9,  5, 15, 10, 11, 14,  1,  7,  6,  0,  8, 13
};

int S7[4][16]=
{
4, 11,  2, 14, 15,  0,  8, 13,  3, 12,  9,  7,  5, 10,  6,  1,
13,  0, 11,  7,  4,  9,  1, 10, 14,  3,  5, 12,  2, 15,  8,  6,
1,  4, 11, 13, 12,  3,  7, 14, 10, 15,  6,  8,  0,  5,  9,  2,
6, 11, 13,  8,  1,  4, 10,  7,  9,  5,  0, 15, 14,  2,  3, 12
};

int S8[4][16]=
{
13,  2,  8,  4,  6, 15, 11,  1, 10,  9,  3, 14,  5,  0, 12,  7,
1, 15, 13,  8, 10,  3,  7,  4, 12,  5,  6, 11,  0, 14,  9,  2,
7, 11,  4,  1,  9, 12, 14,  2,  0,  6, 10, 13, 15,  3,  5,  8,
2,  1, 14,  7,  4, 10,  8, 13, 15, 12,  9,  0,  3,  5,  6, 11
```

```
};

int shifts_for_each_round[16] = { 1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1 };
int _56bit_key[56];
int _48bit_key[17][48];
int text_to_bits[99999], bits_size=0;
int Left32[17][32], Right32[17][32];
int EXPtext[48];
int XORtext[48];
int X[8][6];
int X2[32];
int R[32];
int chiper_text[64];
int encrypted_text[64];

int XOR(int a, int b) {
return (a ^ b);
}

void Dec_to_Binary(int n)
{
   int binaryNum[1000];
   int i = 0;
   while (n > 0) {
      binaryNum[i] = n % 2;
      n = n / 2;
      i++;
   }
   for (int j = i - 1; j >= 0; j--) {
text_to_bits[bits_size++] = binaryNum[j];
}
}
int F1(int i)
{
int r, c, b[6];
for (int j = 0; j < 6; j++)
b[j] = X[i][j];

r = b[0] * 2 + b[5];
c = 8 * b[1] + 4 * b[2] + 2 * b[3] + b[4];
if (i == 0)
return S1[r][c];
else if (i == 1)
return S2[r][c];
else if (i == 2)
return S3[r][c];
else if (i == 3)
return S4[r][c];
else if (i == 4)
return S5[r][c];
```

```
else if (i == 5)
return S6[r][c];
else if (i == 6)
return S7[r][c];
else if (i == 7)
return S8[r][c];
}
int PBox(int pos, int bit)
{
int i;
for (i = 0; i < 32; i++)
if (P[i] == pos + 1)
break;
R[i] = bit;
}
int ToBits(int value)
{
int k, j, m;
static int i;
if (i % 32 == 0)
i = 0;
for (j = 3; j >= 0; j--)
{
m = 1 << j;
k = value & m;
if (k == 0)
X2[3 - j + i] = '0' - 48;
else
X2[3 - j + i] = '1' - 48;
}
i = i + 4;
}
int SBox(int XORtext[])
{
int k = 0;
for (int i = 0; i < 8; i++)
for (int j = 0; j < 6; j++)
X[i][j] = XORtext[k++];

int value;
for (int i = 0; i < 8; i++)
{
value = F1(i);
ToBits(value);
}
}
void expansion_function(int pos, int bit)
{
for (int i = 0; i < 48; i++)
if (E[i] == pos + 1)
```

```
EXPtext[i] = bit;
}
void cipher(int Round, int mode)
{
for (int i = 0; i < 32; i++)
expansion_function(i, Right32[Round - 1][i]);
for (int i = 0; i < 48; i++)
{
if (mode == 0)
XORtext[i] = XOR(EXPtext[i], _48bit_key[Round][i]);
else
XORtext[i] = XOR(EXPtext[i], _48bit_key[17 - Round][i]);
}

SBox(XORtext);
for (int i = 0; i < 32; i++)
PBox(i, X2[i]);
for (int i = 0; i < 32; i++)
Right32[Round][i] = XOR(Left32[Round - 1][i], R[i]);
}
void finalPermutation(int pos, int bit)
{
int i;
for (i = 0; i < 64; i++)
if (Final_Permutation[i] == pos + 1)
break;
encrypted_text[i] = bit;
}
void Encrypt_each_64_bit (int plain_bits [])
{
int IP_result [64] , index=0;
for (int i = 0; i < 64; i++) {
IP_result [i] = plain_bits[ Iintial_Permutation[i] ];
}
for (int i = 0; i < 32; i++)
Left32[0][i] = IP_result[i];
for (int i = 32; i < 64; i++)
Right32[0][i - 32] = IP_result[i];
for (int k = 1; k < 17; k++)
{ // processing through all 16 rounds
cipher(k, 0);
for (int i = 0; i < 32; i++)
Left32[k][i] = Right32[k - 1][i]; // right part comes as it is to next round left part
}
for (int i = 0; i < 64; i++)
{ // 32bit swap as well as Final Inverse Permutation
if (i < 32)
chiper_text[i] = Right32[16][i];
else
chiper_text[i] = Left32[16][i - 32];
```

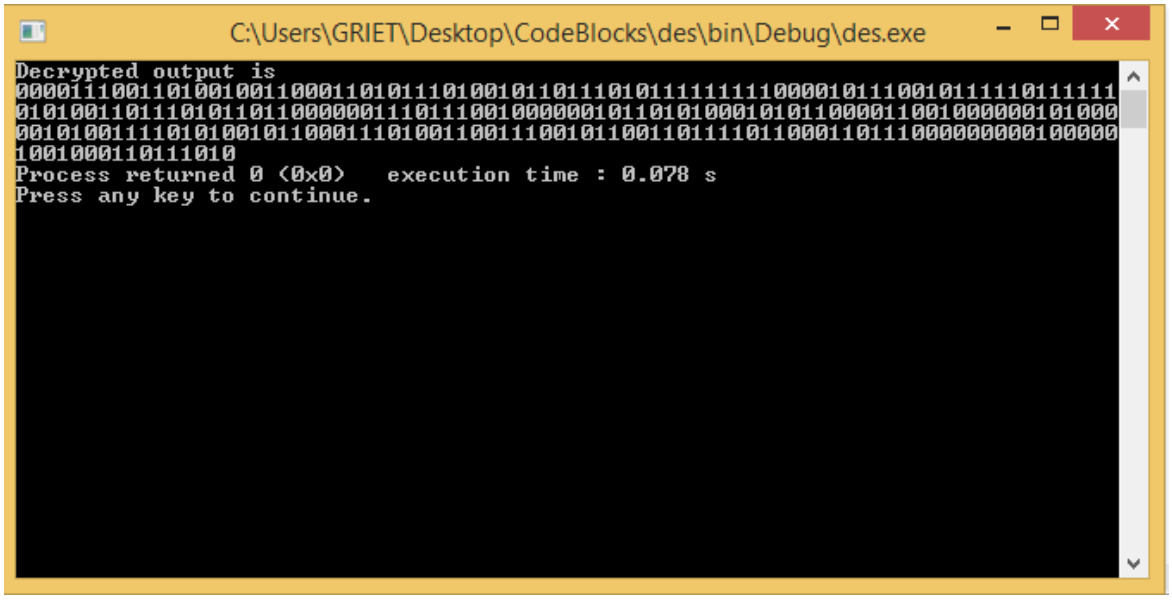
```
finalPermutation(i,  chiper_text[i]);
}
for (int i = 0; i < 64; i++)
printf("%d", encrypted_text[i]);
}
void convert_Text_to_bits(char *plain_text){
for(int i=0;plain_text[i];i++){
int asci = plain_text[i];
Dec_to_Binary(asci);
}
}
void key56to48(int round, int pos, int bit)
{
int i;
for (i = 0; i < 56; i++)
if (Permutated_Choice2[i] == pos + 1)
break;
_48bit_key[round][i]  =  bit;
}
int key64to56(int pos, int bit)
{
int i;
for (i = 0; i < 56; i++)
if (Permutated_Choice1[i] == pos + 1)
break;
_56bit_key[i] = bit;
}
void key64to48(int  key[])
{
int k, backup[17][2];
int CD[17][56];
int C[17][28], D[17][28];

for (int i = 0; i < 64; i++)
key64to56(i, key[i]);

for (int i = 0; i < 56; i++)
if (i < 28)
C[0][i] = _56bit_key[i];
else
D[0][i - 28] = _56bit_key[i];
for (int x = 1; x < 17; x++)
{
int shift = shifts_for_each_round[x - 1];
for (int i = 0; i < shift; i++)
backup[x - 1][i] = C[x - 1][i];
for (int i = 0; i < (28 - shift); i++)
C[x][i] = C[x - 1][i + shift];
k = 0;
for (int i = 28 - shift; i < 28; i++)
```

```
C[x][i] = backup[x - 1][k++];
for (int i = 0; i < shift; i++)
backup[x - 1][i] = D[x - 1][i];
for (int i = 0; i < (28 - shift); i++)
D[x][i] = D[x - 1][i + shift];
k = 0;
for (int i = 28 - shift; i < 28; i++)
D[x][i] = backup[x - 1][k++];
}
for (int j = 0; j < 17; j++)
{
for (int i = 0; i < 28; i++)
CD[j][i] = C[j][i];
for (int i = 28; i < 56; i++)
CD[j][i] = D[j][i - 28];
}
for (int j = 1; j < 17; j++)
for (int i = 0; i < 56; i++)
key56to48(j, i, CD[j][i]);
}
int main(){
char plain_text[] = "tomarrow we wiil be declaring war";
convert_Text_to_bits(plain_text);
key64to48(Original_key); // it creates all keys for all rounds
int _64bit_sets = bits_size/64;
printf("Decrypted output is\n");
for(int i=0;i<= _64bit_sets ;i++) {
Encrypt_each_64_bit  (text_to_bits + 64*i);
}
return 0;
}
```

Output:



4.b) Write a program to implement RSA algorithm

Aim: Implement the RSA asymmetric cryptographic algorithm

Description: RSA is a widely used cryptographic algorithm that was first introduced in 1977. It uses public and private key pairs to encrypt and decrypt data. Though RSA can be used in several applications, its computational complexity makes it unsuitable for encrypting large messages or files.

Program:
```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
// Function to calculate the greatest common divisor
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
long long mod_exp(long long base, long long exponent, long long modulus)
{
    long long result = 1;
    while (exponent > 0)
    {
        if (exponent % 2 == 1)
       {
            result = (result * base) % modulus;
        }
        base = (base * base) % modulus;
        exponent = exponent / 2;
    }
    return result;
}

int main() {
    long long p, q, n, phi, e, d;
    long long encrypted, decrypted;
    long long message;

    // Step 1: Ask user for two large prime numbers
    printf("Enter a large prime number p: ");
    scanf("%lld", &p);
    printf("Enter another large prime number q: ");
    scanf("%lld", &q);
```

```
    // Step 2: Calculate n = p * q
    n = p * q;

    // Step 3: Calculate the Euler's Totient Function, phi(n)
    phi = (p - 1) * (q - 1);

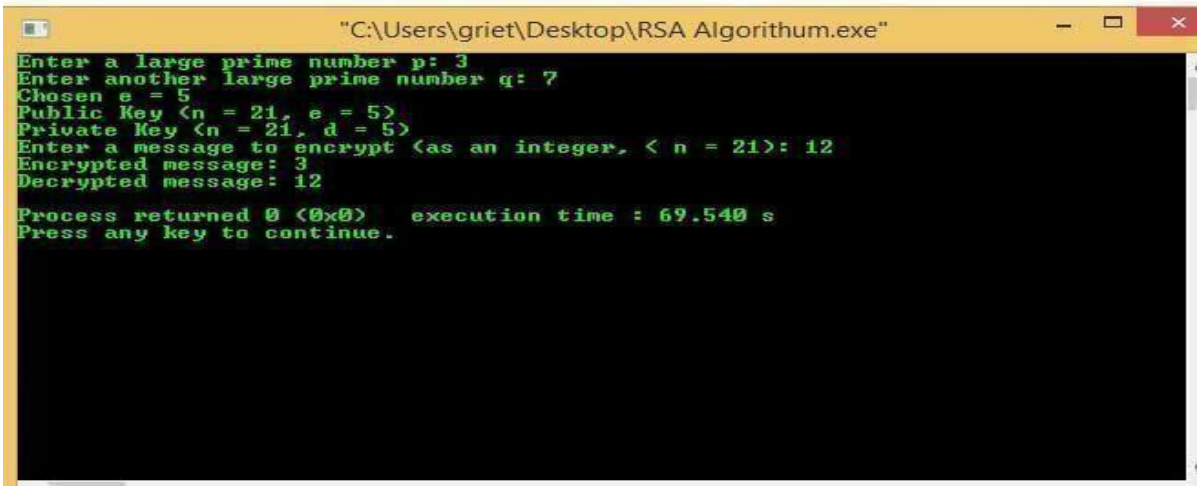
    // Step 4: Dynamically choose e such that 1 < e < phi and gcd(e, phi) = 1
    e = 2;
    while (gcd(e, phi) != 1) {
        e++;
    }
    printf("Chosen e = %lld\n", e);
    // Step 5: Compute d, the modular multiplicative inverse of e modulo phi
    // Using a brute force approach to find d such that (d * e) % phi = 1
    d = 0;
    while ((e * d) % phi != 1) {
     d++;
    }
    // Display public and private keys
    printf("Public Key (n = %lld, e = %lld)\n", n, e);
    printf("Private Key (n = %lld, d = %lld)\n", n, d);
    //Step 6: Get the message to encrypt (as an integer)
   do {
        printf("Enter a message to encrypt (as an integer, < n = %d): ", n);
        scanf("%d", &message);
    } while (message >= n);  // Ensure the message is smaller than n

    // Step 7: Encrypt the message

    encrypted = mod_exp(message, e, n);
    printf("Encrypted message: %lld\n", encrypted);

    decrypted = mod_exp(encrypted, d, n);
    printf("Decrypted message: %lld\n", decrypted);
    return 0;
}
```

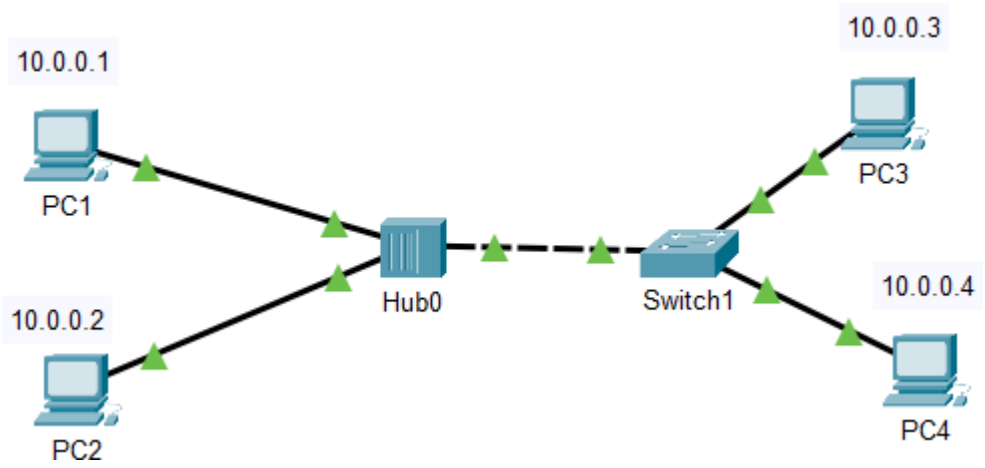
Output:



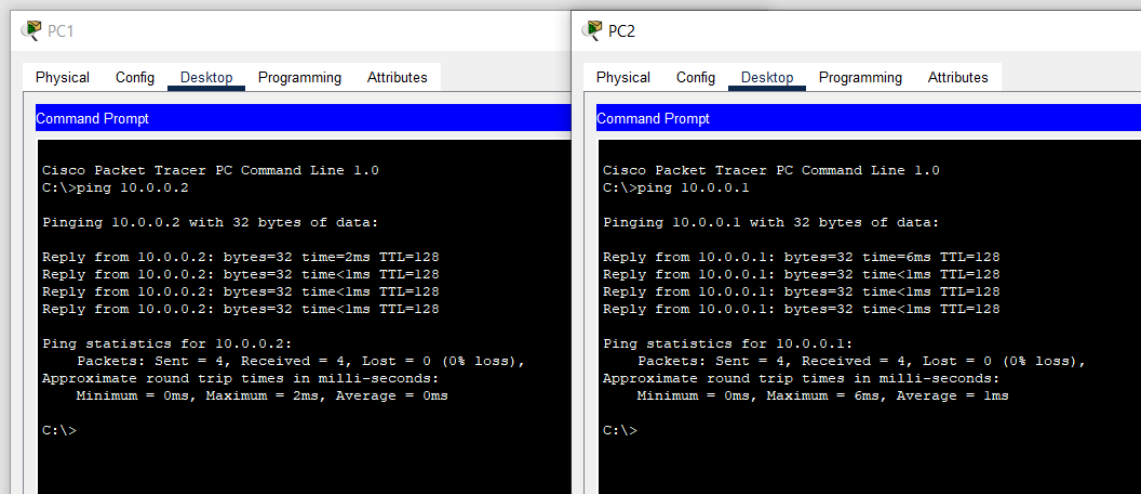
Task 5

Task 5a): Configure network devices, such as hubs and switches within a network topology using Packet Tracer software.

Aim: Configure network devices like hubs, switches within a network



Configure the network systems with the above-mentioned IP address and then ping from other systems to test the flow of packets.



b). Construct a single LAN and understand the concepts and operation of ARP.

Aim: Implement the operations of ARP

Description:

What is ARP?

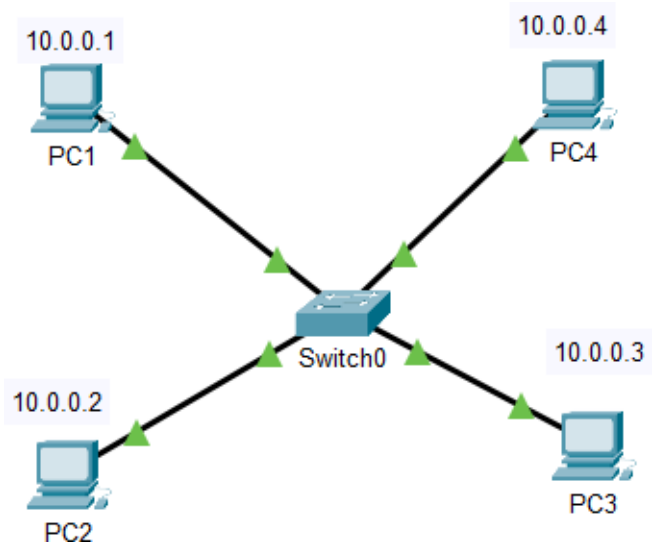
The **Address Resolution Protocol** (**ARP**) is a communication protocol used for discovering the physical address, such as a MAC address, associated with a given internet layer address, typically an IPv4 address.

How ARP Works?

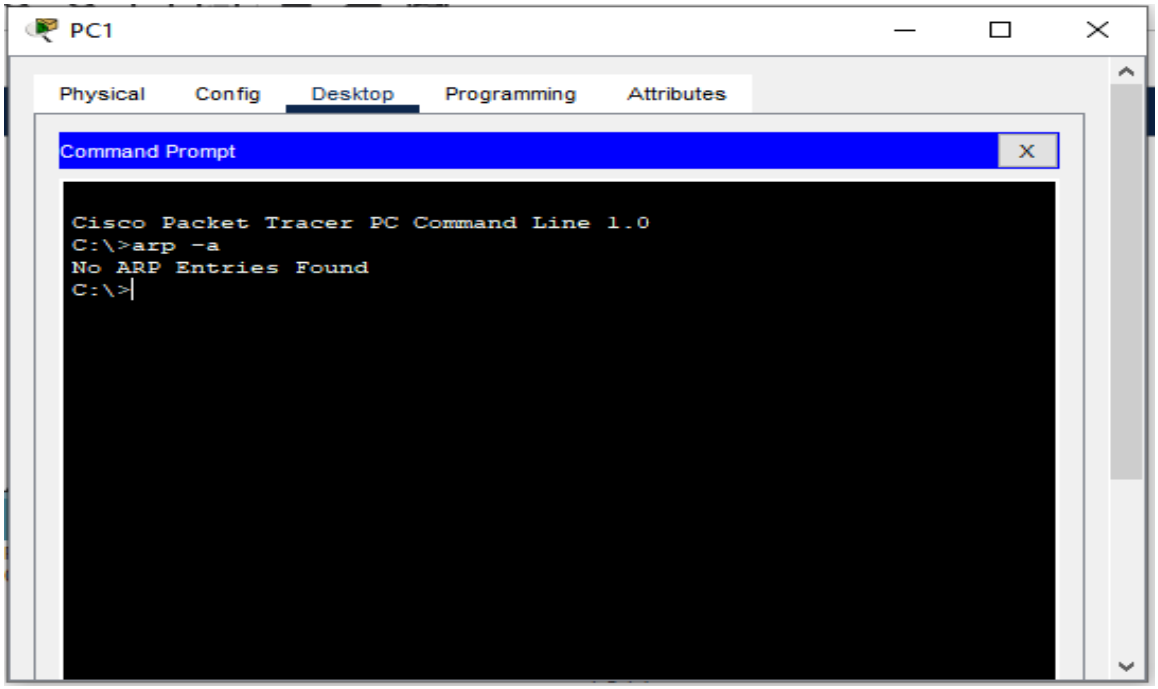
Imagine a device that wants to communicate with others over the internet. What does ARP do? It broadcast a packet to all the devices of the source network. The devices of the network peel the header of the data link layer from the **Protocol Data Unit (PDU)** called frame and transfer the packet to the network layer (layer 3 of OSI) where the network ID of the packet is validated with the destination IP's network ID of the packet and if it's equal then it responds to the source with the MAC address of the destination, else the packet reaches the gateway of the network and broadcasts packet to the devices it is connected with and validates their network ID. The above process continues till the second last network device in the path reaches the destination where it gets validated and ARP, in turn, responds with the destination MAC address.

Program:

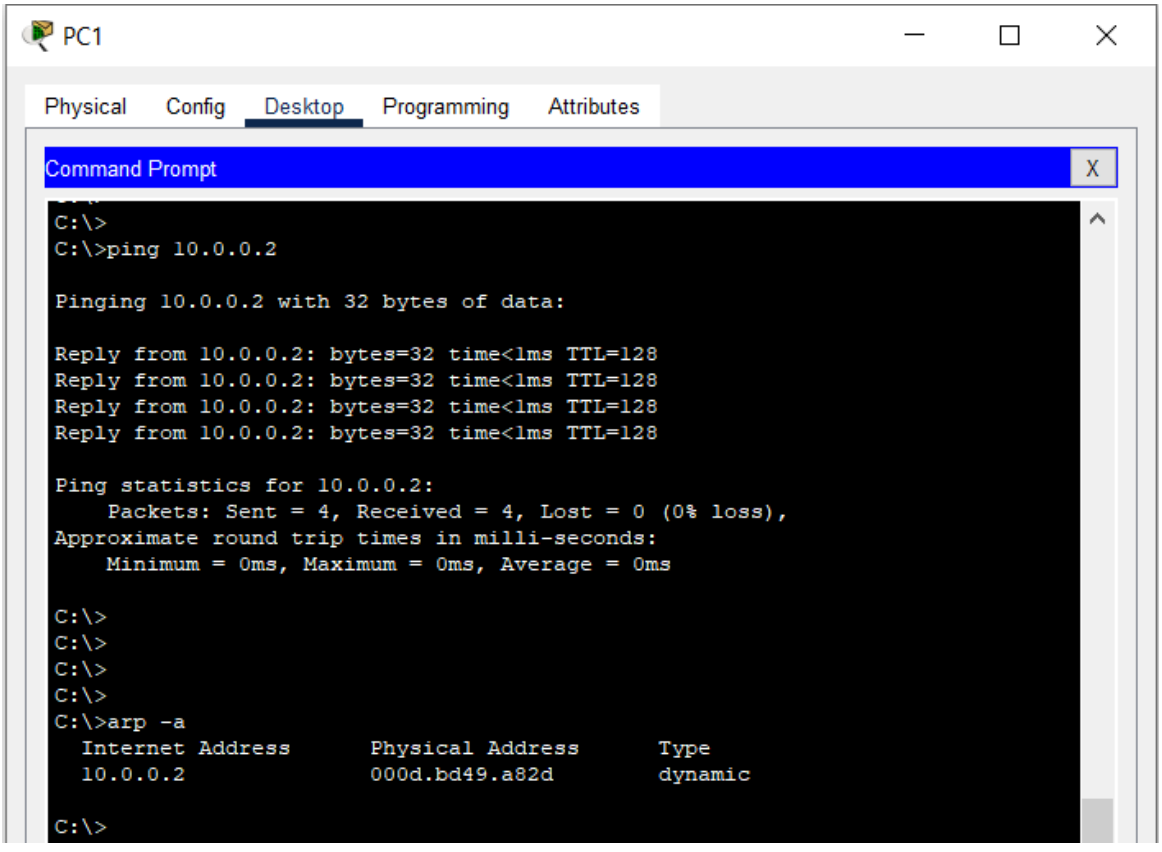
Below topology illustrates 4 PCs with IP address ranging from 10.0.0.1- 10.0.0.4



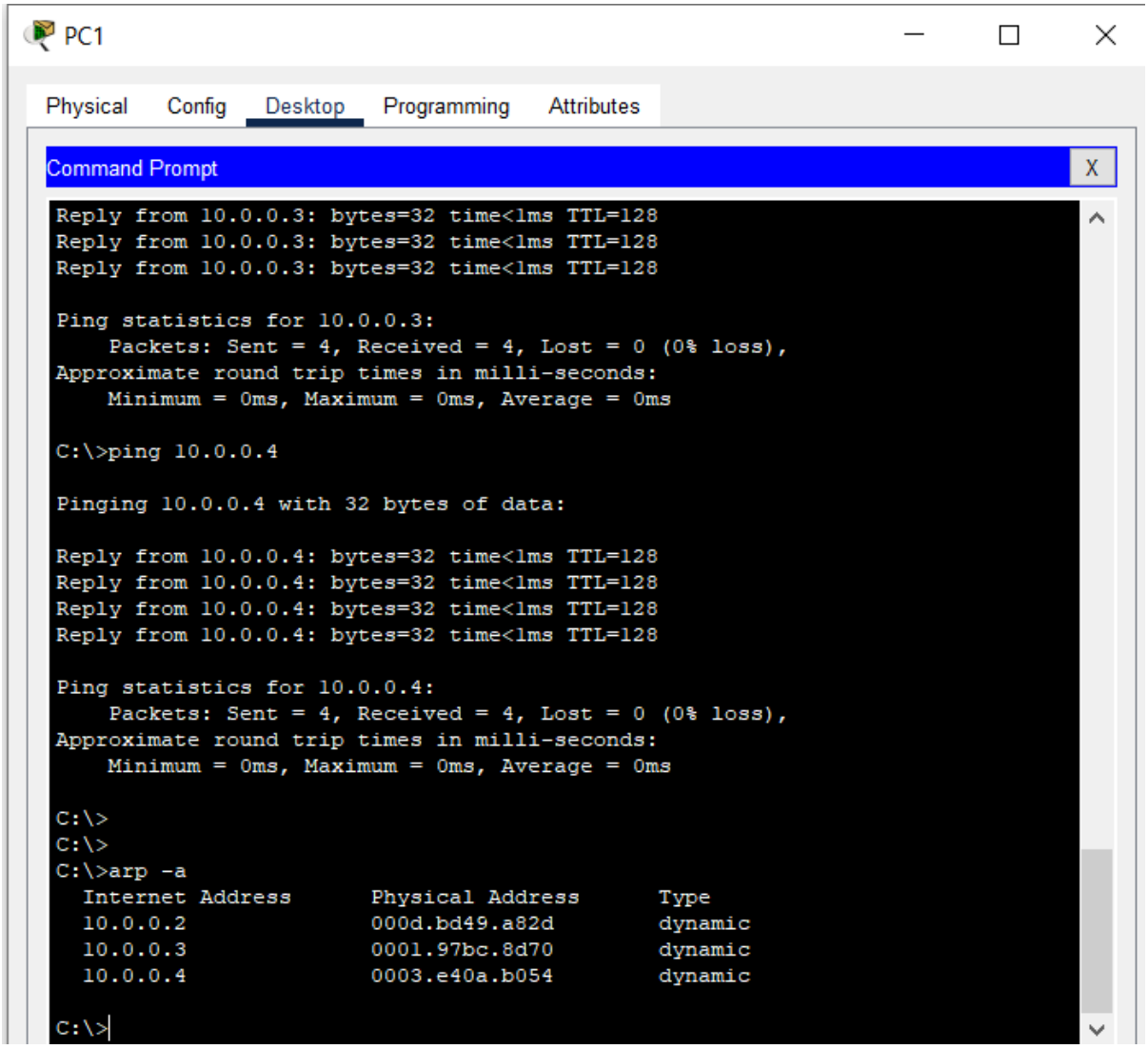
Initially if we look into ARP table of PC 1 there are no entries



Now ping to PC2 from PC1 as illustrated below. And check the arp table.



Now from PC1 ping PC3 and PC4 and watch the arp tables



Now you can ping from other systems and see how the ARP tables are constructed.

TASK-6

TASK 6: a). Configure and implementation of a Switch within a Network using Packet Tracer.

Aim: Implementation of a switch in packet tracer

Description: The Switch is a network device that is used to segment the networks into different subnetworks called subnets or LAN segments. It is responsible for filtering and forwarding the packets between LAN segments based on MAC address.

Switches have many ports, and when data arrives at any port, the destination address is examined first and some checks are also done and then it is processed to the devices. Different types of communication are supported here like unicast, multicast, and broadcast communication.

Modes of operation:

Mode	Purpose	Prompt	Command to enter	Command to exit
User EXEC	Allow you to connect with remote devices, perform basic tests, temporary change terminal setting and list system information	Router >	Default mode after booting. Login with password, if configured.	Use **exit** command
Privileged EXEC	Allow you to set operating parameters. It also includes high level testing and list commands like show, copy and debug.	Router #	Use **enable** command from user exec mode	Use **exit** command
Global Configuration	Contain commands those affect the entire system	Router(config)#	Use **configure terminal** command from privileged exec mode	Use **exit** command
Interface Configuration	Contain commands those modify the operation of an interface	Router(config-if)#	Use **interface *type number*** command from global configuration mode	Use **exit** command to return in global configuration Mode
Sub-Interface Configuration	Configure or modify the virtual interface created from physical interface	Router(config-subif)	Use **interface *type sub interface*** number command from global configuration mode or interface configure mode	Use **exit** to return in previous mode. Use **end** command to return in privileged exec mode.

Program:

Step 1. Open the packet tracer desktop and take a switch (PT-Switch) from the devices.

Step 2: Configure the Host name of the swicth0.

- Click on switch0 and go to Command Line Interface.
- Then change the hostname to “sh”

Command:

switch>
switch>en
switch#conf t
switch(config)#hostname sh

Step 3: Set a message of the day (MOTD) banner for the users.

Command:

sh(config)#banner motd $

Step 4: Set up line control password and enable secret password.

To configure the Line Control password and Enable secret follow the below commands:

sh#conf t sh(config)#
sh(config)#line con 0
sh(config-line)#password griet123
sh(config-line)#login
sh(config-line)#exit sh(config)#enable
secret griet12345
sh(config)#service password-encryption // encrypts the passwordsh(config)#exit

Step 5: Verify the password

- When you try to log in first, it will ask for the **line control password.**
- Then, to configure the terminal it will ask to **enable a secret password.**

To save the run configuration to startup file use the below command:

Command:

sh#copy run startup-config (OR) write
sh# no ip domain-lookup // used to prevent the router from trying to resolve incorrectly pasted commands in the cli by sending out a DNS query

Select the switch – goto cli mode and type the below configuration commands.

Switch>
Switch>enable

```
Switch#config terminal
Switch(config)#hostname sh
sh(config)#banner motd #Warning Unauthorised access is prohibited#

sh(config)#line con 0
sh(config-line)#password griet1234
sh(config-line)#login
sh(config-line)#exit

sh(config)#enable secret griet5678
sh(config)#service password-encryption

sh(config)#no ip domain-lookup

sh#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]


sh#show start
sh#show startup-config
Using 1238 bytes
!
version 15.0
no service timestamps log datetime msec
no service timestamps debug datetime msec
service password-encryption
!
hostname sh
!
enable secret 5 $1$mERr$vyUGBRk3bfoMV8qV.wJrB0
!
!
!
no ip domain-lookup
!
!
!
spanning-tree mode pvst
spanning-tree extend system-id
!
interface FastEthernet0/1
!
interface FastEthernet0/2
!
interface FastEthernet0/3
!
interface FastEthernet0/4
```

```
!
interface FastEthernet0/5

!< deleted some part>
!
interface FastEthernet0/20
!
interface FastEthernet0/21
!
interface FastEthernet0/22
!
interface FastEthernet0/23
!
interface FastEthernet0/24
!
interface GigabitEthernet0/1
!
interface GigabitEthernet0/2
!
interface Vlan1
no ip address
shutdown
!
banner motd ^CWarning Unauthorised access is prohibited^C
!
!
!
line con 0
password 7 08265E470C0D5445415F
login
!
line vty 0 4
login
line vty 5 15
login
!
```

Task 6B : Learn and Implement basic commands.

Aim: To implement the basic commands

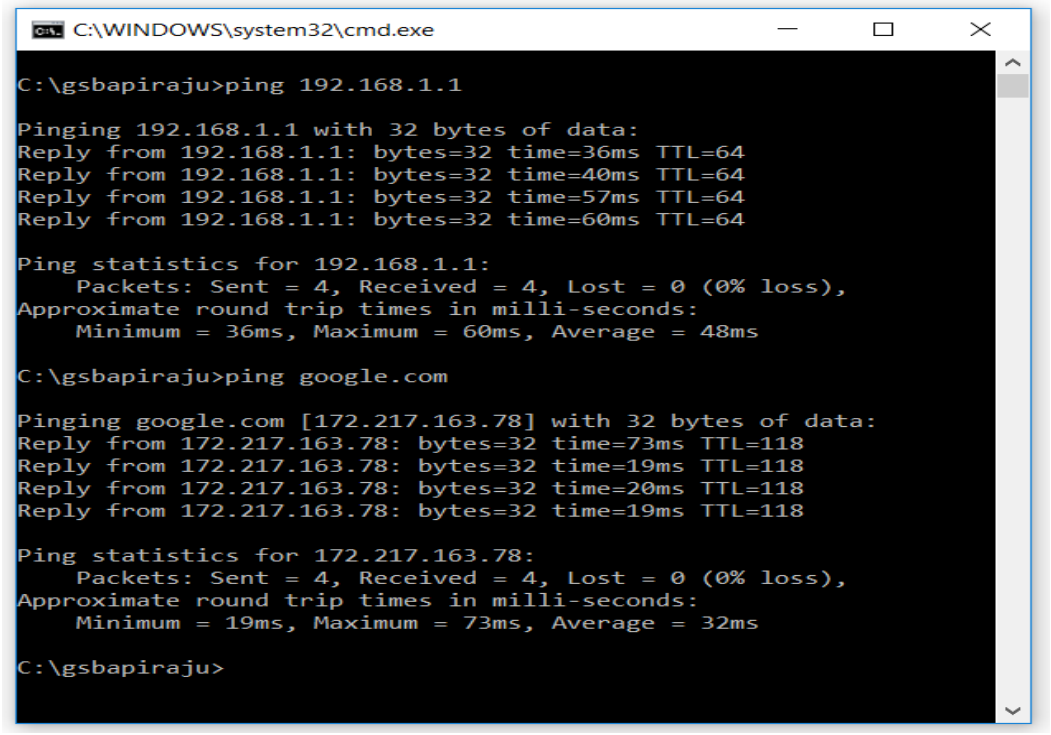
Program:

1. Ping

Ping is most commonly used network tool used to test the connection between the source and destination host.

Ping command uses Internet Control Message Protocol (ICMP) to send an echo packet from the source host to a destination host and listen to the response. If the source host receives a response from the destination host, this host is reachable. If not there is a connection error.

Using Ping command the user can identify in which area the connection problem is there, is it local or outside their LAN.

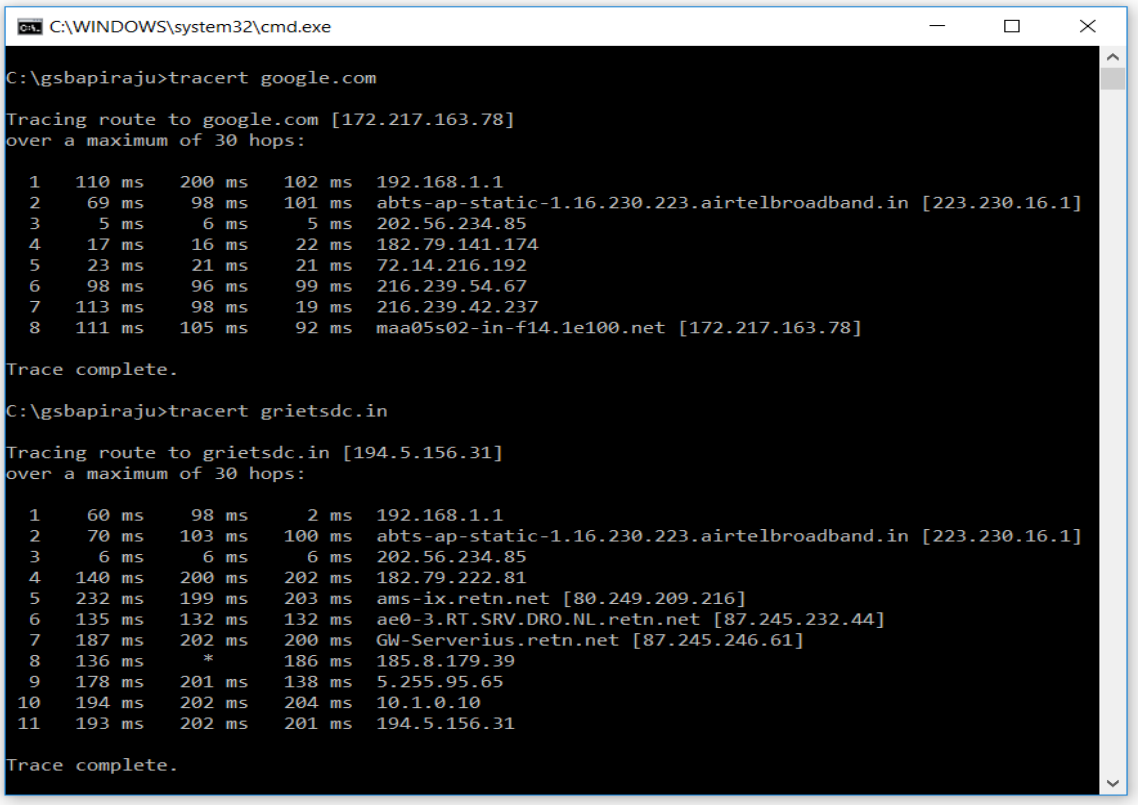


Ex: You can ping either by using the IP address or by the website name or URL. In the below example I pinged to my wireless router with its IP Address and google.com by its dom

2. Tracert/traceroute.

Ping is a basic tool to check the basic connectivity. But if you want to identify the complete path from the source node to the destination node than tracert/traceroute utility is very useful.

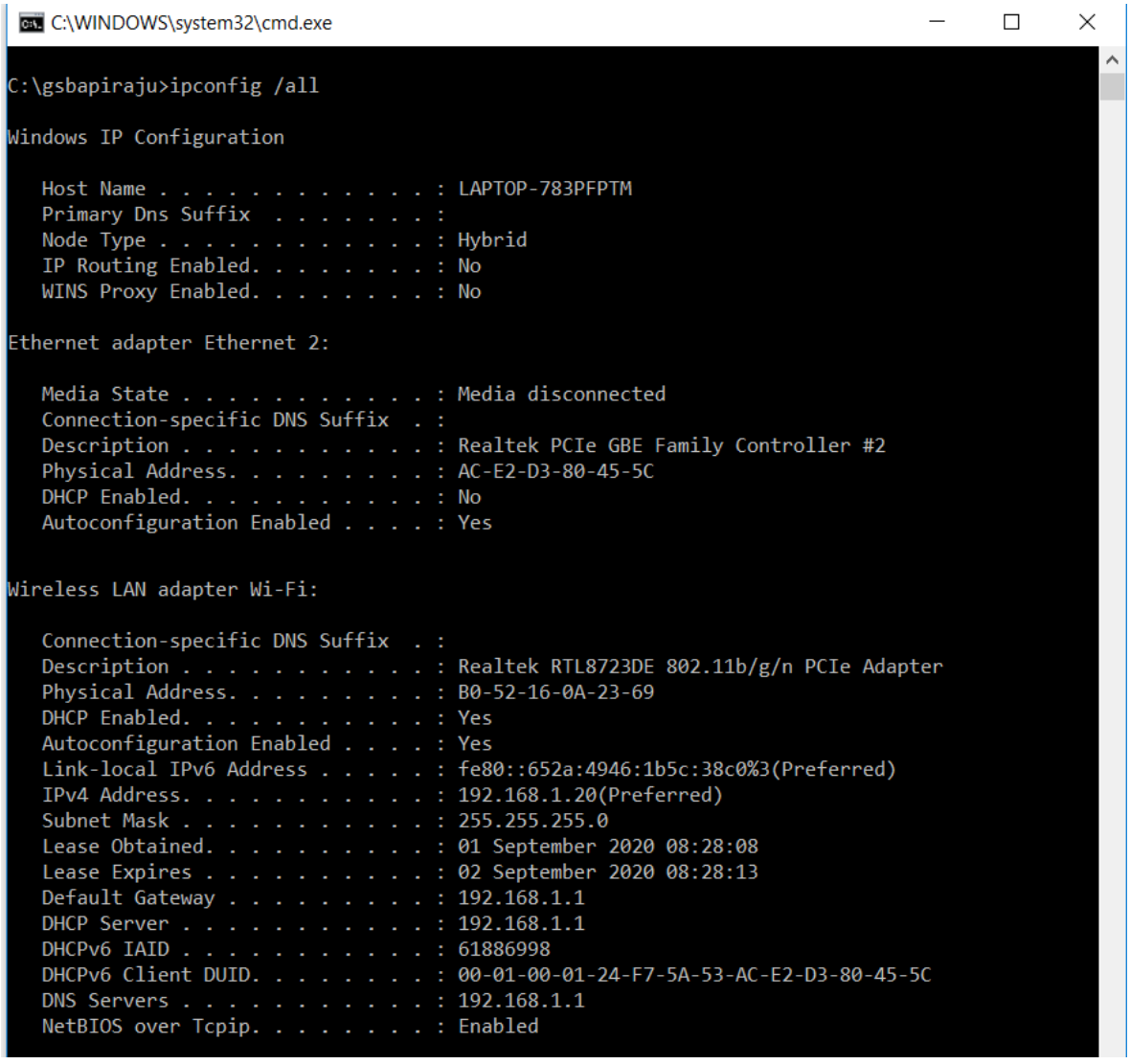
The tracert utility for windows and traceroute utility for Linux gives you the entire path, including the number of hops packet travelled.



3. Ipconfig

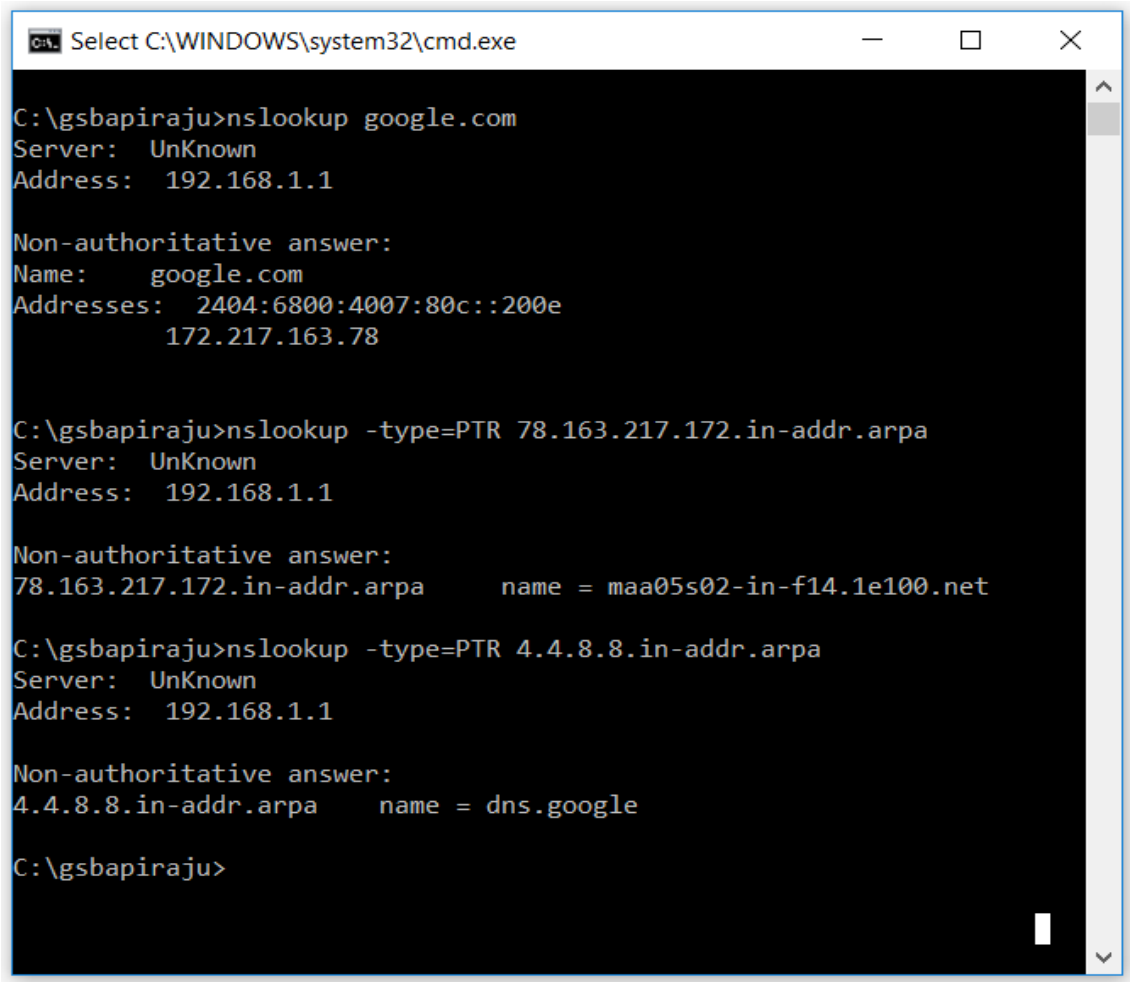
Ipconfig is one of the most important tool for system admins for troubleshooting networking issue. It is a command-line tool that shows the current TCP/IP configuration of the installed networking stack of a computer connected to a network.

This tool includes a number of switches to perform different actions. In the below example I am using /all which Produces a detailed configuration report for all interfaces. You can observe the 48 bit MAC address, IPaddress, DHCP details etc.



4. Nslookup

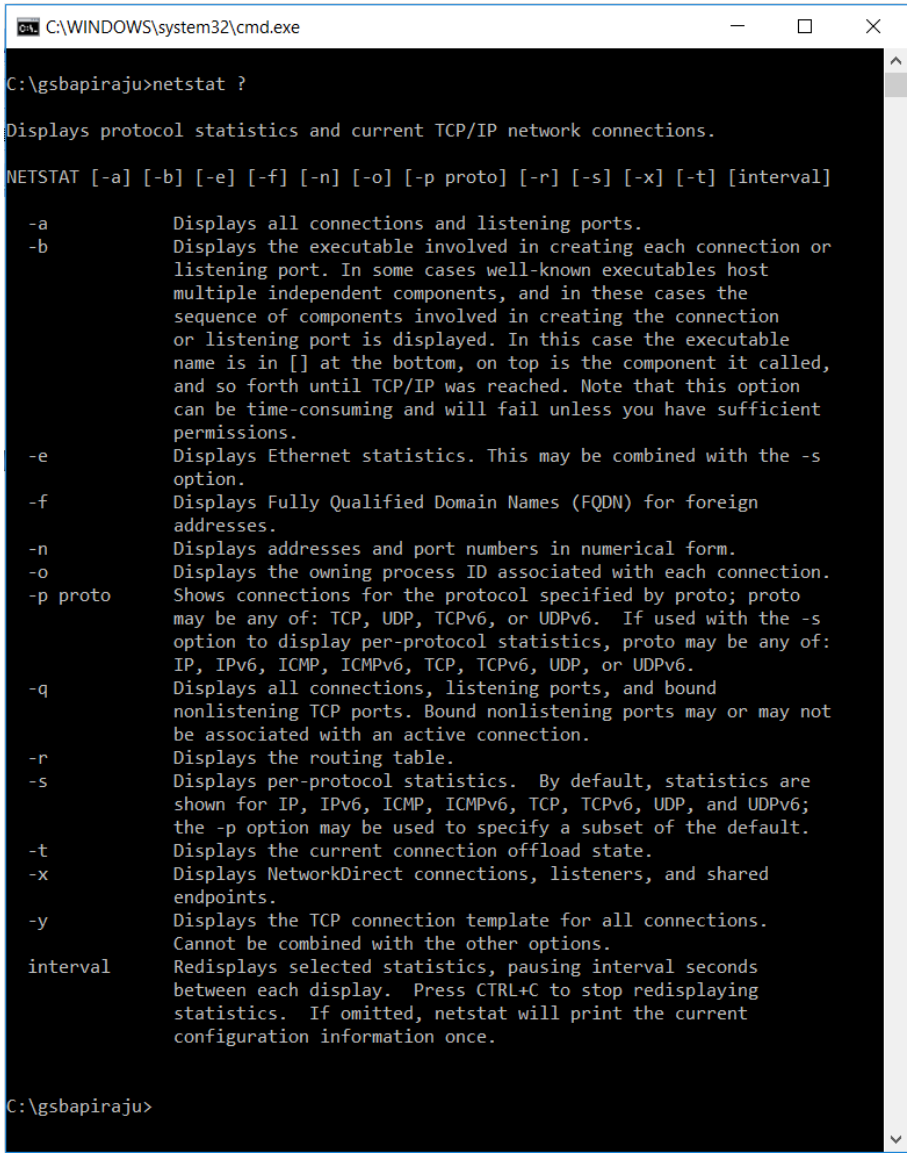
Some of the most common networking issues revolve around issues with Dynamic Name System (DNS) address resolution issues. nslookup or "name server lookup" is a network administration command-line tool used for querying the Domain Name System to obtain domain name or IP address mapping, or other DNS records. This utility can be used to lookup the specific IP address(es) associated with a domain name. If this utility is unable to resolve this information, there is a DNS issue.



A typical DNS lookup is used to determine which IP address is associated with a hostname. A reverse DNS lookup is used for the opposite, to determine which hostname is associated with an IP address. Sometimes reverse DNS lookups are required for diagnostic purposes.

5. Netstat.

Netstat (*network statistics)* is a program that's controlled via commands issued in the command line. It delivers basic statistics on all network activities and informs users on which **ports and addresses** the corresponding connections (TCP, UDP) are running and which ports are open for tasks. The below example illustrates various switches of netstat.



Task 7

Task 7: a). Configure and implementation of a Router within a Network using Packet Tracer.

Aim: To configure and implement router

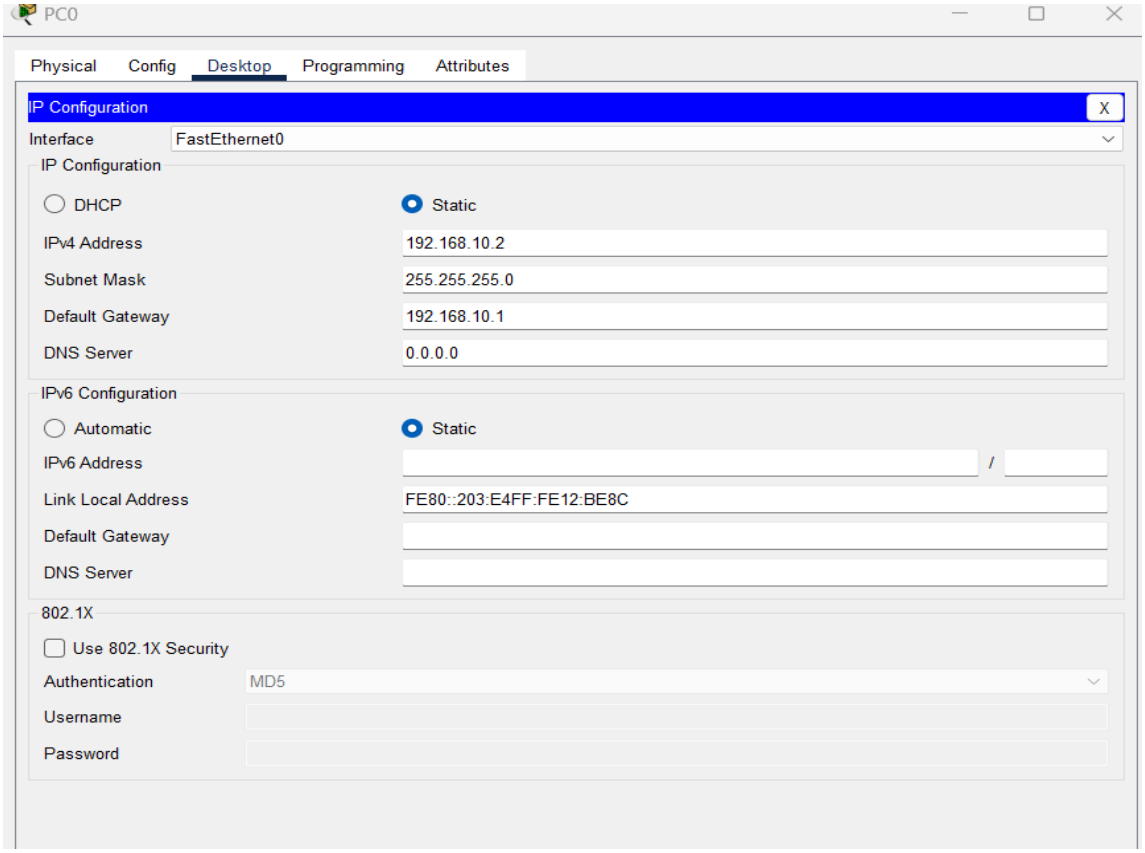
Description: A router is a device that connects two or more packet-switched networks or subnetworks. It serves two primary functions: managing traffic between these networks by forwarding data packets to their intended IP addresses.

Program:

Step 1: Set Up the Network Topology

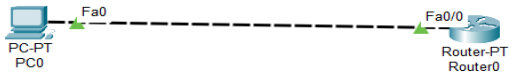
1. **Open Packet Tracer**: Launch Cisco Packet Tracer.
2. **Add Devices**:
    - Drag and drop a router (e.g., 2911 or any suitable model) onto the workspace.
    - Add a couple of PCs (end devices) for testing connectivity.
    - Optionally, you can add switches if you need to connect multiple PCs.
    - Need to provide Default Gateway (i.e. Router interface IP address)

(The **default gateway** is the node that forwards the packet from the source to other **networks)**



3. **Connect Devices**:
    - Use the connection tool (cable icon) to connect the devices. For example, connect the PCs to a switch and the switch to the router.

- Use appropriate cables



Step 2: Configure the Router

1. **Access the Router CLI**:
   - Click on the router and go to the "CLI" tab.
2. **Enter Configuration Mode**:

   Configure ip address of pc 192.168.10.2

```
Router>en
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#int Fa0/0
Router(config-if)#ip address 192.168.10.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to
up

Router#en
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#enable secret griet1234
Router(config)#login
Router(config)#no ip domain-lookup
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#line console 0
Router(config-line)#password 1234
Router(config-line)#login
Router(config-line)#exit
Router(config)#line vty 0 4
Router(config-line)#password g1234
Router(config-line)#login
Router(config-line)#exit
Router(config)#
```

Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#exit

Router con0 is now available

Press RETURN to get started.

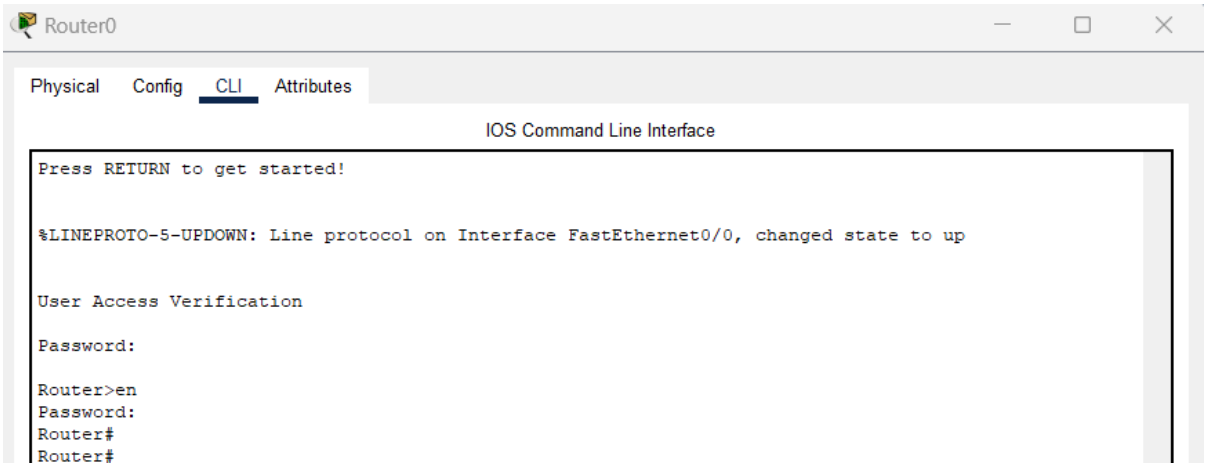
Step 3: To test the console and enable secret passwords

User Access Verification

Password: 1234

Router>en
Password: griet1234
Router#exit

Note: To verify password, need to exit from all commands and enter into user mode

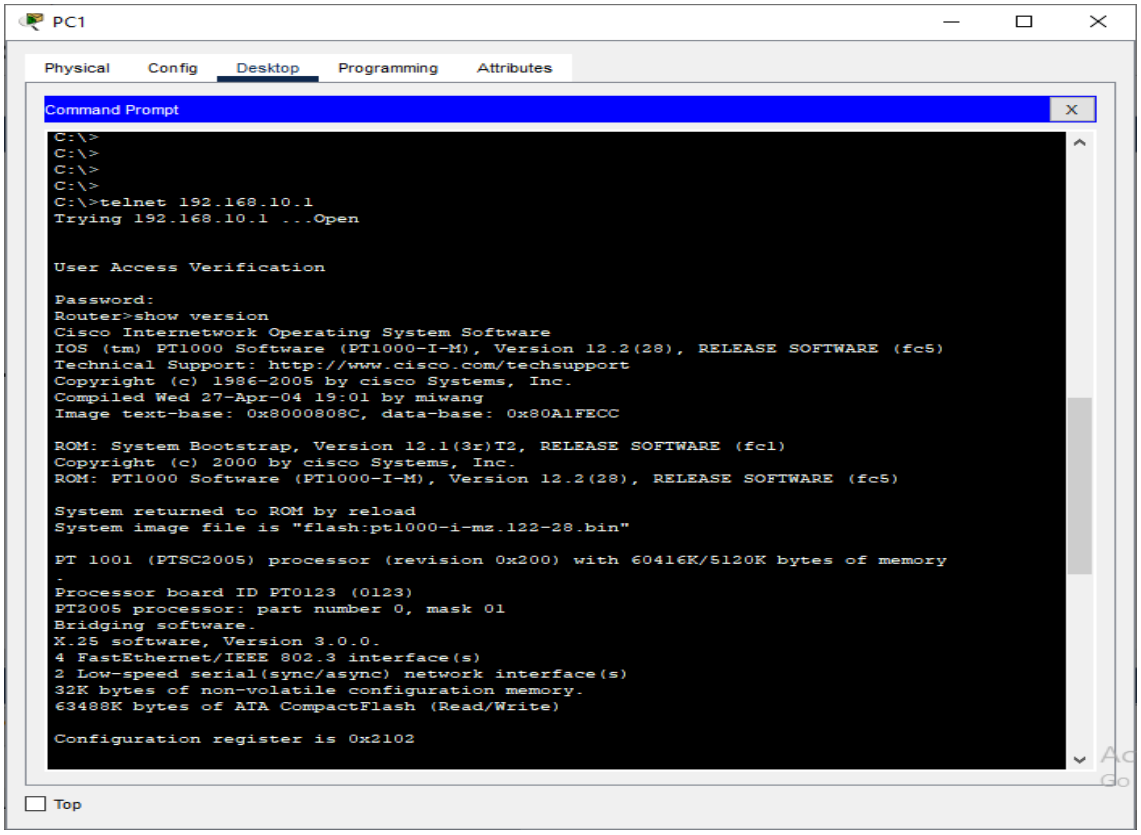


Step 4: To test vty password

Goto pc ->command prompt

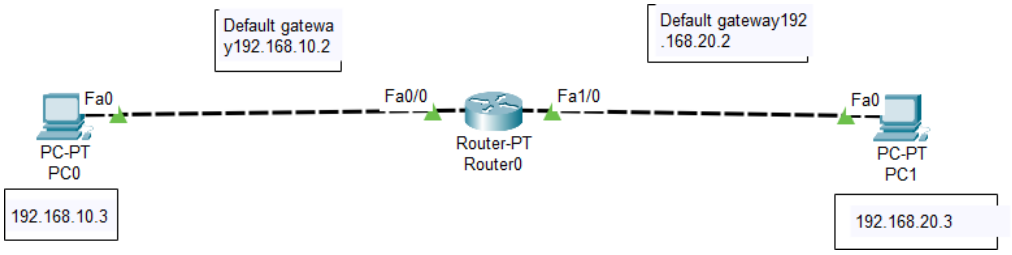
Need to give password of Vty

telnet 192.168.10.1(this is router ip)



Sub-task:

Same as above to implement PC to PC communication using Router

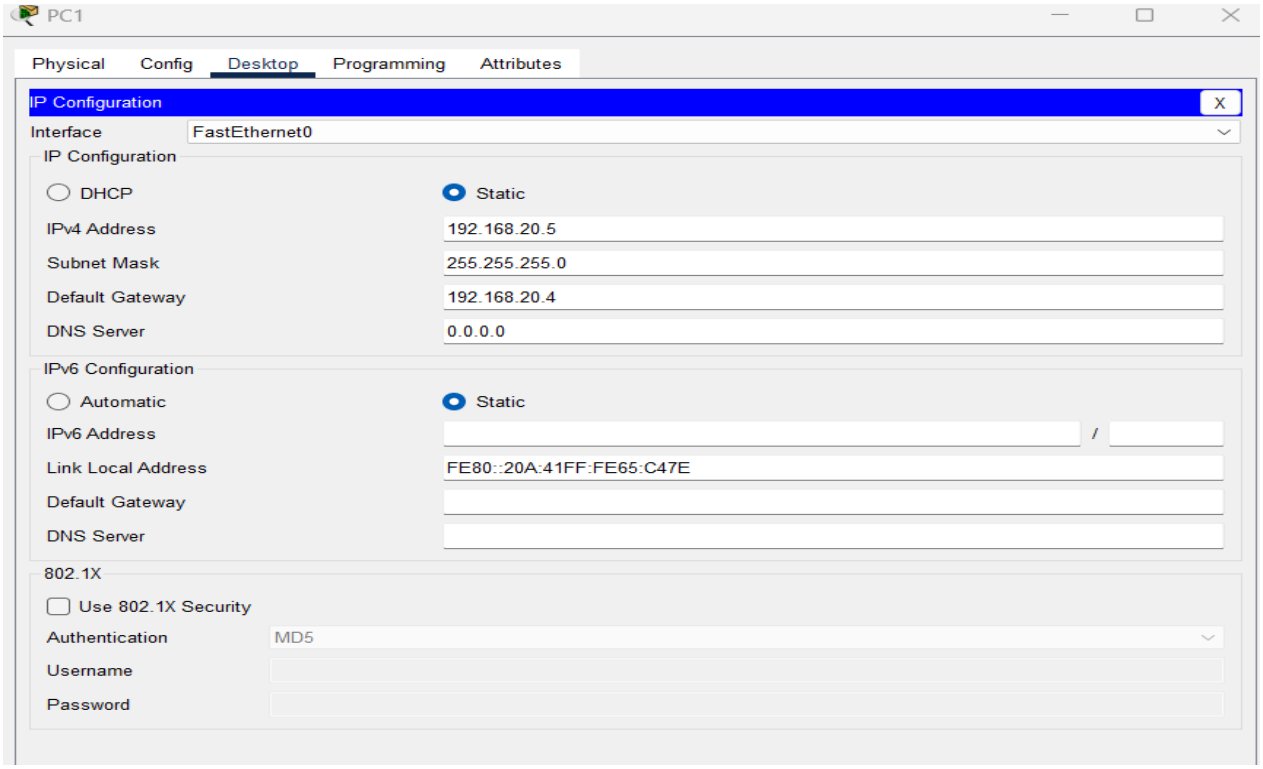


Configure ip address of pc-192.168.10.2
Configure router below steps
Router>en
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#int Fa0/0
Router(config-if)#ip address 192.168.10.1 255.255.255.0
Router(config-if)#no shutdown
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up

---

[optional to add



Router(config-if)#exit
Router(config)#int Fa1/0
Router(config-if)#ip address 192.168.20.2 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet1/0, changed state to up

Router(config-if)#exit
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet1/0, changed state to up
]

Router#en
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#enable secret griet1234
Router(config)#login
Router(config)#no ip domain-lookup
Router(config)#exit

%SYS-5-CONFIG_I: Configured from console by console

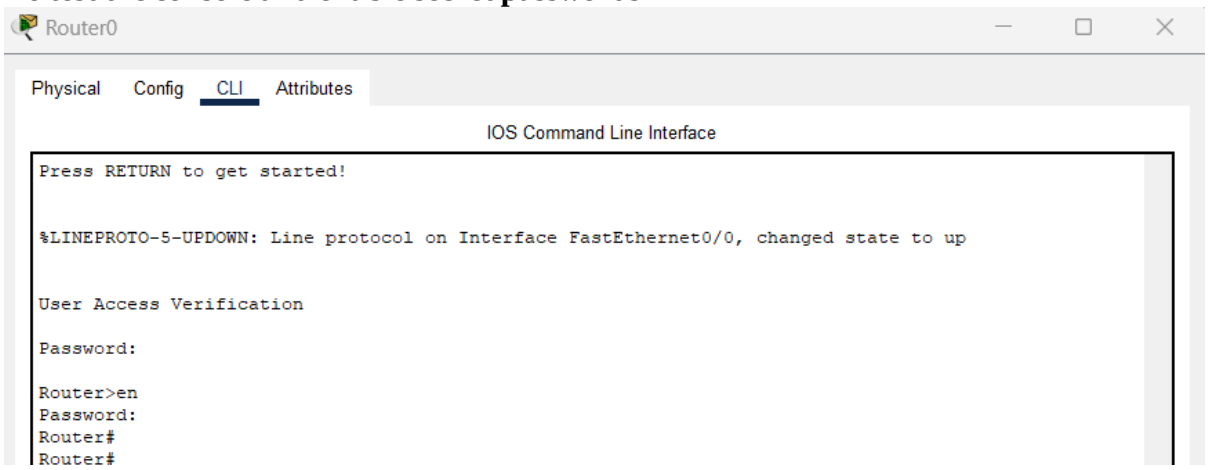
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#line console 0
Router(config-line)#password 1234
Router(config-line)#login
Router(config-line)#exit
Router(config)#line vty 0 4
Router(config-line)#password g1234
Router(config-line)#login
Router(config-line)#exit
Router(config)#
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#exit

Router con0 is now available

Press RETURN to get started.

To test the console and enable secret passwords

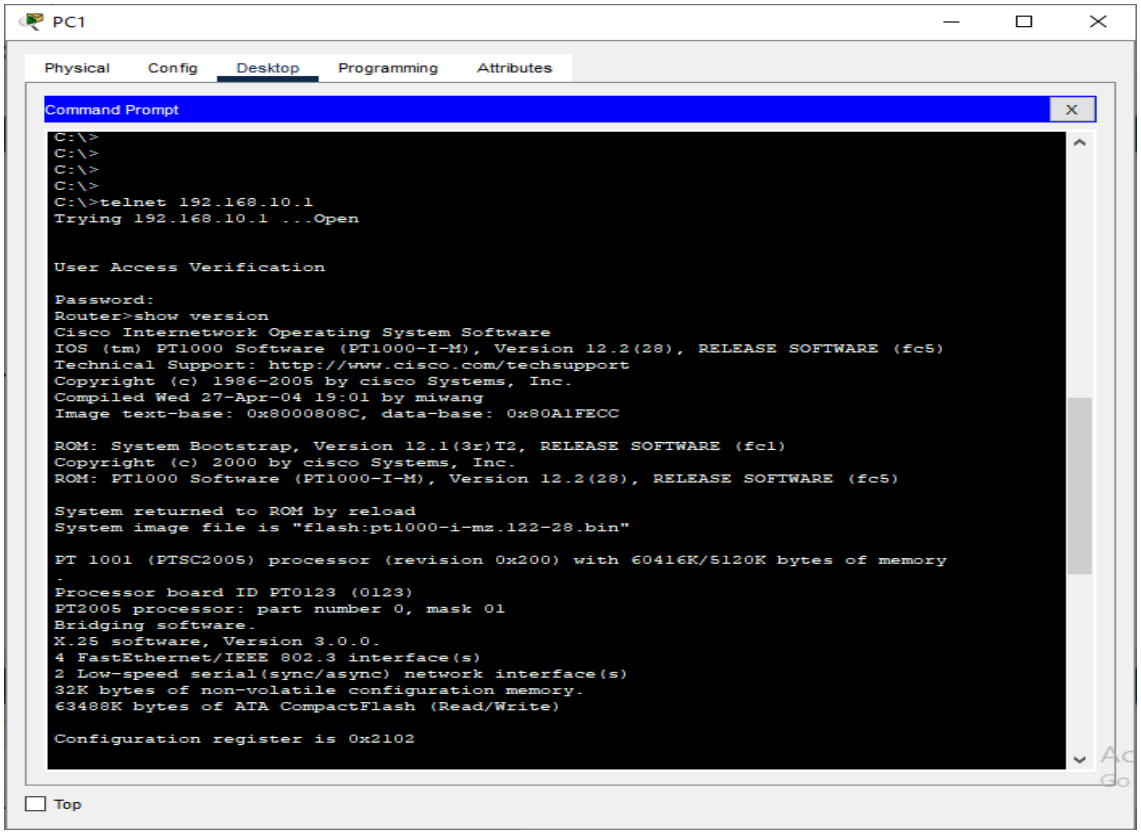


User Access Verification

Password: 1234

Router>en
Password: griet1234
Router#exit

To test vty password

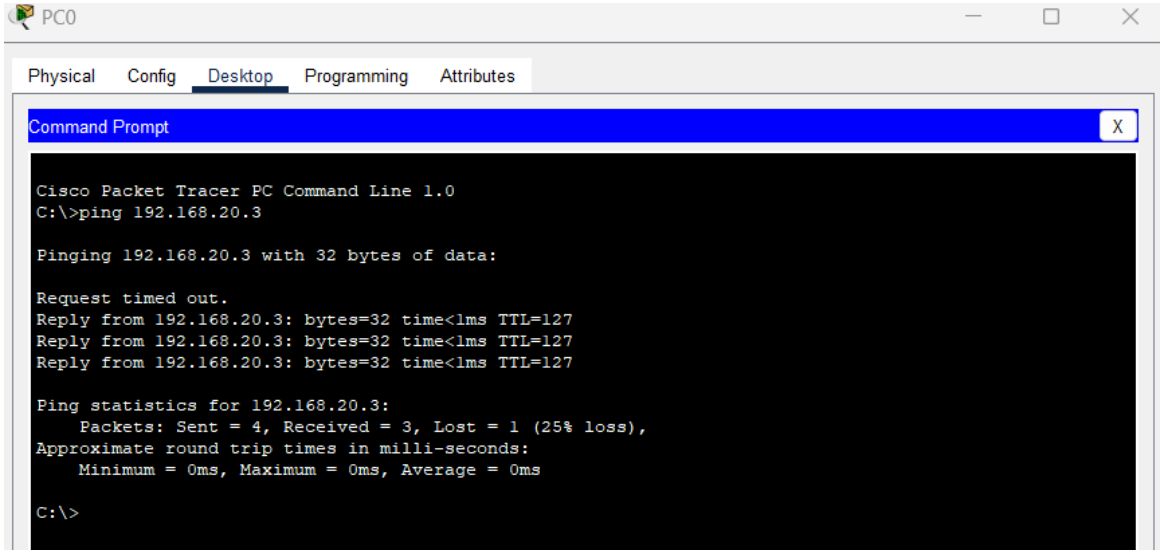


Goto pc ->command prompt

telnet 192.168.10.1(this is router ip)

Test Connectivity

- **Ping** from one PC to another (e.g., from **PC0** to **PC1**) to ensure static routing is functioning properly.



b). Configure and examine Network Address Translation (NAT).

Aim: To configure and examine Network Address Translation

Description:

Network Address Translation (NAT)

NAT is a method used to remap IP addresses by modifying network address information in the IP header of packets while they are in transit. NAT is commonly used to enable multiple devices on a local network to share a single public IP address for Internet access.

Types of NAT:

1. **Static NAT**: Maps one private IP address to one public IP address. It's useful for hosting services that must be accessible from outside the network.
2. **Dynamic NAT**: Maps multiple private IP addresses to a pool of public IP addresses.
3. **PAT (Port Address Translation)**: Also known as **NAT overload**, it maps multiple private IP addresses to a single public IP address by differentiating sessions with unique port numbers.

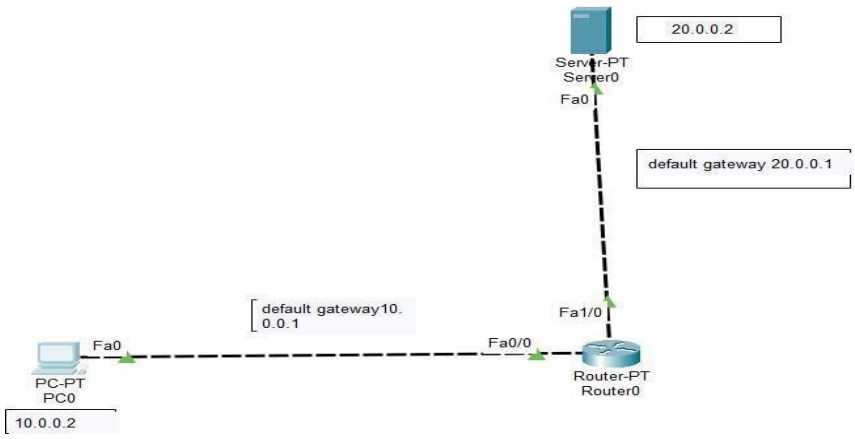
- **ip nat inside source static**:
    - This specifies that you are configuring a **static NAT** translation. Static NAT creates a permanent, one-to-one mapping between an inside local address (private IP) and an inside global address (public IP).
    - **inside_local_ip**: The private IP address of the internal device.
    - **outside_global_ip**: The public IP address accessible from the internet.

Example Use Case

Suppose you have a web server inside your private network with the IP address 10.0.0.2, and you want external users to be able to access it using the public IP address 200.100.50.25. The static NAT configuration ensures that when users connect to 200.100.50.25, the router translates the request to 10.0.0.2 and forwards the traffic to the web server.

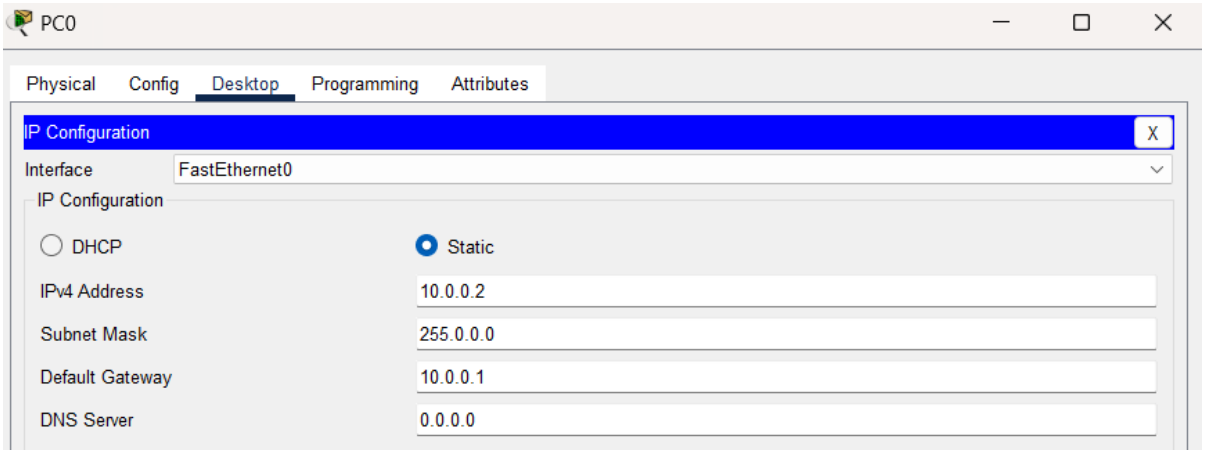
Program:

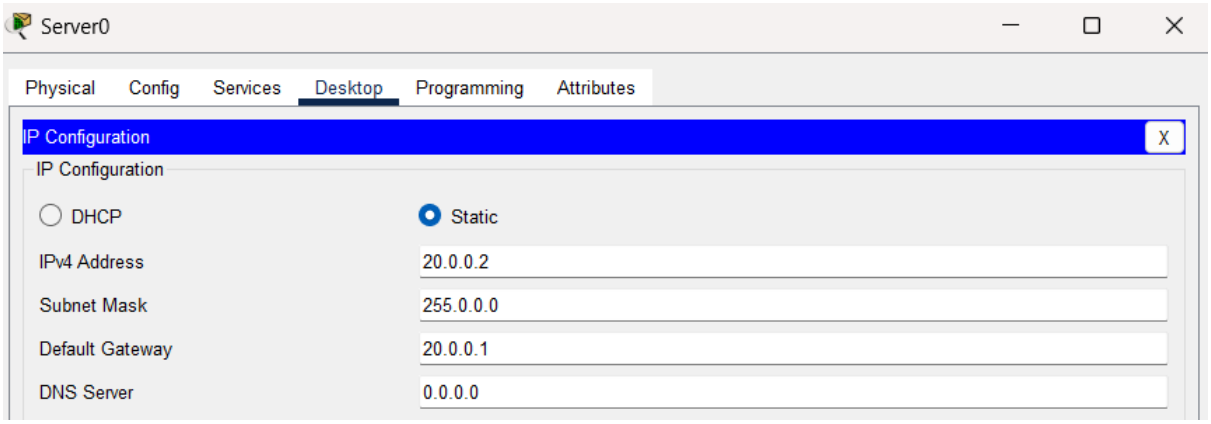
i). Static routing



Step 1:

IP configurations:





Step 2:

Router configuration:

Route>
Router>en
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#int fa0/0
Router(config-if)#ip add 10.0.0.1 255.0.0.0
Router(config-if)#no shut

Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up

Router(config-if)#ip nat inside
Router(config-if)#int fa1/0
Router(config-if)#ip add 20.0.0.1 255.0.0.0
Router(config-if)#no shut

%LINK-5-CHANGED: Interface FastEthernet1/0, changed state to up

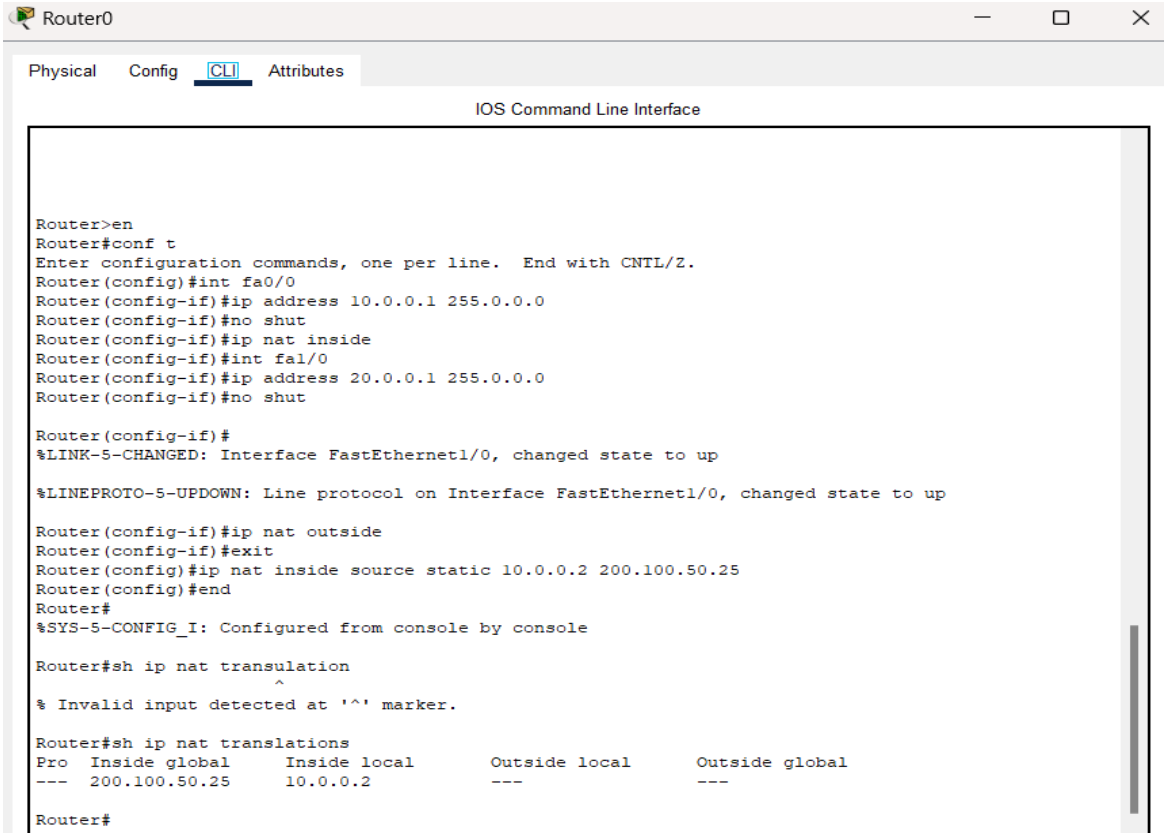
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet1/0, changed state to up

Router(config-if)#
Router(config-if)#ip nat outside
Router(config-if)#exit

Router(config)#ip nat inside source static 10.0.0.2 200.100.50.25
Router(config)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console

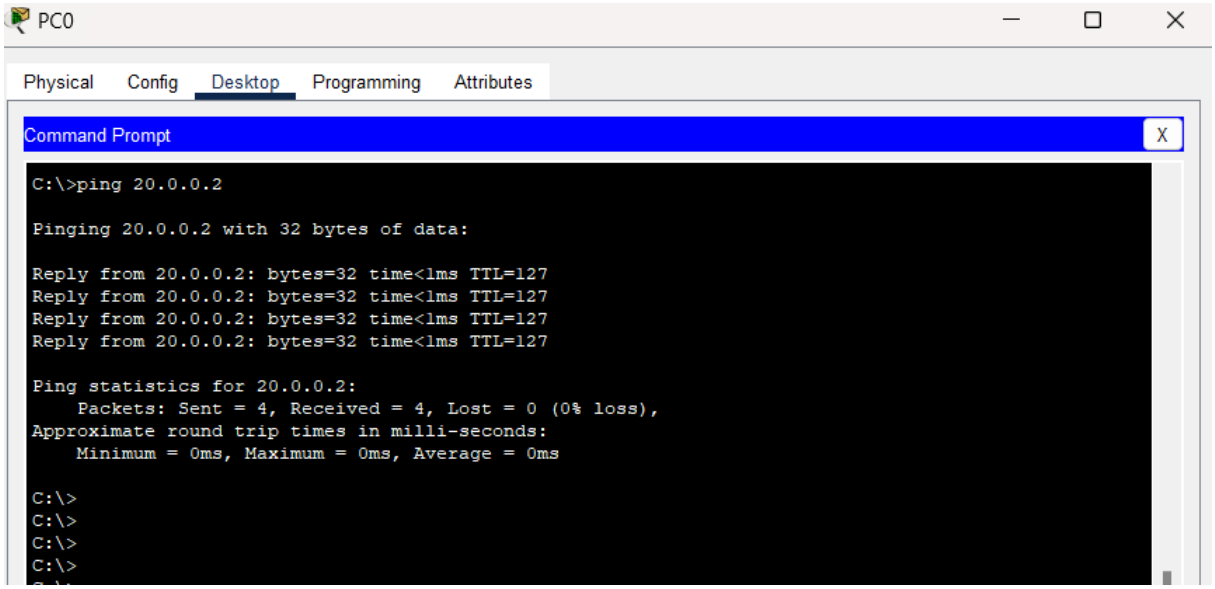
Router#
Router#sh ip nat translations
Pro Inside global Inside local Outside local Outside global
--- 200.100.50.25 10.0.0.2 --- ---

Router#

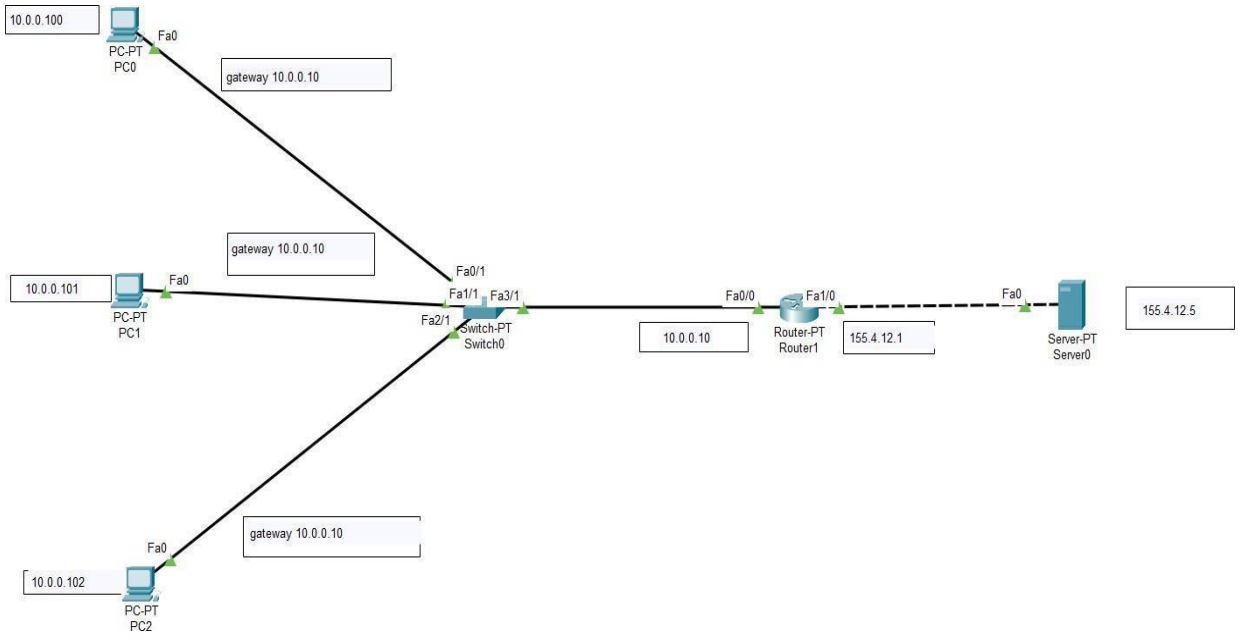


Step 3: Test Connectivity

To test the connection ping from the terminal



ii). Dynamic routing



Router>en
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#int fa0/0
Router(config-if)#ip address 10.0.0.10 255.0.0.0
Router(config-if)#no shut

Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up

Router(config-if)#ip nat inside

Router(config-if)#ex
Router(config)#int fa1/0
Router(config-if)#ip address 155.4.12.1 255.255.0.0
Router(config-if)#no shut

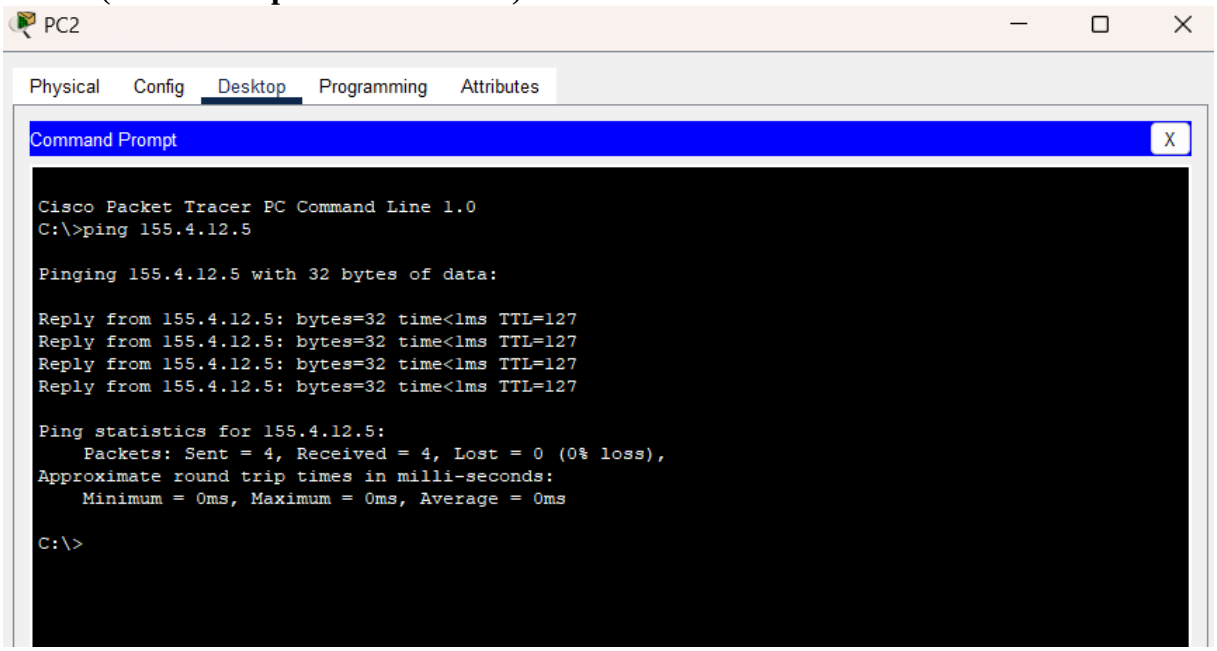
Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet1/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet1/0, changed state to up

Router(config-if)#ip nat outside
Router(config-if)#ex
Router(config)#access-list 1 permit 10.0.0.0 0.0.0.255
Router(config)#ip nat pool newpool 155.4.12.1 155.4.12.3 netmask 255.255.255.0
Router(config)#ip nat inside source list 1 pool newpool
Router(config)#ip route 0.0.0.0 0.0.0.0 155.4.12.5
Router(config)#ex
Router#
%SYS-5-CONFIG_I: Configured from console by console

Note:

- To test the nat translations, need ping from source system to destination system. (Ex: from PC2 **c:\>ping 155.4.12.5**)
- After ping, go to Router configuration and check for nat translations (**Router#sh ip nat translations**)



Router#sh ip nat translations
Router#sh ip nat translations
Router#sh ip nat translations
Pro Inside global Inside local Outside local Outside global
icmp 155.4.12.1:1 10.0.0.102:1 155.4.12.5:1 155.4.12.5:1
icmp 155.4.12.1:2 10.0.0.102:2 155.4.12.5:2 155.4.12.5:2
icmp 155.4.12.1:3 10.0.0.102:3 155.4.12.5:3 155.4.12.5:3
icmp 155.4.12.1:4 10.0.0.102:4 155.4.12.5:4 155.4.12.5:4

Task 8

Task-8 : a). Configure network topology to implement VLANs with using Packet Tracer software.

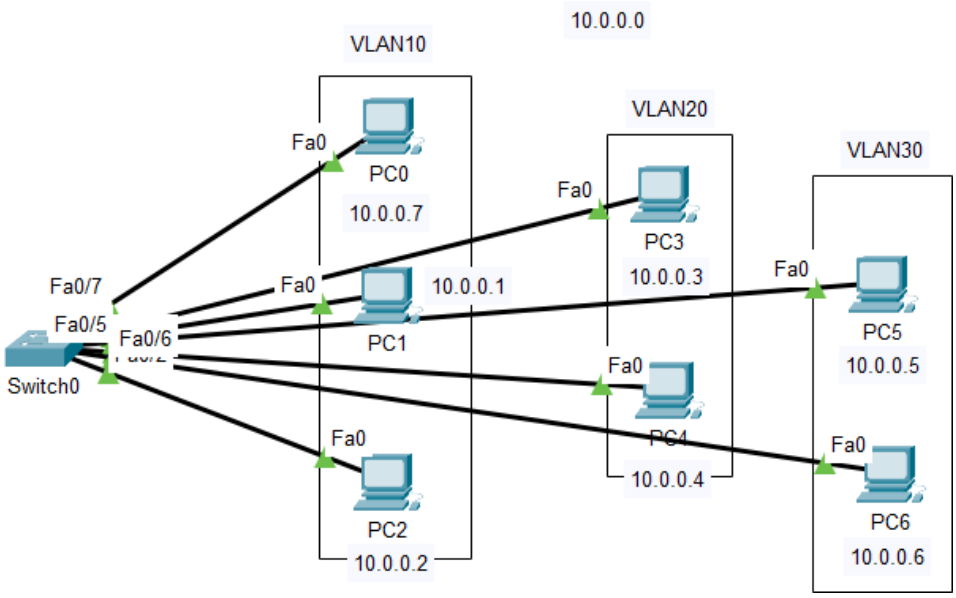
Aim: To configure and implement VLANS using packet tracer

Description:

Intra-vlan: Devices on the same VLAN can communicate directly without needing any special routing

Example: All computers in the accounting department on the same VLAN can easily access each others

INTRA VLAN



1. Set Up the Devices

- **Drag and Drop Devices**: Use at least one switch and multiple PCs (or other end devices).
- **Topology**: Connect the PCs to the switch using Ethernet cables.

2. Access the Switch CLI

- **Open CLI on Switch**: Click on the switch, go to the CLI tab to configure VLANs.

3. Create VLANs

- Enter global configuration mode on the switch by typing:
  - **switchport mode access**: Configures the port as an access port.
  - **switchport access vlan 10**: Assigns the port to VLAN 10. Only traffic for VLAN 10 will be allowed on this port.

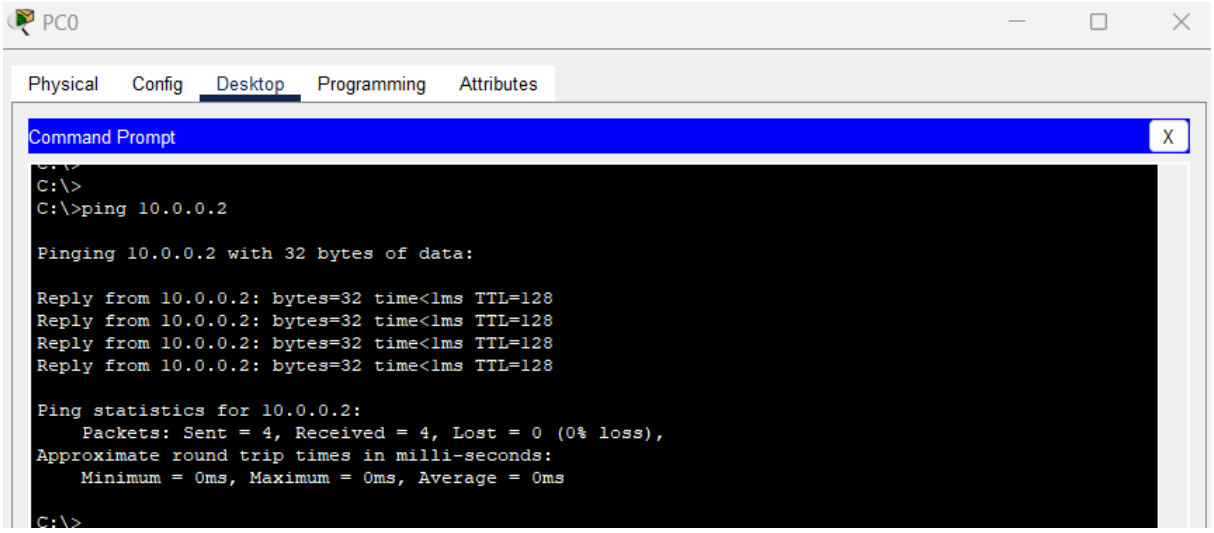
```
Switch>en
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 10
Switch(config-vlan)#name project
Switch(config-vlan)#int f0/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 10
Switch(config-if)#int f2/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 10
Switch(config-if)#int f1/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 10

Switch(config-if)#vlan 20
Switch(config-vlan)#int f3/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 20
Switch(config-if)#int f6/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 20

Switch(config-if)#vlan 30
Switch(config-vlan)#name project3
Switch(config-vlan)#int f7/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 30
Switch(config-if)#int f8/1
Switch(config-if)#switchport mode access
Switch(config-if)#switchport access vlan 30
Switch(config-if)#end
Switch#
```

4. Testing Intra-VLAN Communication

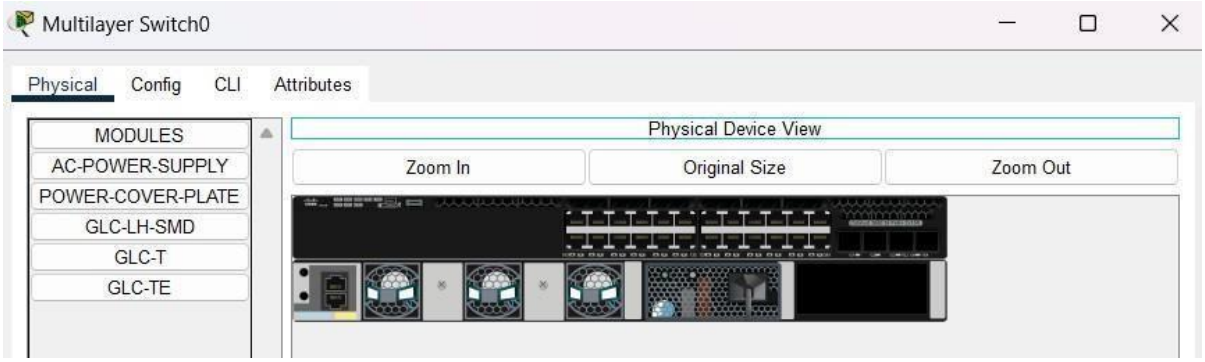
- PCs within the same VLAN should be able to communicate with each other but **not with devices in other VLANs** .
- This verifies that VLANs are segmented and that only intra-VLAN communication is allowed.

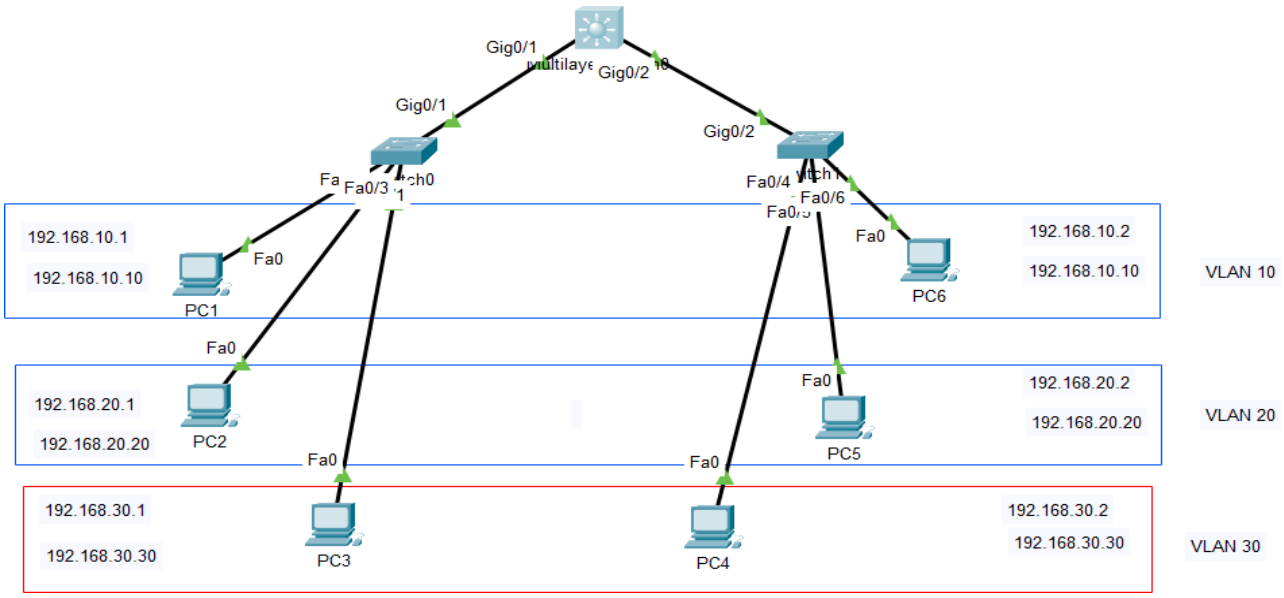


INTERVLAN and Multilayer Switch Configuration:

Inter-VLAN routing on a multilayer switch enables communication between different VLANs (Virtual Local Area Networks) within the same switch, bypassing the need for an external router. A multilayer switch combines the functions of a Layer 2 switch (which operates on MAC addresses) and a Layer 3 router (which uses IP addresses), allowing it to perform routing functions for VLANs internally.

- To configure **Inter-VLAN Routing** using a **multilayer switch**, we'll enable routing between VLANs on the switch itself.
- "Turn On" Multilayer Switch:
  Drag and drop the multilayer switch (like the Cisco 3650-24PS) from the **Network Devices** > **Switches** section in Packet Tracer onto the workspace.
- Note: need to give power supply from physical(drag and drop **AC-POWER-SUPPLY** from physical to physical device view.





1. Configure VLANs
2. Assign Switch Ports to VLANs
3. Enable IP Routing on the Switch
4. Configure Trunk Ports

Switch> Switch>EN Switch# Switch#conf t Switch(config)# Switch(config)#vlan 10 Switch(config-vlan)#vlan 20 Switch(config-vlan)#vlan 30 Switch(config-vlan)#int f0/1 Switch(config-if)#no shut Switch(config-if)#switchport mode access Switch(config-if)#switchport access vlan 10 Switch(config-if)#int f0/2 Switch(config-if)#switchport mode access Switch(config-if)#switchport access vlan 20 Switch(config-if)#int f0/3 Switch(config-if)#switchport mode access Switch(config-if)#switchport access vlan 30 Switch(config-if)#int g0/1 Switch(config-if)#switchport mode trunk Switch(config-if)#end Switch#wr Building configuration... [OK] **Switch#**	Switch> Switch>en Switch#conf t Switch(config)#hostname SW2 SW2(config)# SW2(config)#vlan 10 SW2(config-vlan)#vlan 20 SW2(config-vlan)#vlan 30 SW2(config-vlan)#int f0/4 SW2(config-if)#switchport mode access SW2(config-if)#switchport access vlan 30 SW2(config-if)#int f0/5 SW2(config-if)#switchport mode access SW2(config-if)#switchport access vlan 20 SW2(config-if)#int f0/6 SW2(config-if)#switchport mode access SW2(config-if)#switchport access vlan 10 SW2(config-if)#int g0/2 SW2(config-if)#sw SW2(config-if)#switchport mode trunk SW2(config-if)#end SW2#wr Building configuration... [OK] **SW2#**

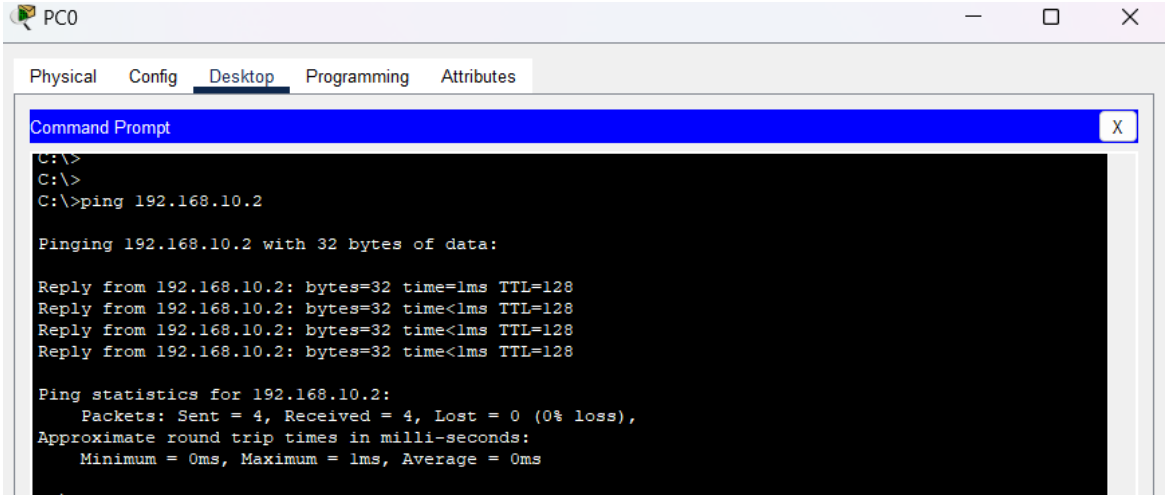
Multilayer Switch0 configuration:

switchport mode trunk: Configures the port as a trunk, allowing it to carry multiple VLANs with 802.1Q tagging.

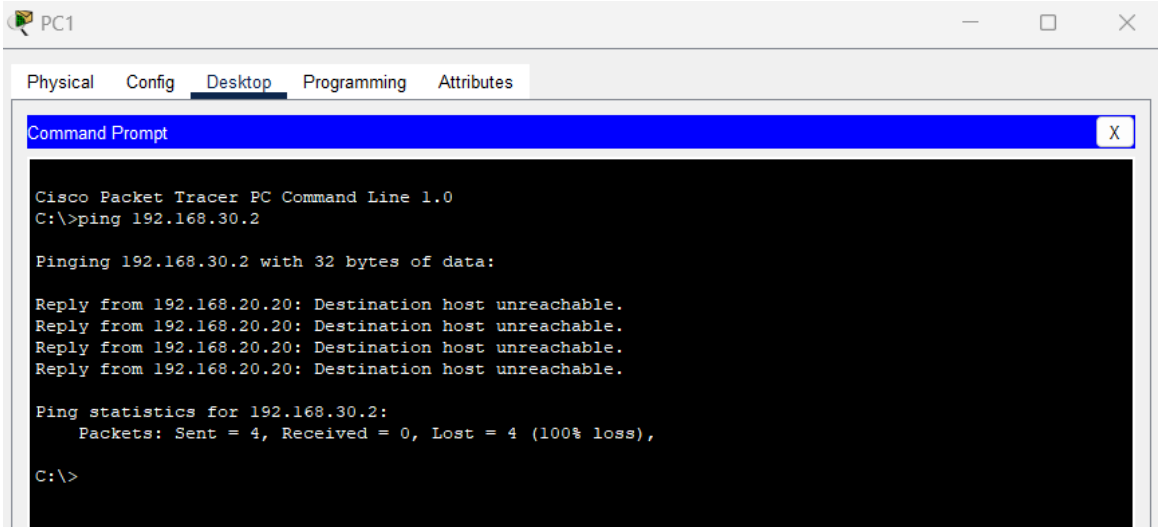
Switch> Switch>en Switch# Switch#conf t Switch(config)#hostname MLSW MLSW(config)#vlan 10 MLSW(config-vlan)#vlan 20 MLSW(config-vlan)#vlan 30 MLSW(config-vlan)#exit MLSW(config)# MLSW(config)#int range g1/0/1-2 MLSW(config-if-range)#switchport mode trunk MLSW(config-if-range)#end **After this you will be able to ping in between the same VLANs. But not other vlans.** Ex: 192.168.10.1 to 10.2 But you cannot access 192.168.10.1 to 20.1	**Below conf. is to access from one pc to all PCs.** MLSW#conf t Enter configuration commands, one per line. End with CNTL/Z. MLSW(config)# MLSW(config)#ip routing MLSW(config)#int vlan 10 MLSW(config-if)# MLSW(config-if)#ip add 192.168.10.10 255.255.255.0 MLSW(config-if)#no shut MLSW(config-if)#int vlan 20 MLSW(config-if)# MLSW(config-if)#ip add 192.168.20.20 255.255.255.0 MLSW(config-if)#no shut MLSW(config-if)# MLSW(config-if)#int vlan 30 MLSW(config-if)# MLSW(config-if)#ip add 192.168.30.30 255.255.255.0 MLSW(config-if)#no shut MLSW(config- if)# MLSW(config- if)#end MLSW# MLSW#wr Building configuration... [OK] **MLSW#**

Test Configurations

- **able to ping in between the same VLANs**



- **able to ping from one PC to all other PCs VLANs**



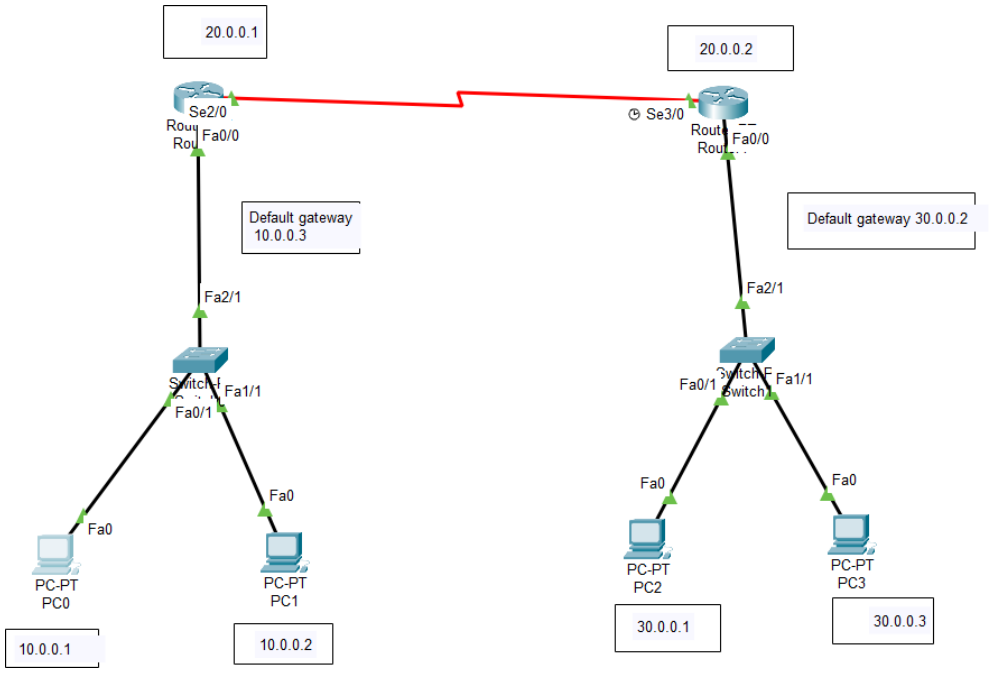
b). Configure network topology and implement static routing using Packet Tracer Software. .STATIC ROUTING:

Aim:

To implement static routing.

Description: Static routing is a network routing system that uses manually configured routes to send traffic to specific destinations. It's often used in smaller networks or when security is a top priority.

Program:



To configure network topology and implement static routing in Cisco Packet Tracer, follow these steps:

Step 1: Design the Network Topology

1. **Open Packet Tracer** and create a new project.
2. **Add Network Devices**:
   - Drag and drop routers, switches, and PCs onto the workspace.
   - For example, use **2 routers**, **2 switches**, and **4 PCs**.
3. **Interconnect the Devices**:
   - Use appropriate cables (copper straight-through for different devices, copper crossover for similar devices) to connect routers, switches, and PCs.
   - Make sure to connect the routers via serial connections interfaces.

Step 2: Configure Routers

For each router, interface IP addresses, and enable the interfaces.

Configure Router0 and Router1

- Enter the CLI of **Router0** and **Router1**
- Configure interface IP addresses:

Router configuration:

Router0
Interfaces:

Fa0/0	10.0.0.3	255.0.0.0
Serial2/0	20.0.0.1	255.0.0.0

Router1

Interfaces:

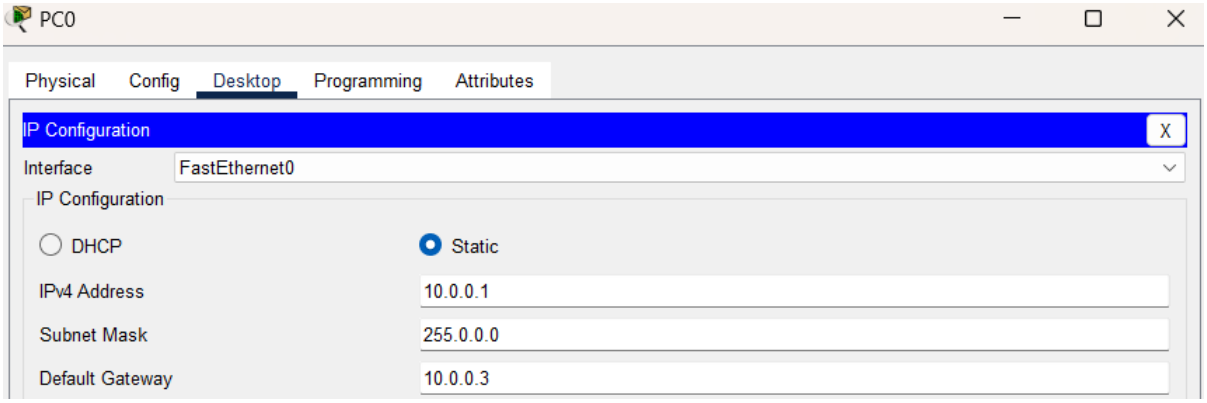
Fa0/0	30.0.0.2	255.0.0.0
Serial3/0	20.0.0.2	255.0.0.0

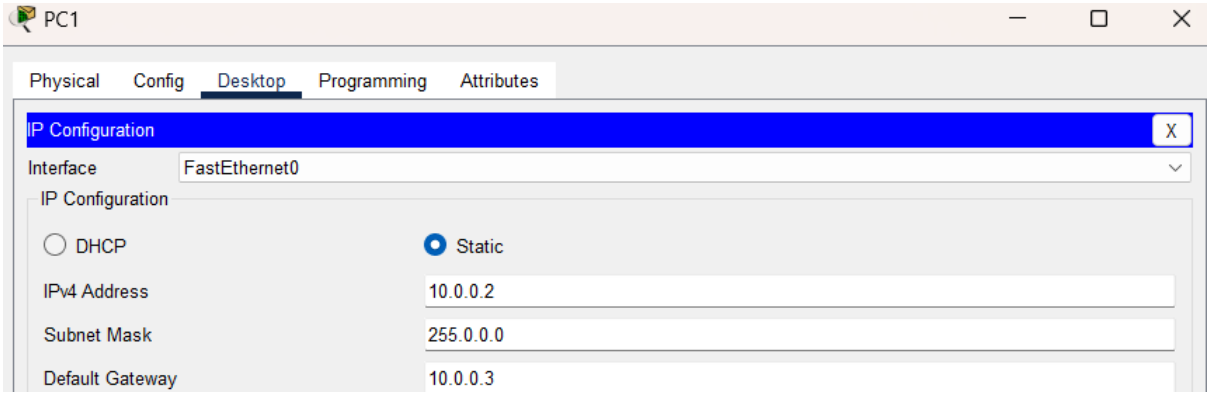
Router 0	Router 1 Router#
Router>en Router# Router#conf t Router(config)# Router(config)#int Fa0/0 Router(config-if)#ip address 10.0.0.3 255.0.0.0 Router(config-if)#no shut Router(config-if)#int Se2/0 Router(config-if)#ip address 20.0.0.1 255.0.0.0 Router(config-if)#no shut Router(config-if)#exit Router(config)# **Router(config)#ip route 30.0.0.0 255.0.0.0 20.0.0.2**	Router#conf t Router(config)# Router(config)#int Fa0/0 Router(config-if)#ip address 30.0.0.2 255.0.0.0 Router(config-if)#no shut Router(config-if)#int Se3/0 Router(config-if)#ip add 20.0.0.2 255.0.0.0 Router(config-if)#no shut Router(config-if)#exit Router(config)# **Router(config)#ip route 10.0.0.0 255.0.0.0 20.0.0.1** %Default route without gateway, if not a point-to-point interface, may impact performance Router(config)#

Step 3: Configure PCs

Assign static IP addresses to each PC.

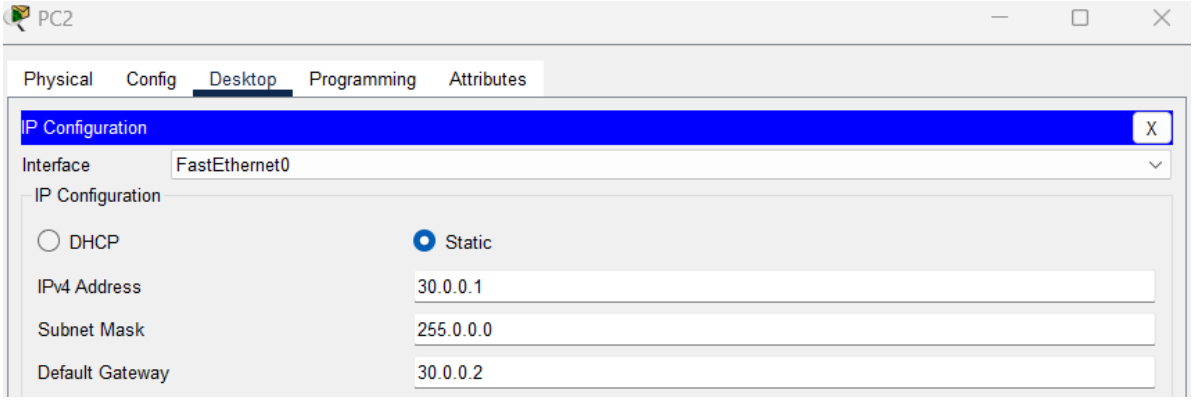
- **PC0**: 10.0.0.1, **Subnet Mask**: 255.0.0.0, **Default Gateway**: 10.0.0.3
- **PC1**: 10.0.0.2, **Subnet Mask**: 255.0.0.0, **Default Gateway**: 10.0.0.3
- **PC2**: 30.0.0.1, **Subnet Mask**: 255.0.0.0, **Default Gateway**: 30.0.0.2
- **PC3**: 30.0.0.3, **Subnet Mask**: 255.0..0.0, **Default Gateway**:30.0.0.2

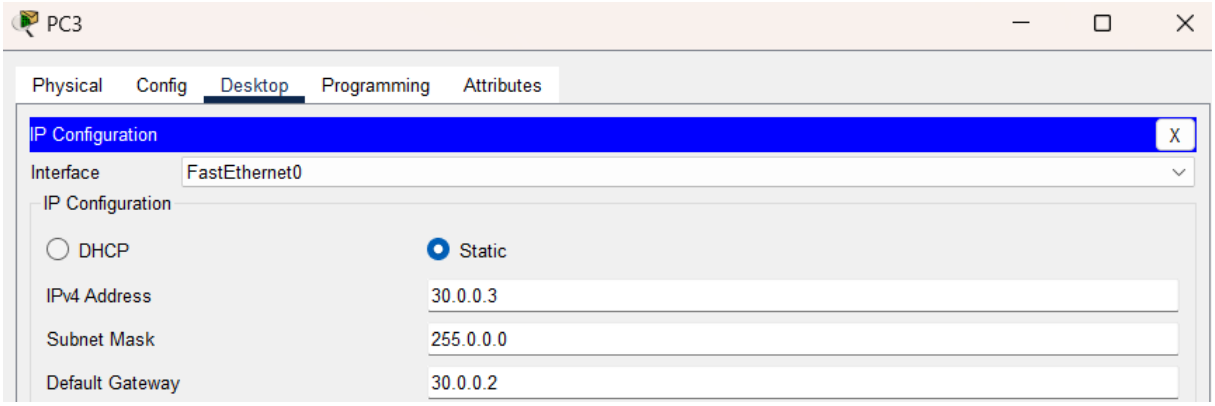


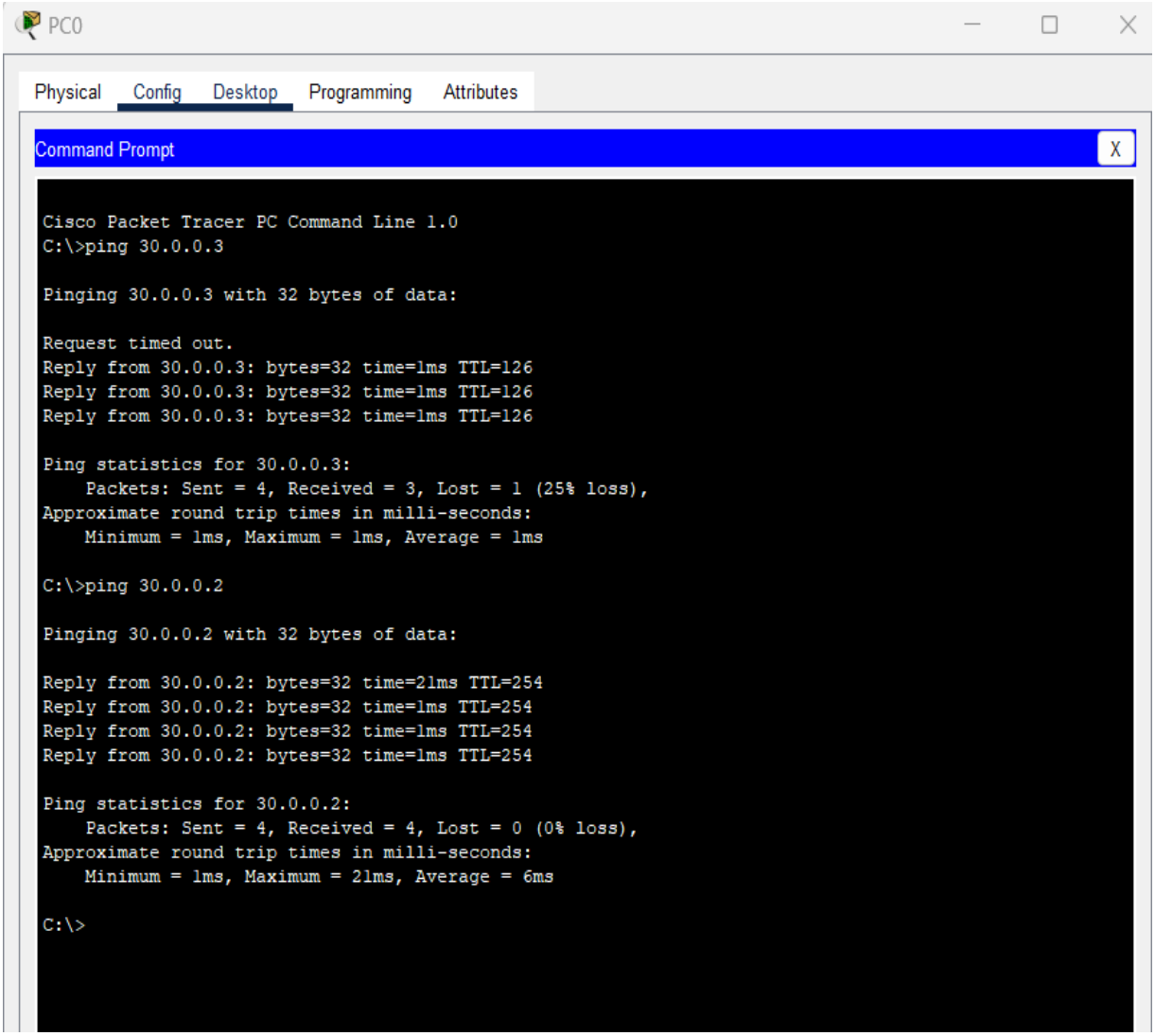


Step 4: Test Connectivity

- **Ping** from one PC to another (e.g., from **PC0** to **PC3**) to ensure static routing is functioning properly.




PC3
Physical
Config
Desktop
Programming
Attributes
IP Configuration
X
Interface
FastEthernet0
IP Configuration
DHCP
Static
IPv4 Address
30.0.0.3
Subnet Mask
255.0.0.0
Default Gateway
30.0.0.2


PC0
Physical
Config
Desktop
Programming
Attributes
Command Prompt
X
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 30.0.0.3
Pinging 30.0.0.3 with 32 bytes of data:
Request timed out.
Reply from 30.0.0.3: bytes=32 time=1ms TTL=126
Reply from 30.0.0.3: bytes=32 time=1ms TTL=126
Reply from 30.0.0.3: bytes=32 time=1ms TTL=126
Ping statistics for 30.0.0.3:
Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
Minimum = 1ms, Maximum = 1ms, Average = 1ms
C:\>ping 30.0.0.2
Pinging 30.0.0.2 with 32 bytes of data:
Reply from 30.0.0.2: bytes=32 time=21ms TTL=254
Reply from 30.0.0.2: bytes=32 time=1ms TTL=254
Reply from 30.0.0.2: bytes=32 time=1ms TTL=254
Reply from 30.0.0.2: bytes=32 time=1ms TTL=254
Ping statistics for 30.0.0.2:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 1ms, Maximum = 21ms, Average = 6ms
C:\>

Task-9

a). Configure network topology and implement dynamic routing protocol such as RIP, OSPF using Packet Tracer.

Aim: To implement dynamic routing protocols RIP, OSPF

Description:

RIP

Routing Information Protocol (RIP) is a distance-vector routing protocol used to enable routers in a network to dynamically share routing information. It's one of the simplest routing protocols and is typically used in smaller networks due to its limitations in scalability and performance.

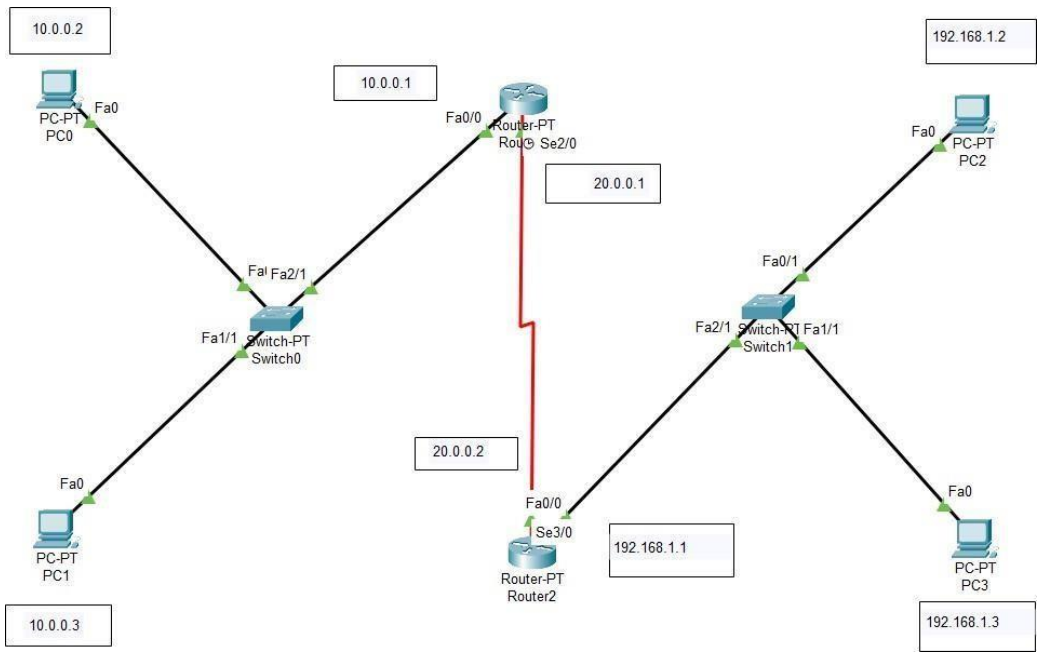
Configure Hosts

Assign IP addresses to hosts as per the diagram:

- PC0: IP 10.0.0.2, Gateway 10.0.0.1.
- PC1: IP 10.0.0.3, Gateway 10.0.0.1.
- PC2: IP 192.168.1.2, Gateway 192.168.1.1.
- PC3: IP 192.168.1.3, Gateway 192.168.1.1.

Network Overview

- Router1 (R1) connects:
  - 10.0.0.0 network on Fa0/0.
  - 20.0.0.0 network on Se2/0.
- Router2 (R2) connects:
  - 20.0.0.0 network on Se3/0.
  - 192.168.1.0 network on Fa0/0.



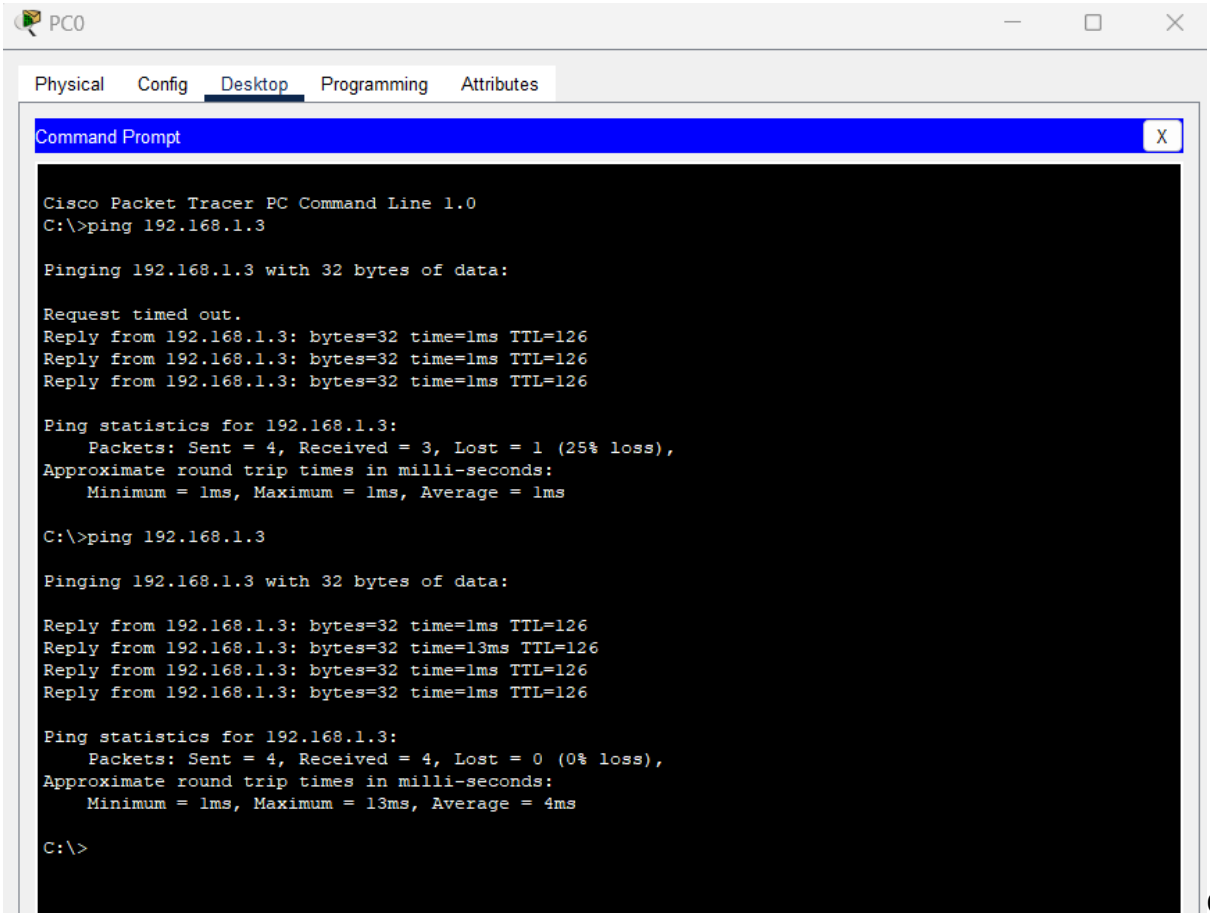
R0(Router0)	R1 (Router1)
Router 0 Router>en Router# Router#conf t Router(config)# Router(config)#int fa0/0 Router(config-if)#ip address 10.0.0.1 255.0.0.0 Router(config-if)#no shut Router(config-if)#int se2/0 Router(config-if)#ip address 20.0.0.1 255.0.0.0 Router(config-if)#no shut Router(config-if)#exit Router(config)#exit Router#write memory Router>en Router# Router#conf t Router(config)# Router(config)#router rip Router(config)#version 2 Router(config-if)#network 10.0.0.0 Router(config-if)#network 20.0.0.0 Router(config-if)#no auto-summary Router(config-if)#exit	Router 1 Router>en Router# Router#con t Router(config)# Router(config)#int fa0/0 Router(config-if)#ip address 192.168.1.1 255.0.0.0 Router(config-if)#no shut Router(config-if)#int se3/0 Router(config-if)#ip address 20.0.0.2 255.0.0.0 Router(config-if)#no shut Router(config-if)#exit Router(config)#exit Router#write memory Router>en Router# Router#conf t Router(config)# Router(config)#router rip Router(config)#version 2 Router(config-if)#network 20.0.0.0 Router(config-if)#network 192.168.1.0 Router(config-if)#no auto-summary Router(config-if)#exit

Test the Configuration:

From PC0 (10.0.0.2), ping PC3 (192.168.1.3)

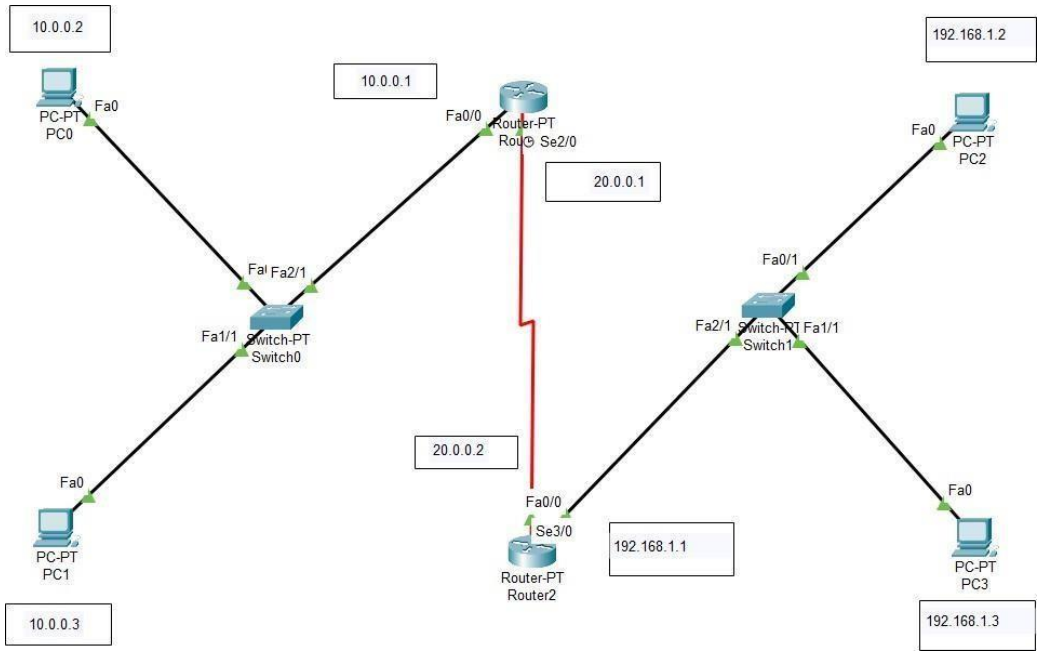
Command:

ping 192.168.1.3


PC0
Physical
Config
Desktop
Programming
Attributes
Command Prompt
X
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.3
Pinging 192.168.1.3 with 32 bytes of data:
Request timed out.
Reply from 192.168.1.3: bytes=32 time=1ms TTL=126
Reply from 192.168.1.3: bytes=32 time=1ms TTL=126
Reply from 192.168.1.3: bytes=32 time=1ms TTL=126
Ping statistics for 192.168.1.3:
Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
Minimum = 1ms, Maximum = 1ms, Average = 1ms
C:\>ping 192.168.1.3
Pinging 192.168.1.3 with 32 bytes of data:
Reply from 192.168.1.3: bytes=32 time=1ms TTL=126
Reply from 192.168.1.3: bytes=32 time=13ms TTL=126
Reply from 192.168.1.3: bytes=32 time=1ms TTL=126
Reply from 192.168.1.3: bytes=32 time=1ms TTL=126
Ping statistics for 192.168.1.3:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 1ms, Maximum = 13ms, Average = 4ms
C:\>

O

OSPF


10.0.0.2
PC-PT
PC0
Fa0
10.0.0.1
Fa0/0
Router-PT
Se2/0
20.0.0.1
192.168.1.2
Fa0
PC-PT
PC2
Fa0/1
Fa2/1
Switch-PT
Switch1
Fa1/1
Fa
Fa2/1
Fa1/1
Switch-PT
Switch0
20.0.0.2
Fa0
PC-PT
PC1
Fa0/0
Se3/0
Router-PT
Router2
192.168.1.1
Fa0
PC-PT
PC3
10.0.0.3
192.168.1.3

The OSPF stands for Open Shortest Path First. It is a widely used and supported routing protocol. It is an intradomain protocol, which means that it is used within an area or a network.
Open Shortest Path First (OSPF) is a dynamic routing protocol used in IP networks. It is classified as an Interior Gateway Protocol (IGP), meaning it works within a single autonomous system (AS). OSPF ensures that routers can dynamically learn and exchange routes with each other to create a loop-free, optimized routing table.

Configure Hosts

Assign IP addresses to hosts as per the diagram:

- PC0: IP 10.0.0.2, Gateway 10.0.0.1.
- PC1: IP 10.0.0.3, Gateway 10.0.0.1.
- PC2: IP 192.168.1.2, Gateway 192.168.1.1.
- PC3: IP 192.168.1.3, Gateway 192.168.1.1.

Network Overview

- Router1 (R1) connects:
  - 10.0.0.0 network on Fa0/0.
  - 20.0.0.0 network on Se2/0.
- Router2 (R2) connects:
  - 20.0.0.0 network on Se3/0.

192.168.1.0 network on Fa0/0.

R0(Router0)	R1 (Router1)
Router 0 Router>en Router# Router#conf t Router(config)# Router(config)#int fa0/0 Router(config-if)#ip address 10.0.0.1 255.0.0.0 Router(config-if)#no shut Router(config-if)#int se2/0 Router(config-if)#ip address 20.0.0.1 255.0.0.0 Router(config-if)#no shut Router(config-if)#exit Router(config)#exit Router#write memory Router>en Router# Router#conf t Router(config)#	Router 1 Router>en Router# Router#con t Router(config)# Router(config)#int fa0/0 Router(config-if)#ip address 192.168.1.1 255.0.0.0 Router(config-if)#no shut Router(config-if)#int se3/0 Router(config-if)#ip address 20.0.0.2 255.0.0.0 Router(config-if)#no shut Router(config-if)#exit Router(config)#exit Router#write memory Router>en Router# Router#conf t Router(config)#

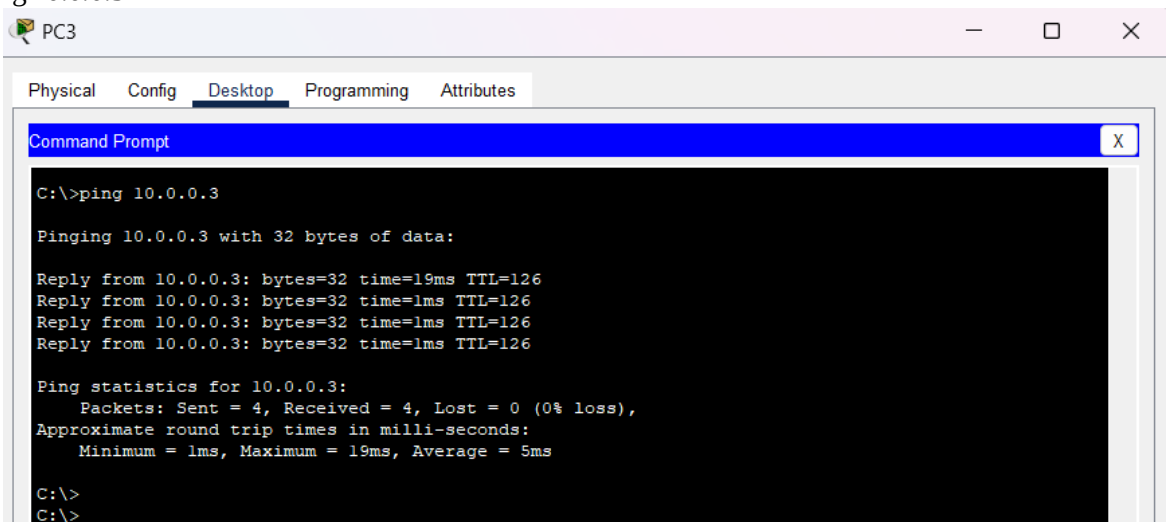
Router(config)#router ospf 1 Router(config-if)#network 10.0.0.0 0.0.0.255 area 0 Router(config-if)#network 20.0.0.0 0.0.0.255 area 0 Router(config-if)#ex Router(config)#ex	Router(config)#router ospf 1 Router(configif)#network 192.168.1.0 0.0.0.255 area 1 Router(config-if)#network 20.0.0.0 0.0.0.255 area 0 Router(config-if)#exit Router(config)#exit

Test the Configuration:

From PC3 (10.0.0.2), ping PC0 (192.168.1.3)

Command:

ping 10.0.0.3



Task 10

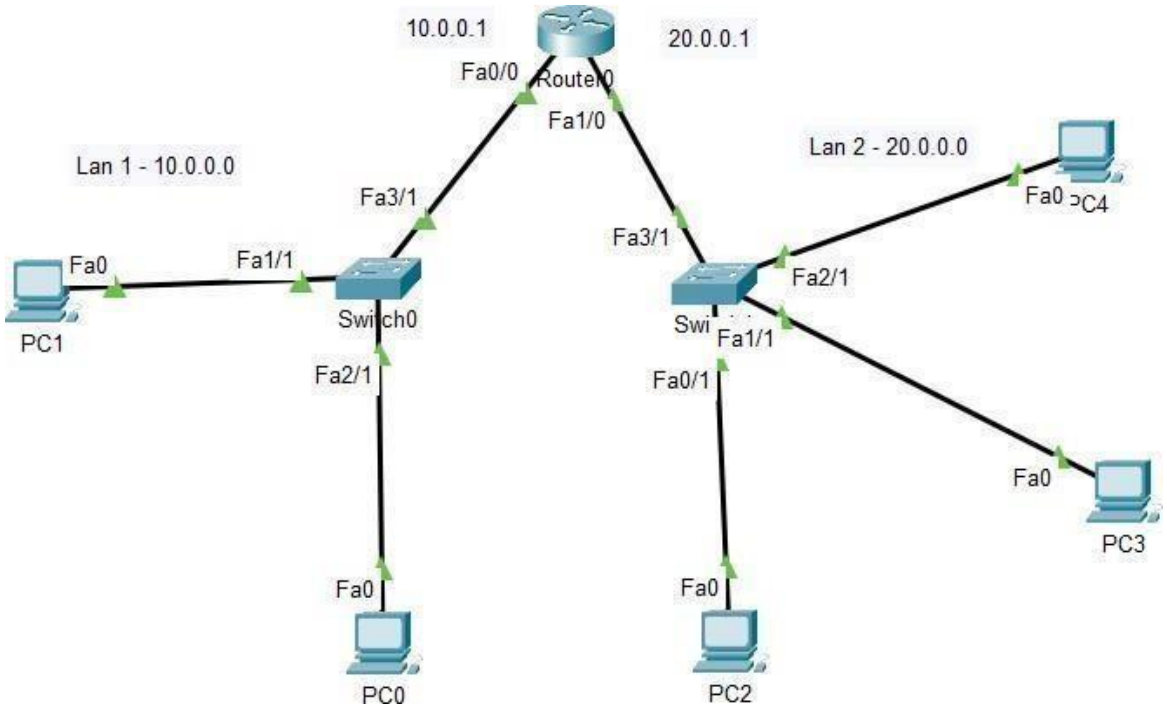
a) Configure DHCP Server in the Network using packet tracer software.

Aim: To implement DHCP server using packet tracer

Description: A DHCP (Dynamic Host Configuration Protocol) server is a network server that automatically assigns IP addresses to devices on a network. It also provides other network configuration parameters, such as default gateways and host names.

Program:

DHCP server:



Step 1: Build network (**don't assign IP's to the PCs)**

Step 2: Configure the Router for DHCP

Router>en

Router#conf t

Enter configuration commands, one per line. End with CNTL/Z.

// **Define DHCP Pools for Each Network**://

//For **10.0.0.0** network//

Router(config)#ip dhcp pool lan1

Router(dhcp-config)#network 10.0.0.0 255.0.0.0
Router(dhcp-config)#default-router 10.0.0.1
Router(dhcp-config)#dns-server 8.8.8.8
Router(dhcp-config)#
//For **20.0.0.0** network//
Router(dhcp-config)#ip dhcp pool lan2
Router(dhcp-config)#network 20.0.0.0 255.0.0.0
Router(dhcp-config)#default-router 20.0.0.1
Router(dhcp-config)#dns-server 8.8.8.8
Router(dhcp-config)#

//**Exclude Router IPs from DHCP Pool**: Prevent the router's interface IPs from being allocated by DHCP.//

Router(dhcp-config)#ip dhcp excluded-address 10.0.0.1
Router(config)#ip dhcp excluded-address 20.0.0.1
Router(config)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console
Router#wr memory
Building configuration...
[OK]

Step 3: Configure Router Interfaces

Router#
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.

//Assign IP addresses to the router interfaces://

Router(config)#int fa0/0
Router(config-if)#ip add 10.0.0.1 255.0.0.0
Router(config-if)#no shut
Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to
up
Router(config-if)#int fa1/0
Router(config-if)#ip add 20.0.0.1 255.0.0.0
Router(config-if)#no shut
Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet1/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet1/0, changed state to
up
Router(config-if)#
Router(config-if)#ex
Router(config)#ex
Router#
%SYS-5-CONFIG_I: Configured from console by console

Step 4: Configure PCs to Use DHCP

1. Click on each PC (**PC0, PC1, PC2, etc.**).
2. Go to the **Desktop** tab.
3. Open the **IP Configuration** window.

4. Select **DHCP**. The PC should automatically receive an IP address from the router.

Step 5: Verify the Configuration

Go back to the router's CLI and verify DHCP bindings:

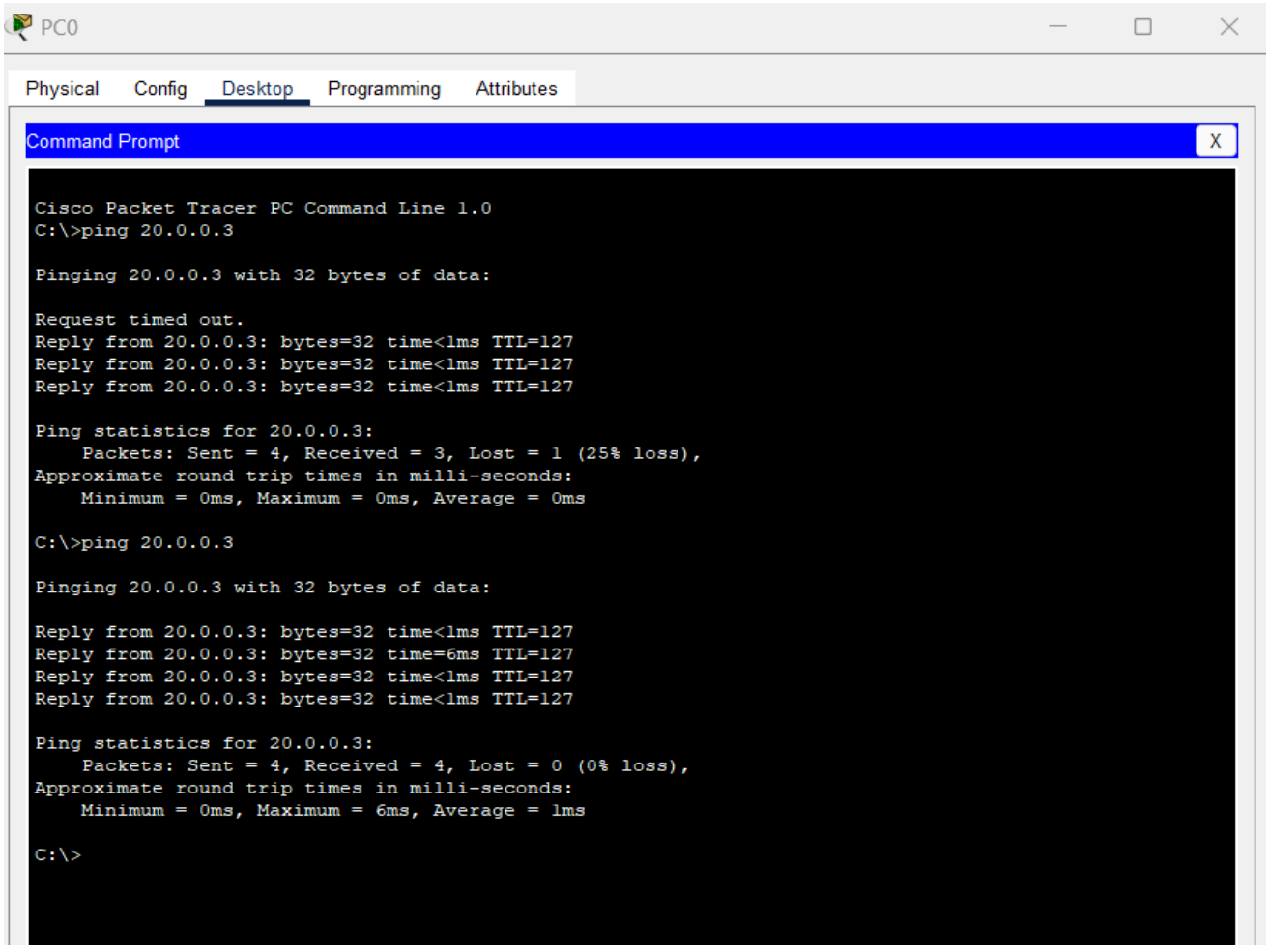
This will display the list of IPs leased to PCs.

Router#sh ip dhcp binding

IP address Client-ID/ Lease expiration Type
Hardware address
10.0.0.2 0060.2F70.9394 -- Automatic
10.0.0.3 0090.21AD.BB03 -- Automatic
20.0.0.2 00E0.F739.0313 -- Automatic
20.0.0.3 0003.E413.8ADD -- Automatic
20.0.0.4 0001.4237.29AD -- Automatic
Router#

Step 6: Test Connectivity

ping 20.0.0.3 (from PC0)



Note: Check the pc's to see the automatically assigned ip addresses

10 b) Configure a remote login using SSH and Telnet.

Aim: To implement remote login using SSH and Telnet.

Description: Remote login using SSH" refers to securely connecting to a remote computer and accessing its command line interface through the Secure Shell (SSH) protocol, which encrypts all data transmitted, while "remote login using Telnet" means connecting to a remote computer using the Telnet protocol

Program:



First connect the console cable from PC0 RS232 port to Switch console port.

From PC0 Terminal Switch> Switch>en Switch#conf terminal Switch(config)# Switch(config)#interface vlan 1 Switch(config-if)#ip address 10.0.0.2 255.0.0.0 Switch(config-if)#no shut Switch(config-if)#end Switch# Switch#sh run Before Telnet configuration Check in command prompt *C:\>telnet 10.0.0.2* *Trying 10.0.0.2 ...* *% Connection timed out; remote host not responding* *C:\>telnet 10.0.0.2* *Trying 10.0.0.2 ...Open* *[Connection to 10.0.0.2 closed by foreign host]*	Switch# Switch#conf t Switch(config)# Switch(config)#username myname password mypass Switch(config)#line vty 0 15 Switch(config-line)#login local Switch(config-line)#transport input telnet Switch(config-line)# After Telnet configuration C:\>telnet 10.0.0.2 Trying 10.0.0.2 ...Open User Access Verification Username: myname Password: Switch>

SSH

Switch#conf t
Switch(config)#
Switch(config)#hostname myswitch
myswitch(config)#ip domain-name mydomain
myswitch(config)#username myname1 password mypass1
myswitch(config)#line vty 0 15
myswitch(config-line)#login local
myswitch(config-line)#transport input ssh
myswitch(config-line)#exit
myswitch(config)#crypto key generate rsa

The name for the keys will be: myswitch.mydomain
Choose the size of the key modulus in the range of 360 to 2048 for your General Purpose Keys. Choosing a key modulus greater than 512 may take a few minutes.

How many bits in the modulus [512]:
% Generating 512 bit RSA keys, keys will be non-exportable...[OK]

myswitch(config)#exit

Now test SSH connection

C:\>ssh -l myname1 10.0.0.2
Password:

myswitch>

Task 11

Task 11 a). Establish a webserver connection using the PCs browser.

Aim: To establish a webserver connection using browser

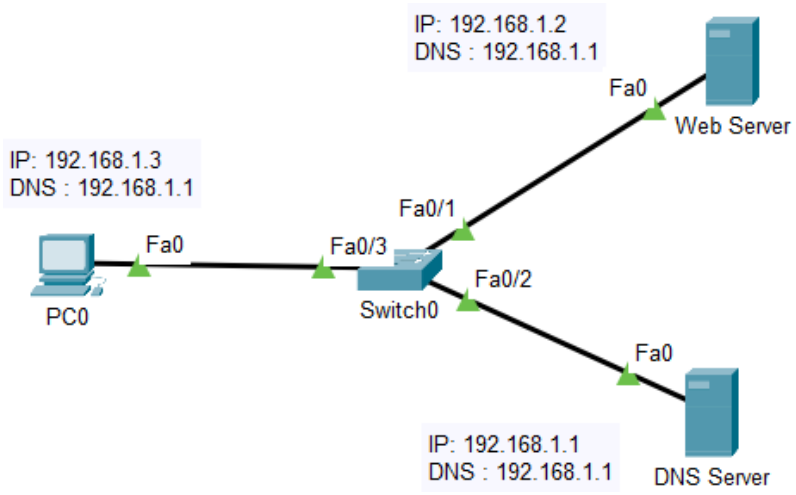
Description:

Webserver

- Hosts and delivers web content (e.g., HTML pages, images, files) to clients like web browsers.
- Uses the HTTP/HTTPS protocol.
- Example: When you enter http://example.com in a browser, the web server
- serves the content (like index.html).

DNS server

- Translates human-readable domain names (e.g., example.com) into IP addresses (e.g., 192.168.1.2), so devices can locate the correct server.
- Example: When you type example.com, the DNS server tells the browser which IP address (e.g., 192.168.1.2) to connect to.

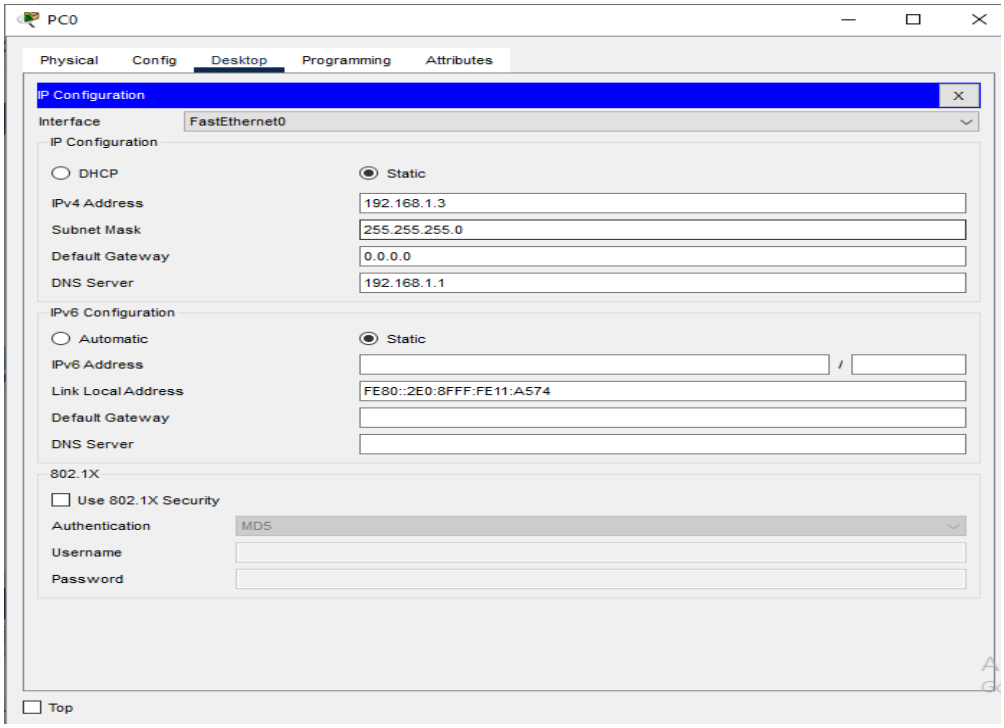


Follow the below steps:

Step 1:

Configure IP addresses

PC0 IP:192.168.1.3 DNS server IP:192.168.1.1



Take **normal server not meraki server from the list of end devices** and name it as webserver

Webserver IP:192.168.1.2 DNS server:192.168.1.1

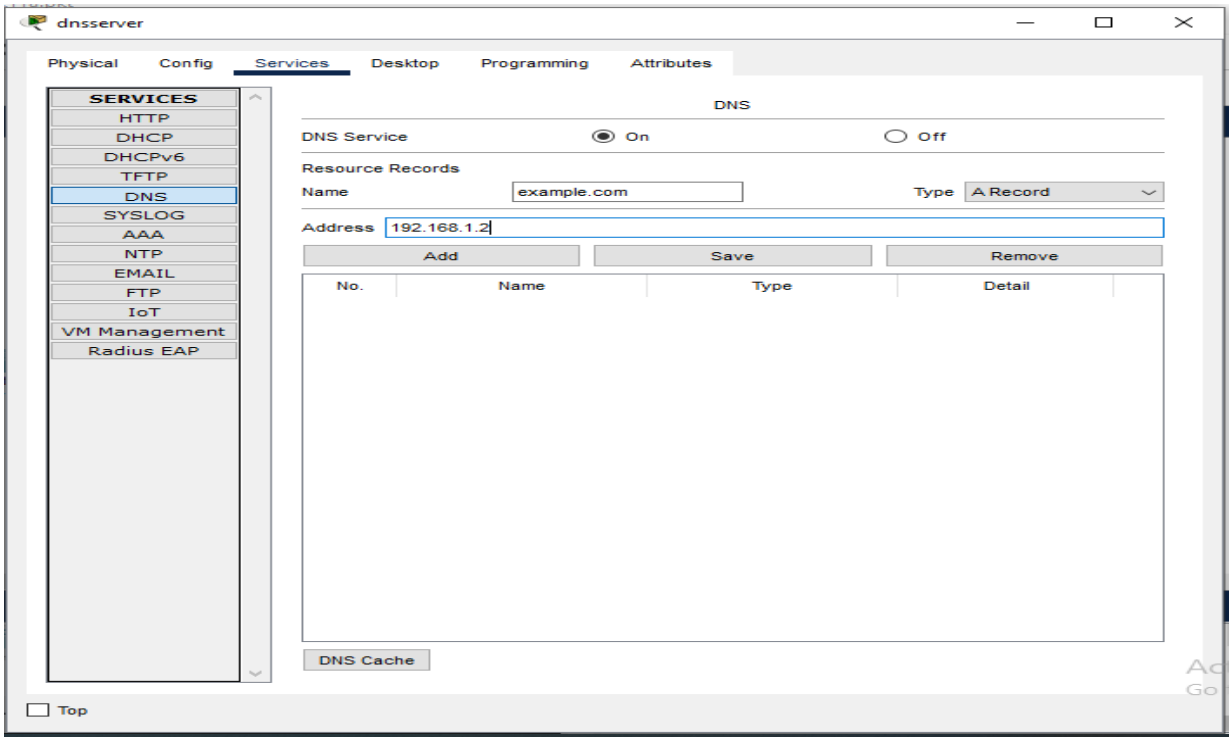
Take one more server from the list and name it as DNS server

DNS server IP:192.168.1.1 DNS server:192.168.1.1

Step 2: Enable DNS service on the DNS server

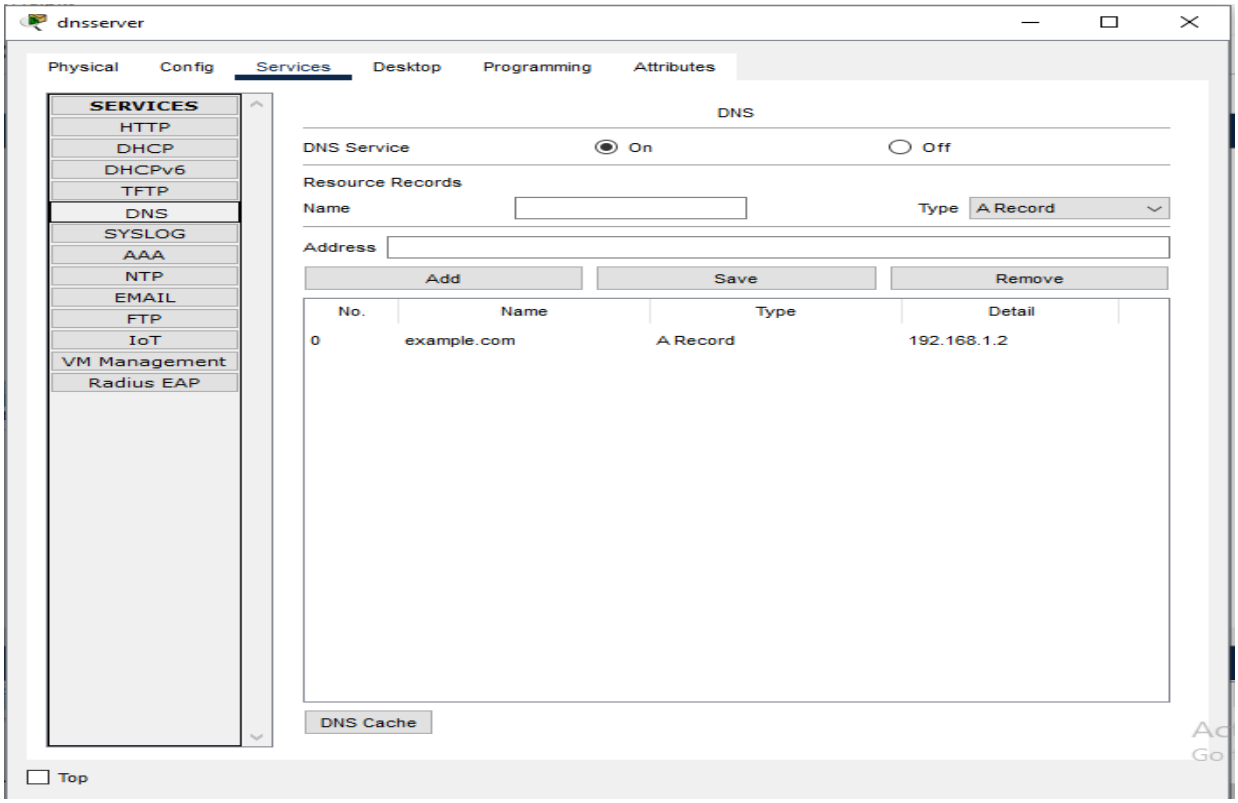
For the server in our topology to serve a webpage, when requested with a domain name instead of an IP address, we need to enable DNS service and add a DNS record.

Goto->dnsserver->services->DNS and give as shown in the screenshot



In the above image, we made the IP address; 192.168.1.2 which is the IP address of the webserver to resolve to example.com

Click on add button and the record will be added as shown in below screenshot

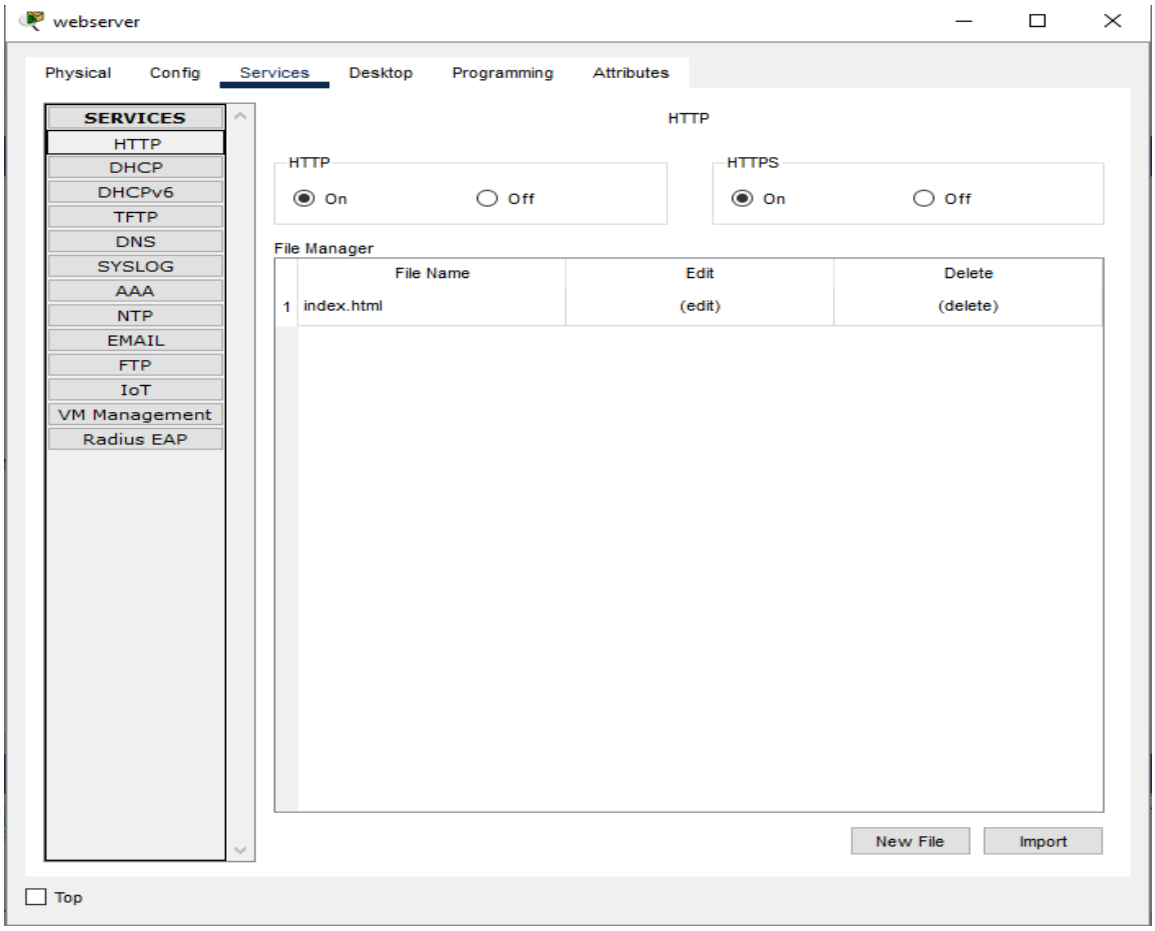


Step 3: Enable HTTP service on the webserver

HTTP/HTTPS allows a client to send and receive data (a webpage) between the webserver and the client's browser.

To Enable HTTP service on the webserver, go to **services>HTTP**

Click the on checkbox for both http and https



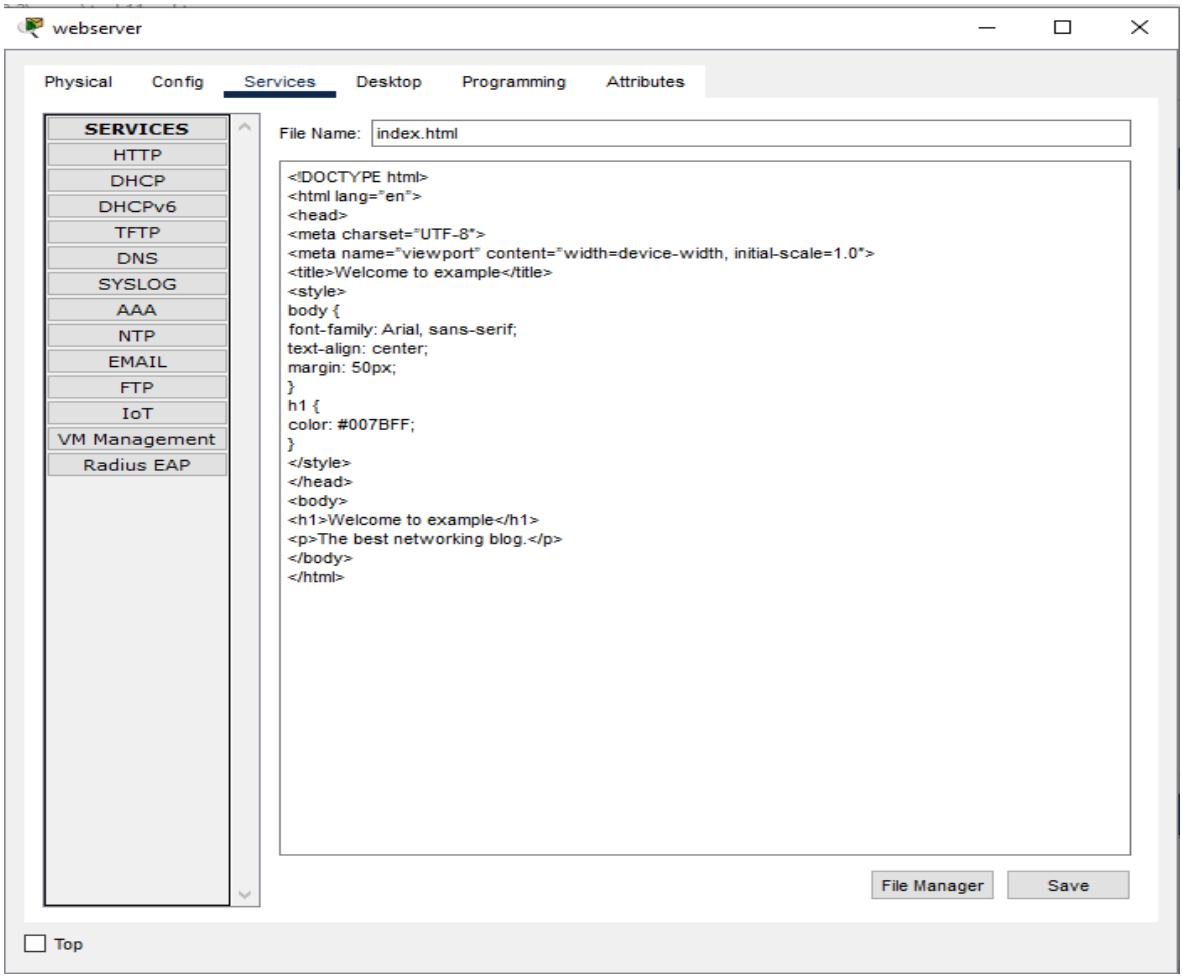
In http tab->we have up to five default webpages that can be served by the webserver. Those are created by Cisco; you can customize them or delete them. In this demonstration, I will delete every other webpage except "index.html." **as we will write our code to create a webpage to test the functionality.**

Step 4: Create the webpage

Edit the index.html code by clicking on **edit** button

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Welcome to example</title>
<style>
body {
font-family: Arial, sans-serif;
```

```
text-align: center;
margin: 50px;
}
h1 {
color: #007BFF;
}
</style>
</head>
<body>
<h1>Welcome to example</h1>
<p>The best networking blog.</p>
</body>
</html>
```



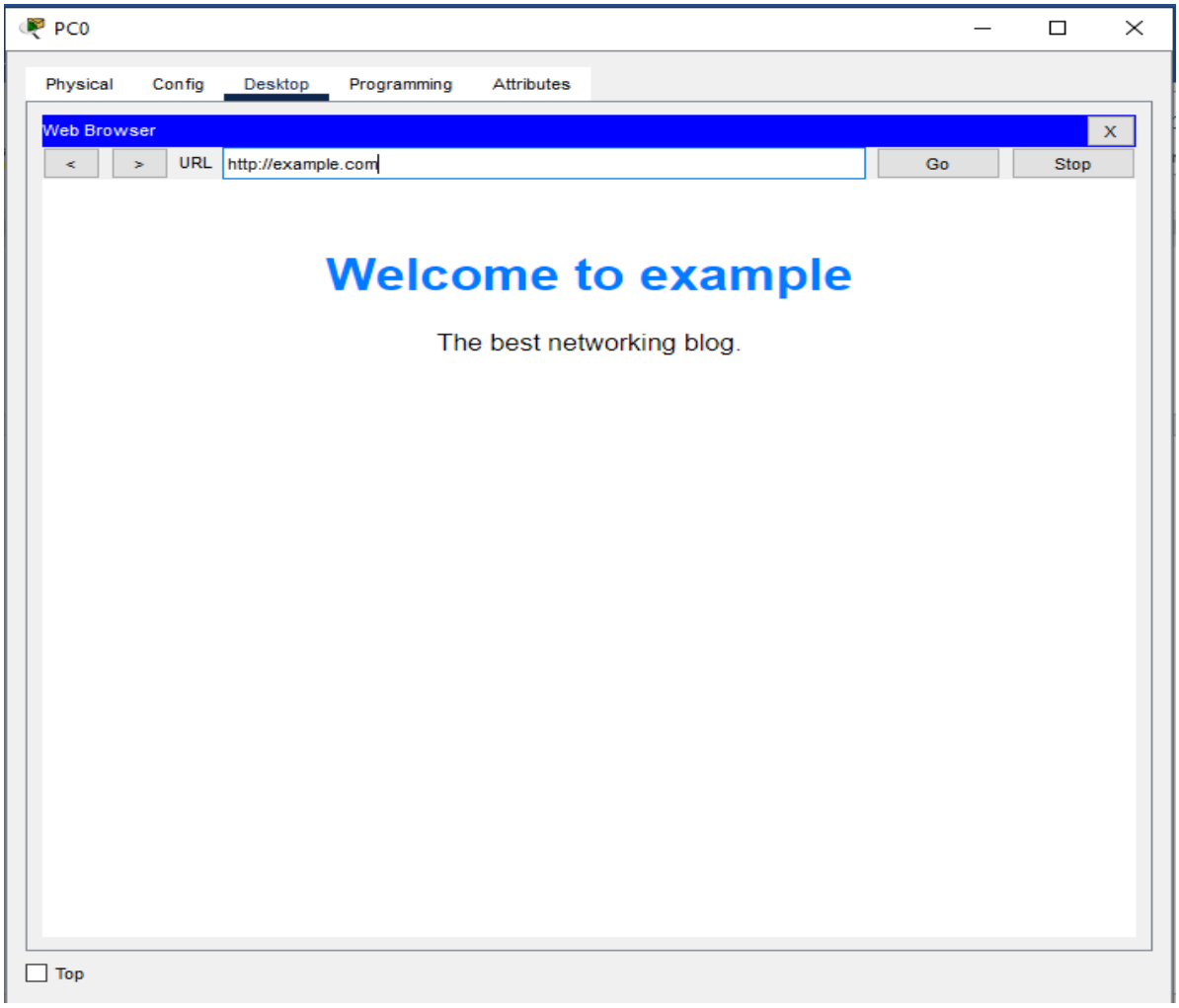
Step 5: View the Webpage

Now that we have enabled DNS service on dns server and HTTP service on the webserver, we can view the webpage we created using an HTTP request and a domain name instead of an IP address.

Test the functionality

Go to Services> webbrowser on PCO.

Then enter the domain address we added to the record of the DNS server. i.e example.com



As shown above, the webpage we created was displayed by typing example.com on the web browser rather than the IP address of the web server, which is 192.168.1.2

Test2:

We can also see server and address and domain name and address allocated

In PC0->goto desktop->command prompt

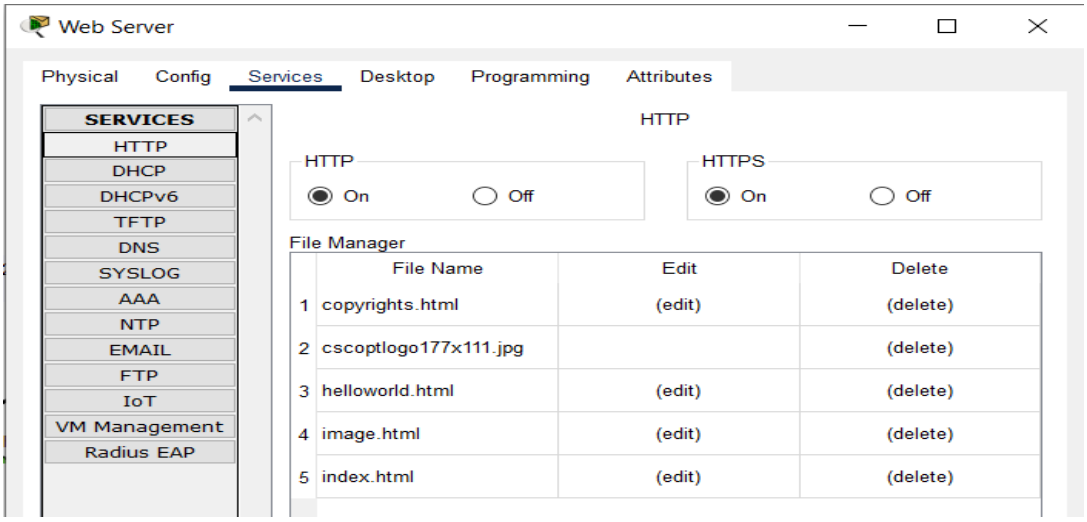
```
C:\>nslookup example.com

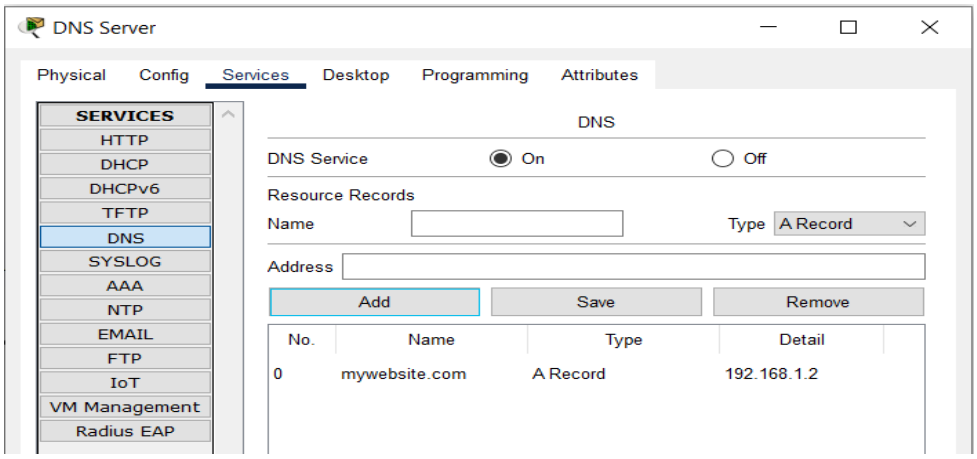
Server:  [192.168.1.1]
Address:  192.168.1.1

Non-authoritative answer:
Name:    example.com
Address:   192.168.1.2
```

Note: We can also use one server to configure both http and DNS configurations

Configure the webserver connection and DNS server





11b) Install wireshark and view ii). Network Traffic and iii).Examine ethernet frames View Wired and Wireless NIC information.

Aim: To check network traffic and examine Ethernet frames

Description:

a. Capture and analyse local ICMP data in wireshark

b. Capture and analyse Remote ICMP data in wireshark

Install wireshark latest version.

Step 1: Retrieve your PC interface addresses.

For this lab, you will need to retrieve your PC IP address and its network interface card (NIC) physical address, also called the MAC address.

a. Open a command window, type **ipconfig /all**, and then press Enter.

b. Note the IP address of your PC interface, its Description, and its MAC (physical)

address Steps

1: Open wireshark and start capturing the packets. The data lines will appear in different colors basedon protocol.

2. open command prompt and

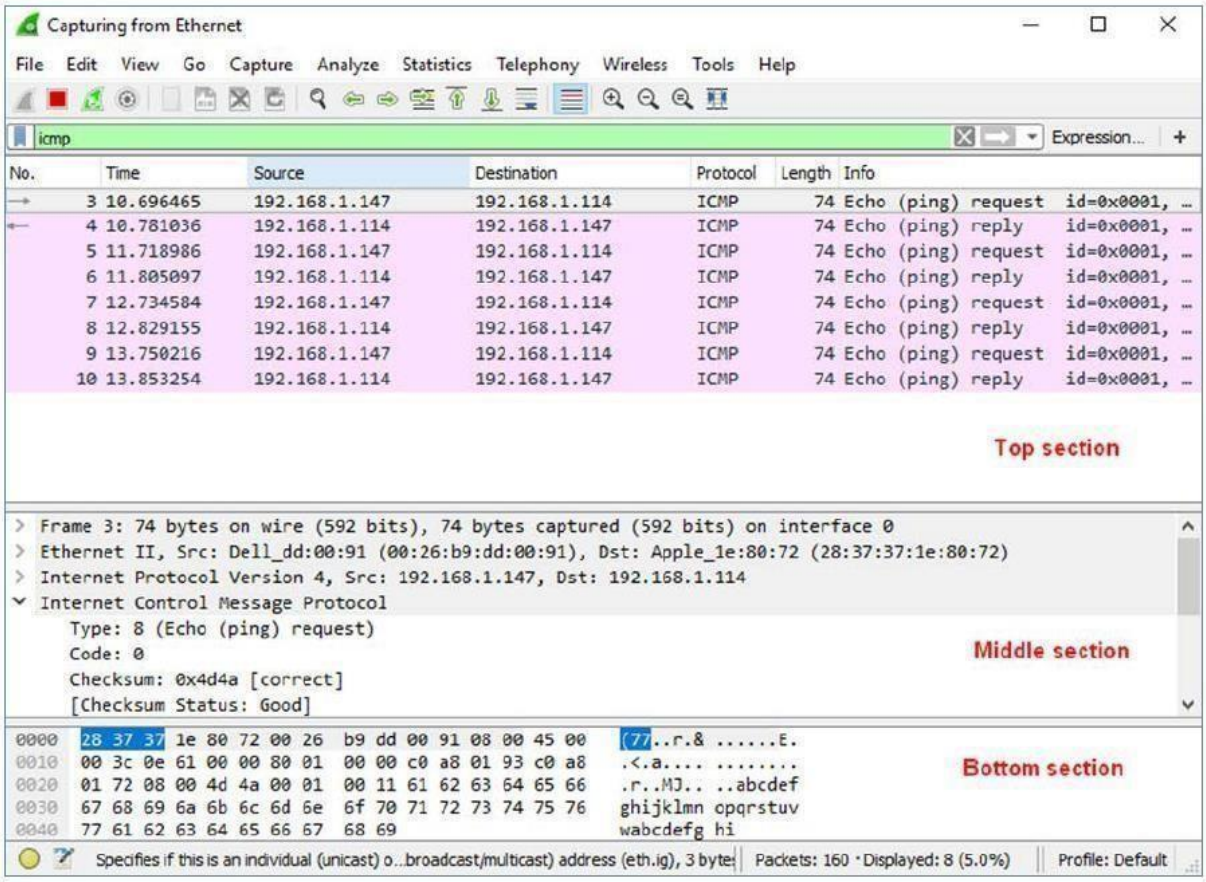
ping any url. Ex: ping google.com
ping sdc.in
ping yahoo.com
ping cisco.com

3. Stop capturing the packets.

Step 3: Examine the captured data.

Wireshark data is displayed in three sections: 1) The top section displays the list of PDU frames captured with a summary of the IP packet information listed; 2) the middle section lists PDU information for the frame selected in the top part of the screen and separates a captured PDU frame by its protocol layers; and 3) the bottom sectiondisplays the raw data of each layer. The raw data is displayed in both hexadecimal and decimal form.

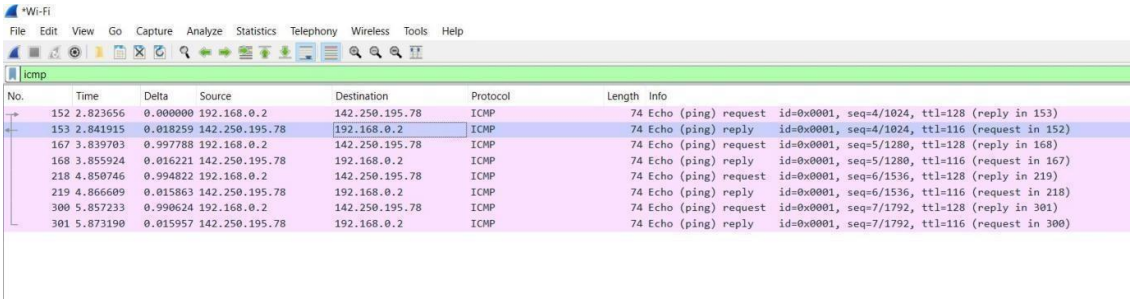
Click the first ICMP request PDU frames in the top section of Wireshark. Notice that the Source column hasyour PC IP address, and the Destination column contains the IP address you pinged.



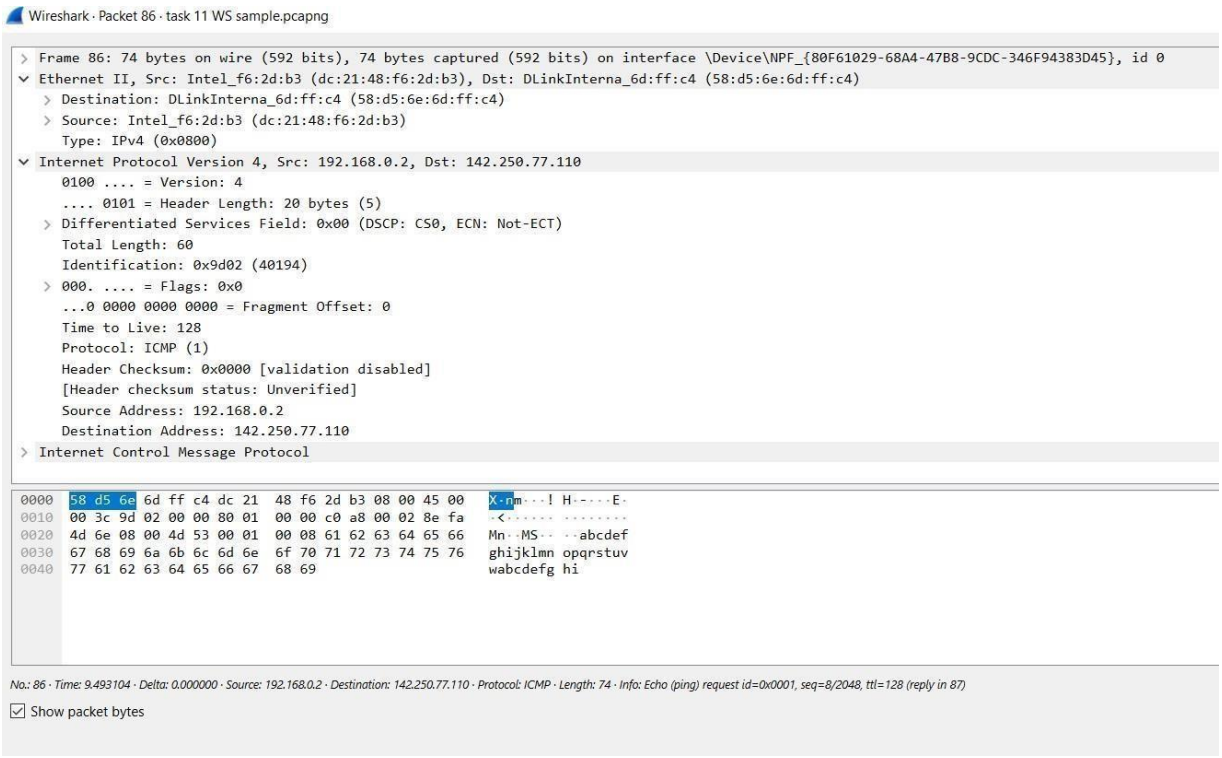
4. Go to the filter bar and type and check the following protocols

ICMP enter.
DNS
TCP
UDP
ARP

And observe the packets.



Examine the Ethernet frame fields in the middle section:



c)View Wired and Wireless NIC information:

Step 1: Use the Network and Sharing Center.

a. Open the Network and Sharing Center by clicking the Windows Start button > Control Panel > View network status and tasks under Network and Internet heading in the Category View.
b. In the left pane, click the Change adapter settings link.
c. The Network Connections window displays, which provides the list of NICs available on this PC. Look for your Local Area Connection and Wireless Network Connection adapters in this window.

Or

1. Right click on start (windows button)
Settings – status – properties

Compare with:

Open a command window prompt and type **ipconfig /all**
And observe the above addresses, both must be same.

Task-12

Task 12a). Adding IoT devices to Smart Homes using Packet Tracer

b). Connect and Monitor IoT Devices using Packet Tracer.

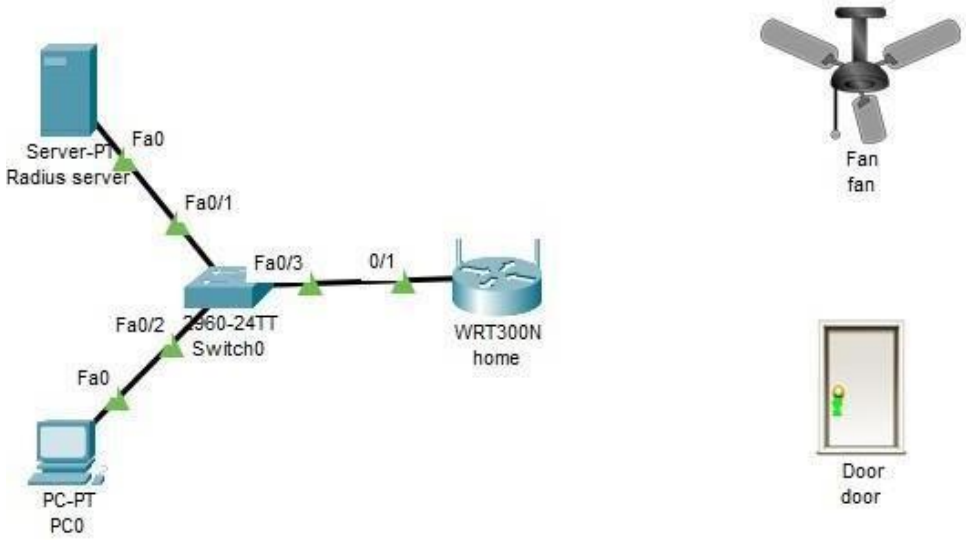
Aim: To add Iot devices to smart homes and connect them

Description: IoT Devices are physical objects embedded with sensors, software, and other technologies that enable them to connect to the internet and exchange data with other devices, systems, or users. They a part of the Internet of Things (IoT) ecosystem, designed to automate processes, enhance efficiency, and provide real- time data insights.

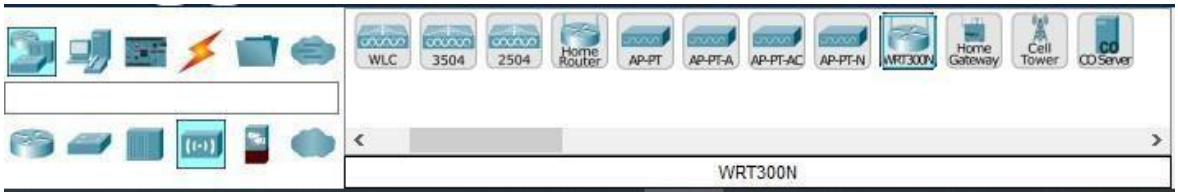
Key Features:

- **Connectivity**: Can connect to networks (Wi-Fi, Bluetooth, or wired).
- **Sensors/Actuators**: Collect data or perform actions (e.g., turning on a light, adjusting temperature).
- **Automation**: Operate automatically based on predefined conditions or commands.
- **Remote Control**: Can be controlled or monitored from anywhere via the internet.

Examples: fan, light,door,security cameras



WRT300N home router -> Under network devices->sub category 4th option (wireless devices)
Click on wireless devices->WRT300N (from last 4th option)



Iot devices (like fan,door)->Under End devices->sub category 2nd option(home)
Click on home->drag and drop fan and door



Description of the network
Radius server:

RADIUS server (Remote Authentication Dial-In User Service) can control access to IoT devices by implementing centralized **authentication, authorization, and accounting (AAA)** mechanisms
 When a user or device (e.g., PC or smartphone) attempts to control or access an IoT device (e.g., fan, door, or lamp), the access request is sent to the RADIUS server via the router.
 The RADIUS server verifies the credentials (e.g., username and password, token, or certificates) provided by the user/device.
 If the credentials are valid, the RADIUS server grants access to the network s; otherwise, it denies the request.

WRT300N router

WRT300N is a wireless router commonly used in networking simulations (like in Cisco Packet Tracer) and real-world scenarios to provide network connectivity.
Its primary purpose is to act as a gateway between connected devices and the wider network (e.g., the internet or a local network).

PC:

 The PC hosts the IoT server or monitoring software, enabling centralized control and management of connected IoT devices.
 It acts as the interface for users to monitor the status or control devices (e.g., turning a lamp on/off or unlocking the door).
IoT Devices:

Fan:

- A smart device that allows remote control of its operations, such as turning on/off and adjusting speed.

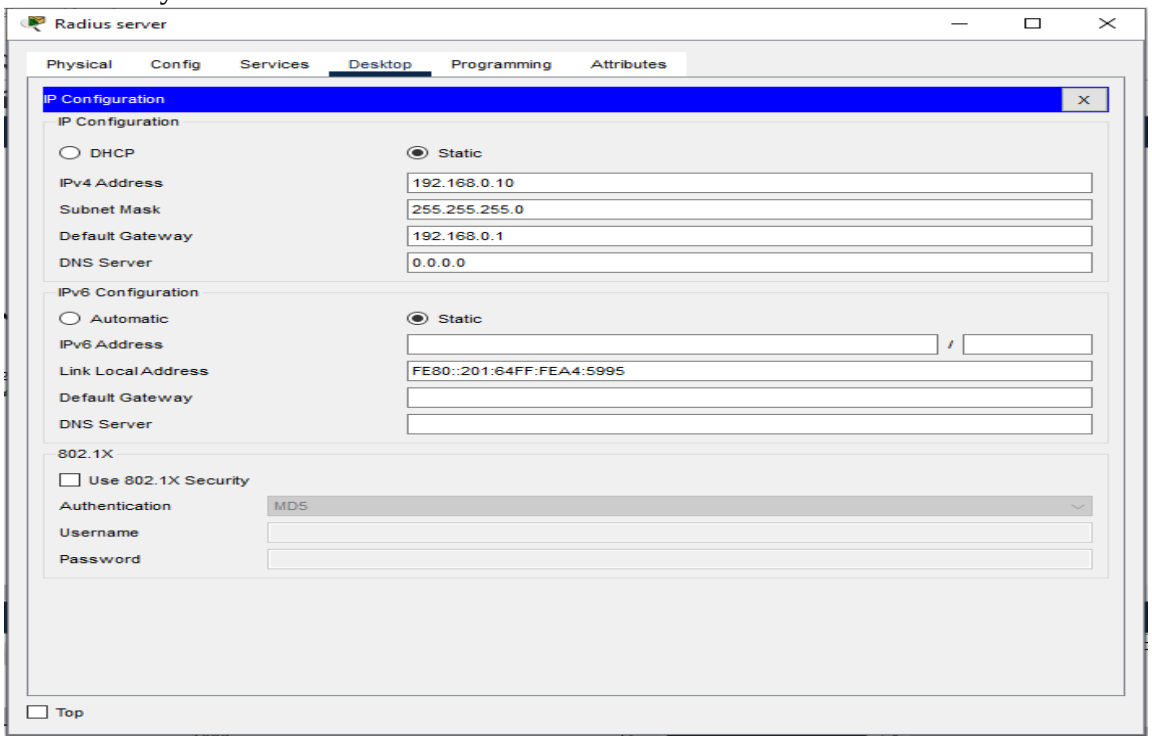
Door (Smart Lock):

- Allows secure, remote locking/unlocking of doors.
- Monitors
- door status (open/closed) for enhanced security.

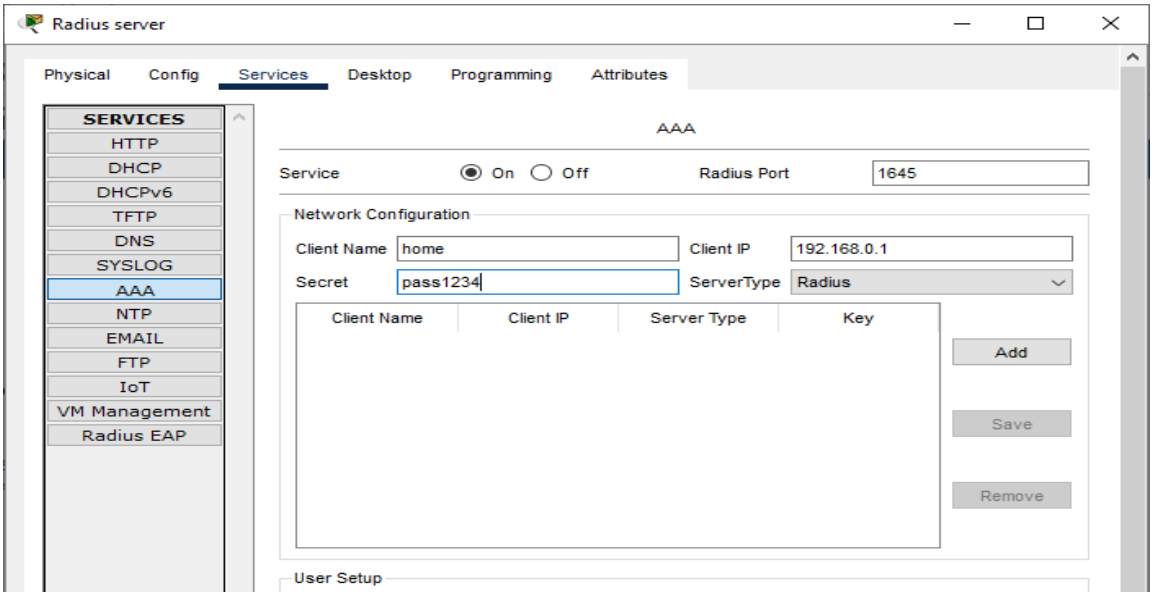
Configurations:

Configuring radium server

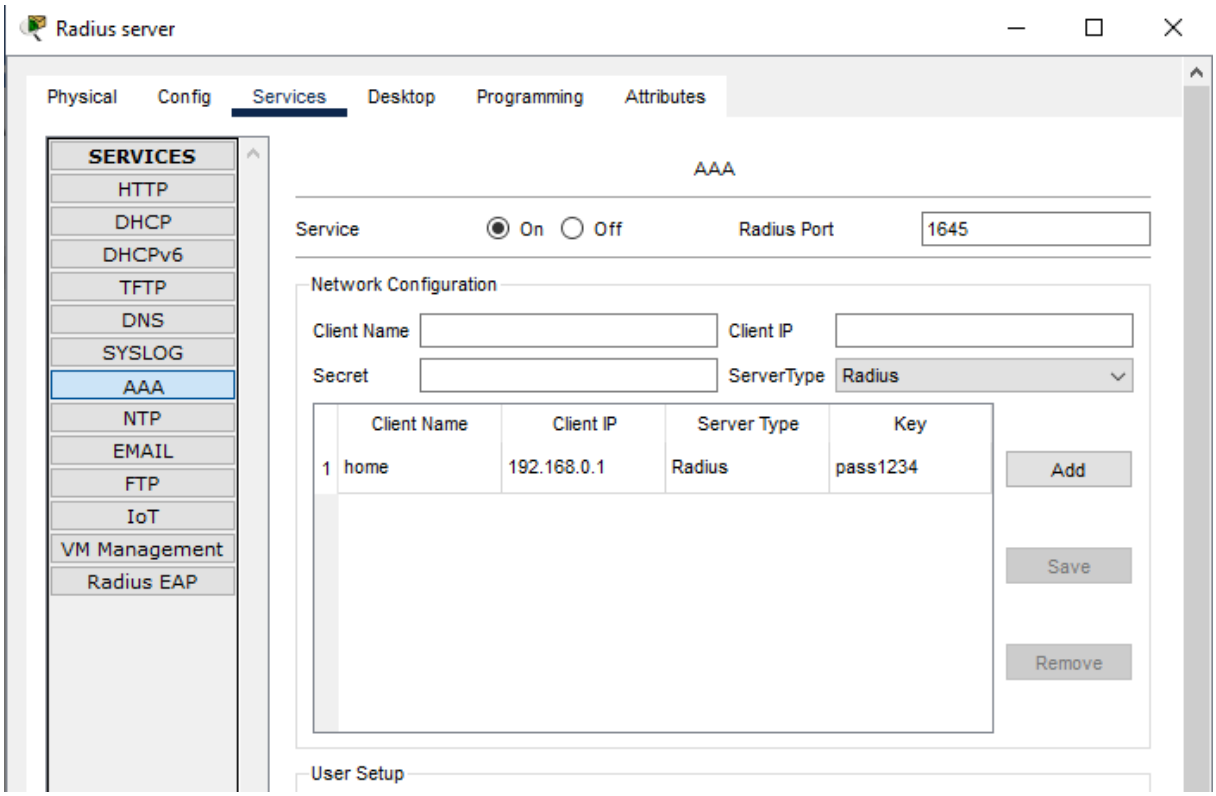
Under config->Give device name as radium server
IP address:192.168.0.10 subnet mask:255.255.255.0
Default Gateway:192.168.0.1



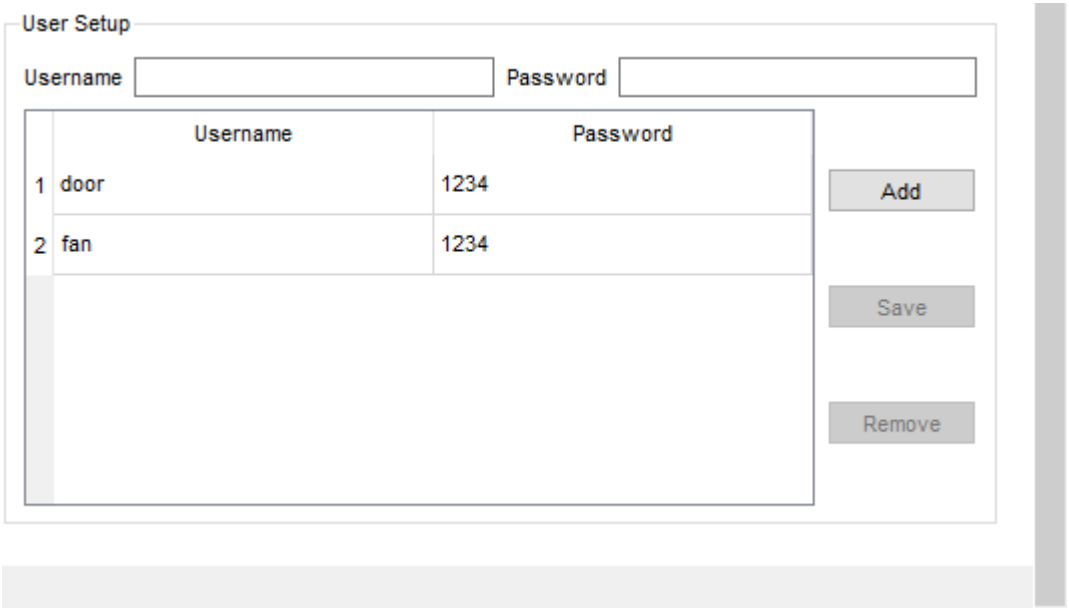
Under services click -> AAA ->click service=on radio button



Click on add and configuration gets added

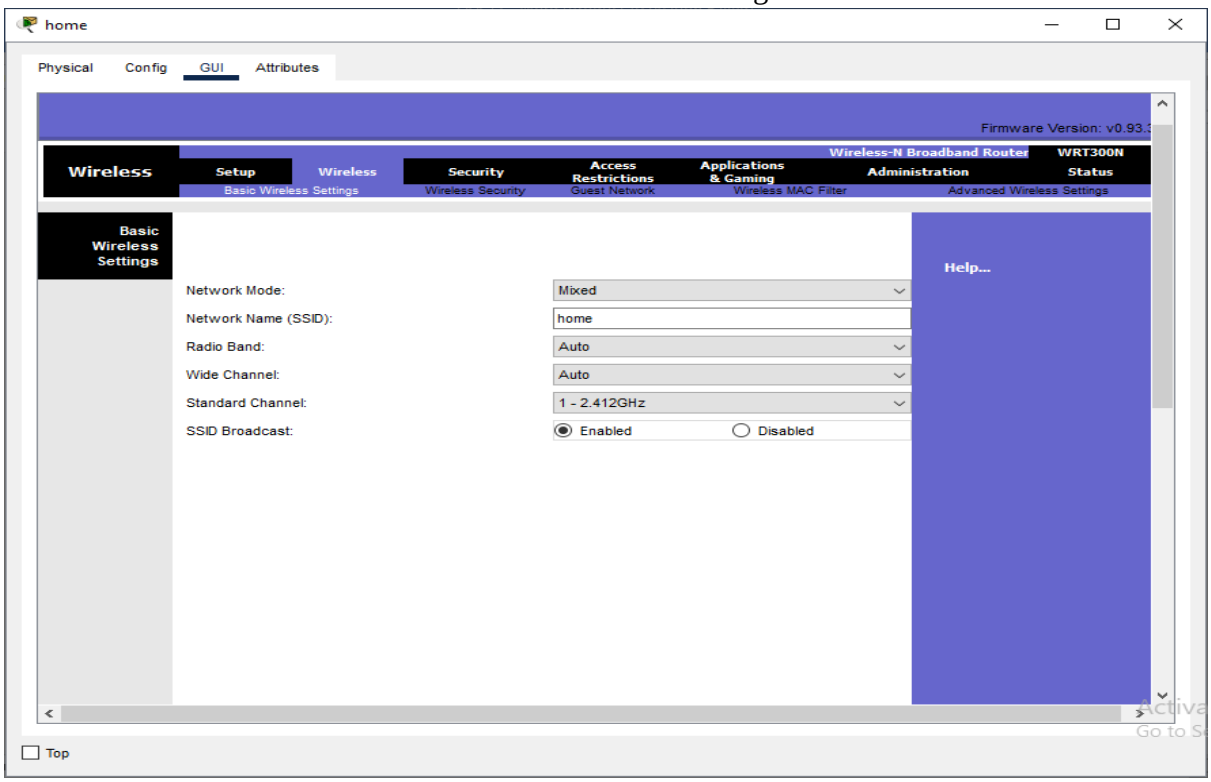


Now setup for user i.e user setup
Need to add username and password for iot devices
In username tab enter fan and in password 1234 and click add. same way do for all devices

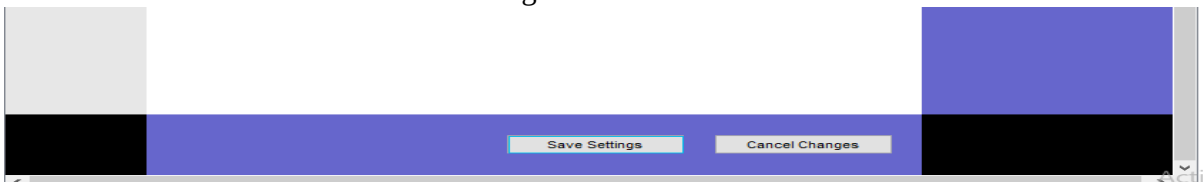


Configuration for Wireless router
Config->give display name as home

In GUI->click on wireless and enter the below shown configuration

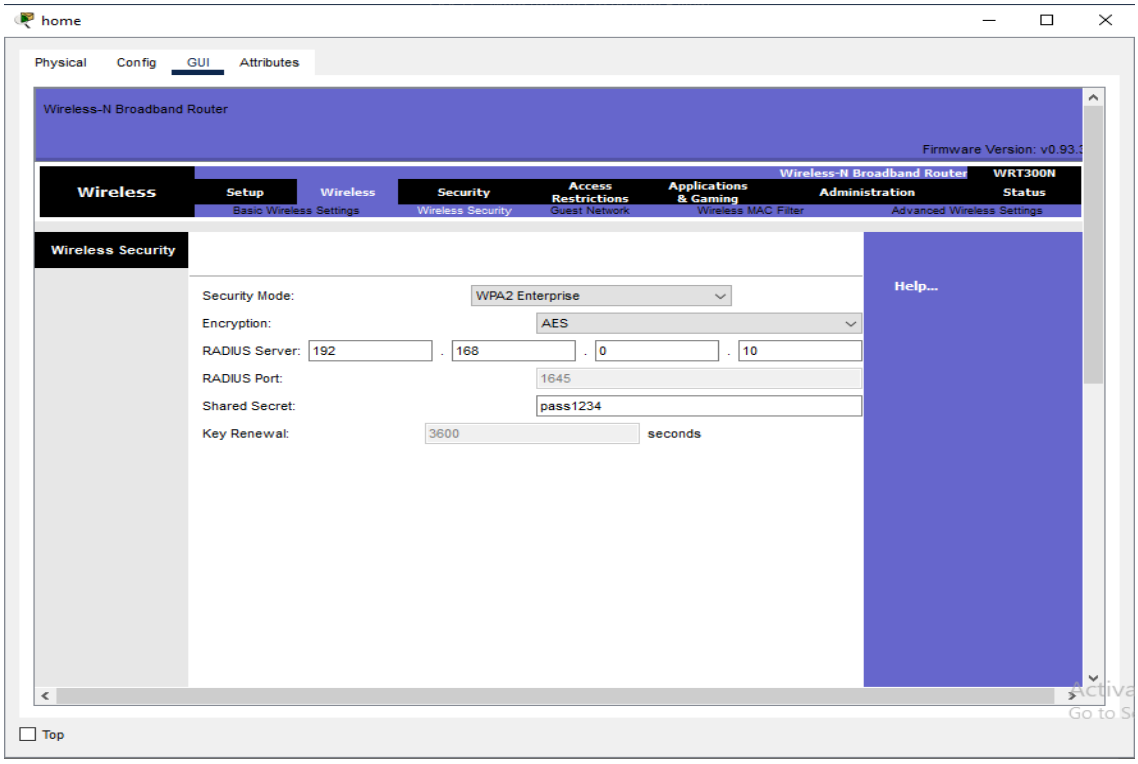


And scroll down and click on save settings

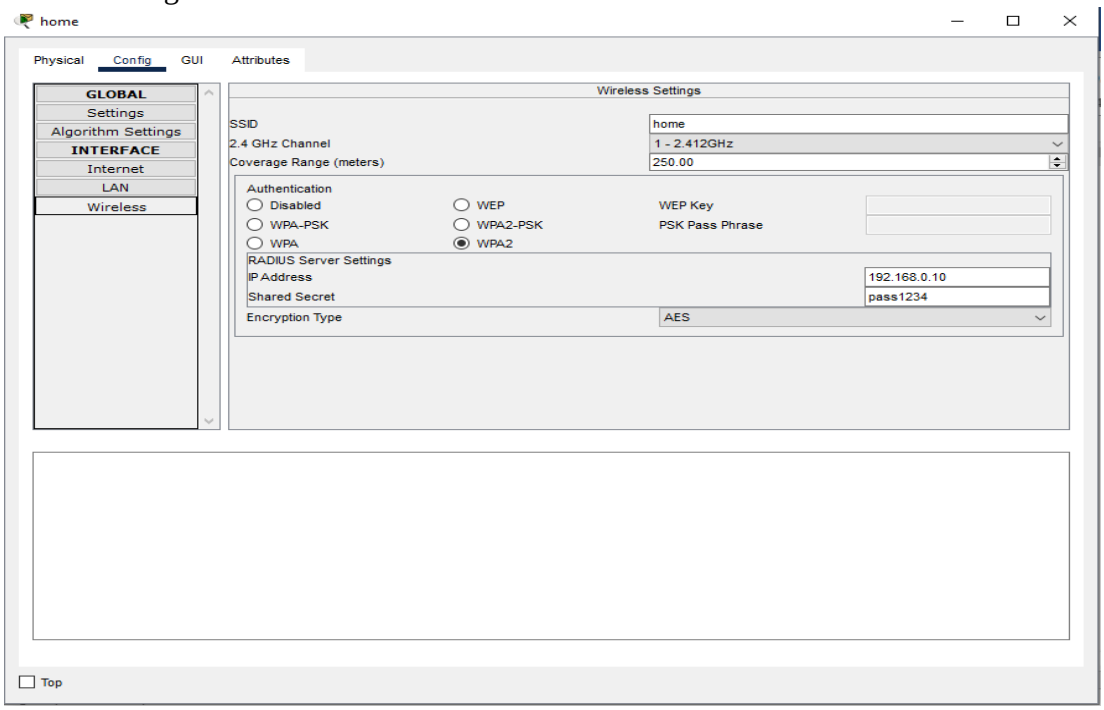


In GUI->under wireless->right side option click on security and add the below shown config



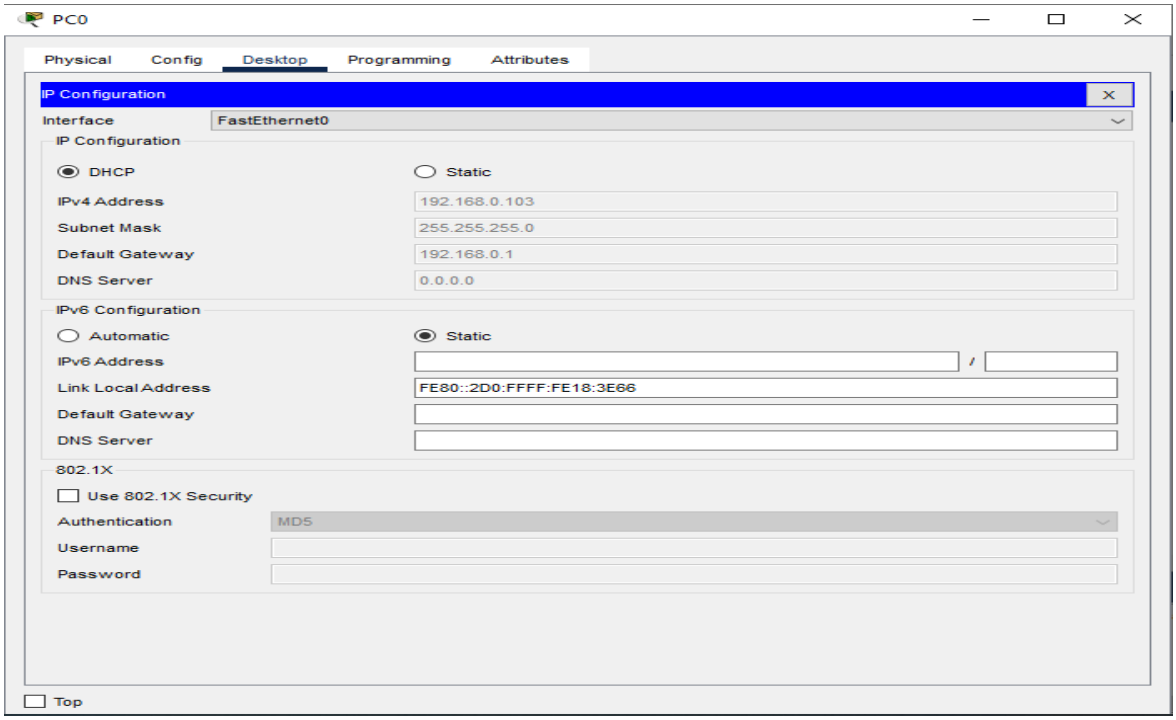


And scroll down and click on save settings
To check if all configurations which we gave are added
Goto->config in router->wireless

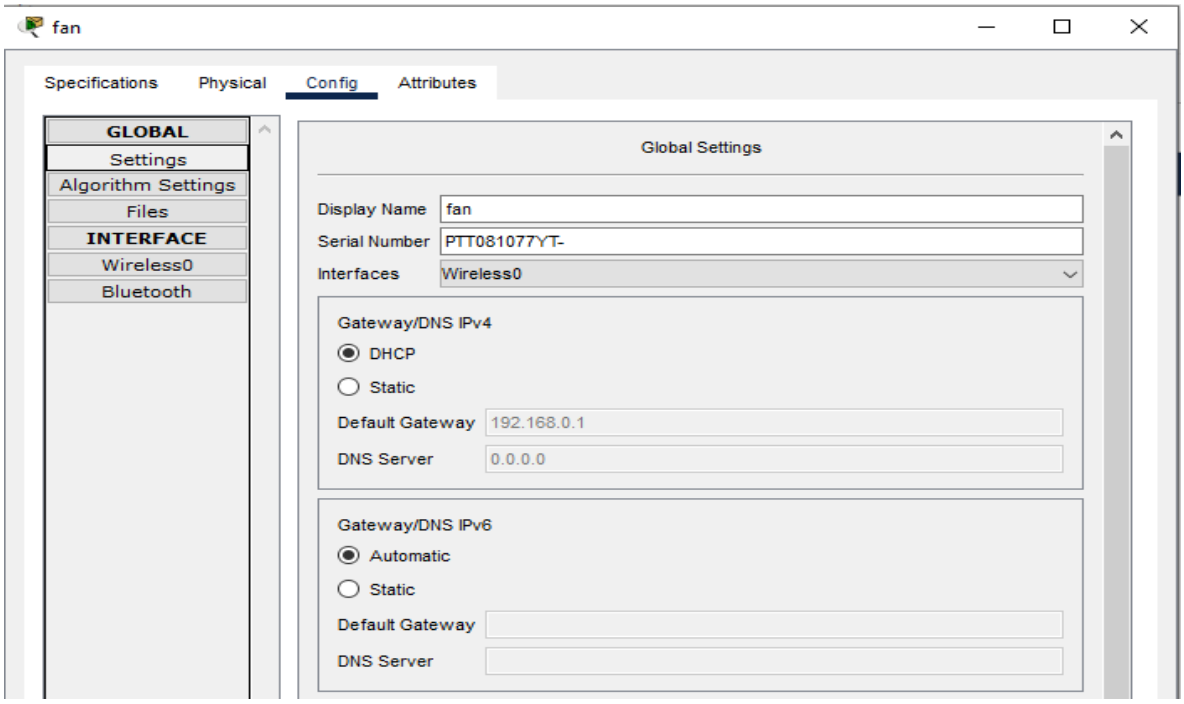


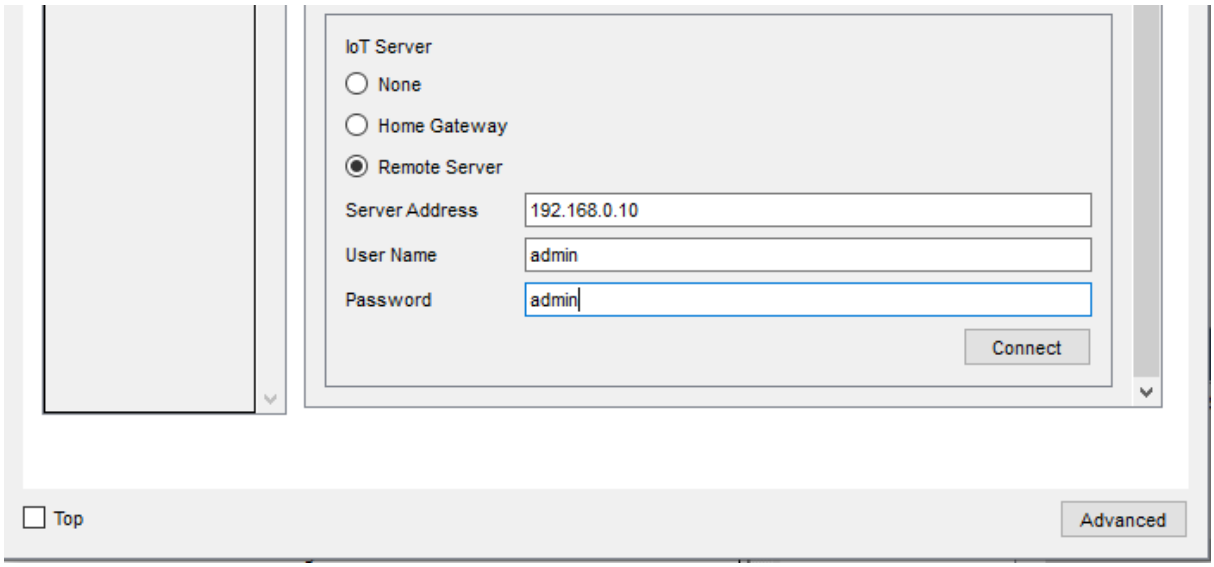
Configuration for PC

Select DHCP



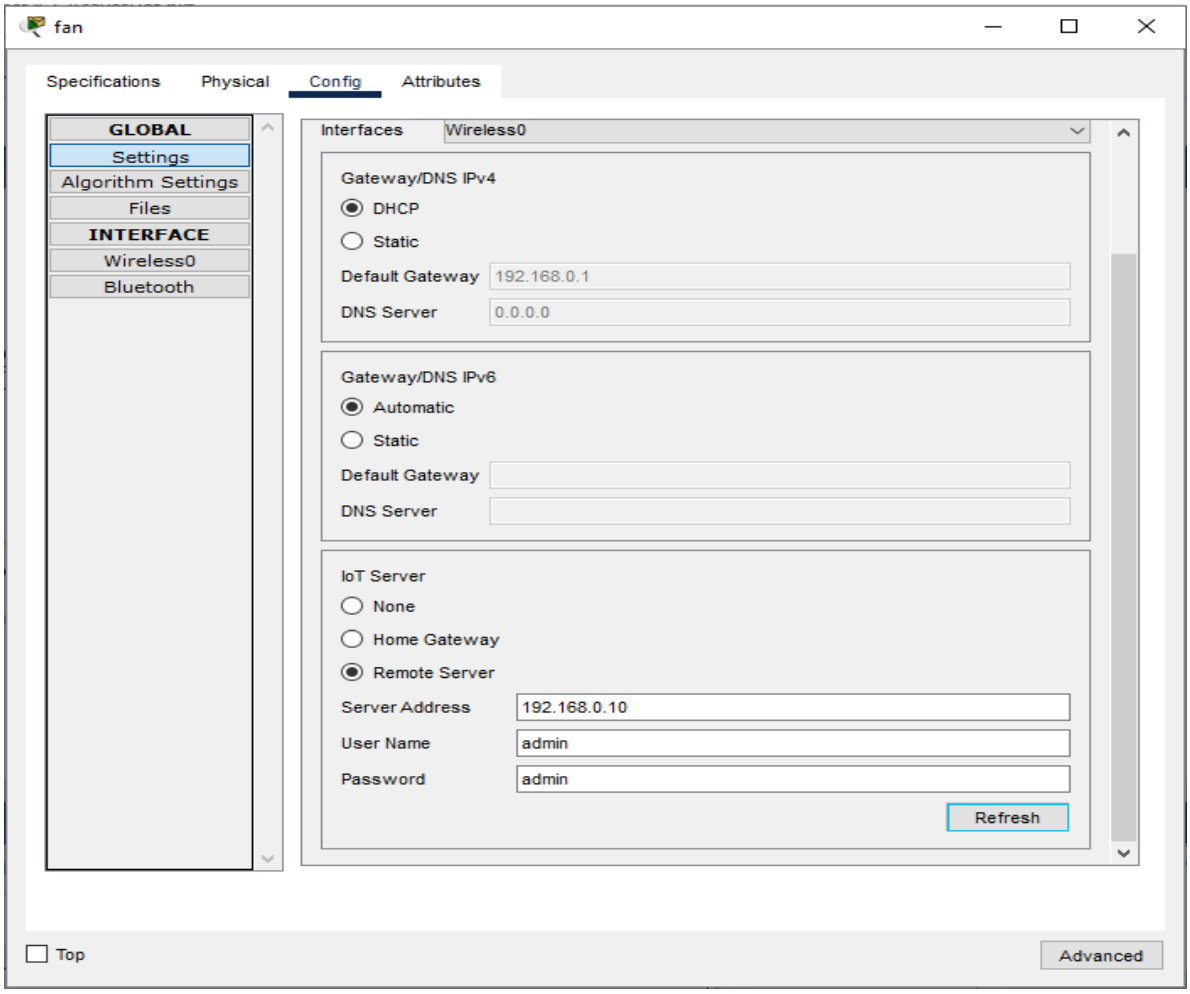
Configuration for fan
Goto->config->change the display name as fan
Goto->settings->follow the screenshot



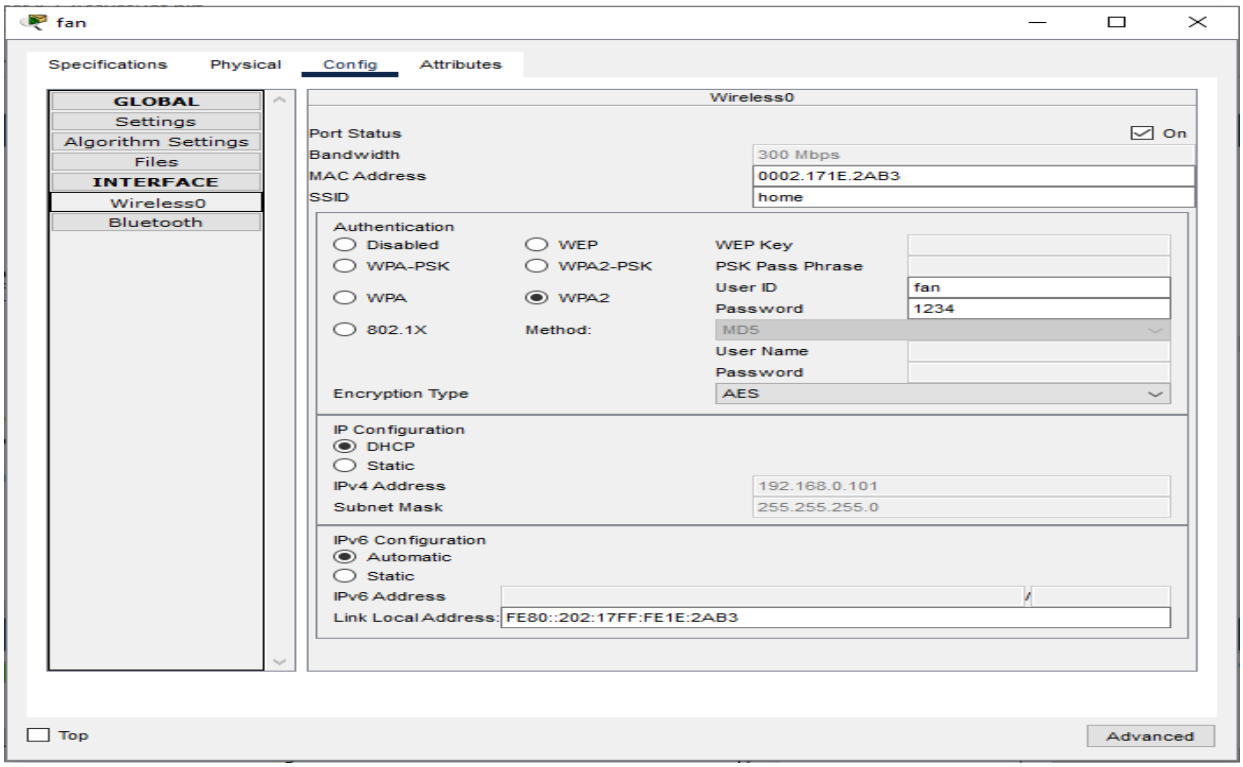


In settings->configure remoteserver as shown above

Below is the screenshot we get after connecting



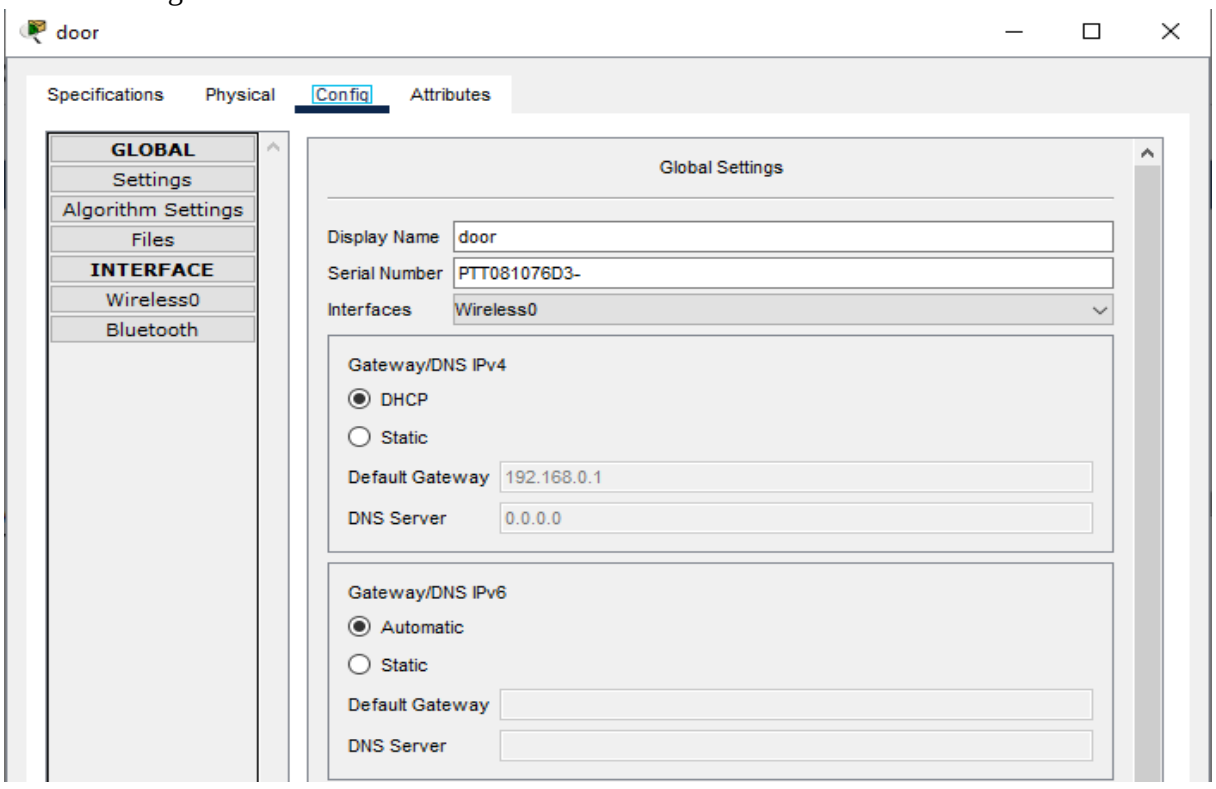
Under->config->wireless0->do the below config as shown in the screenshot



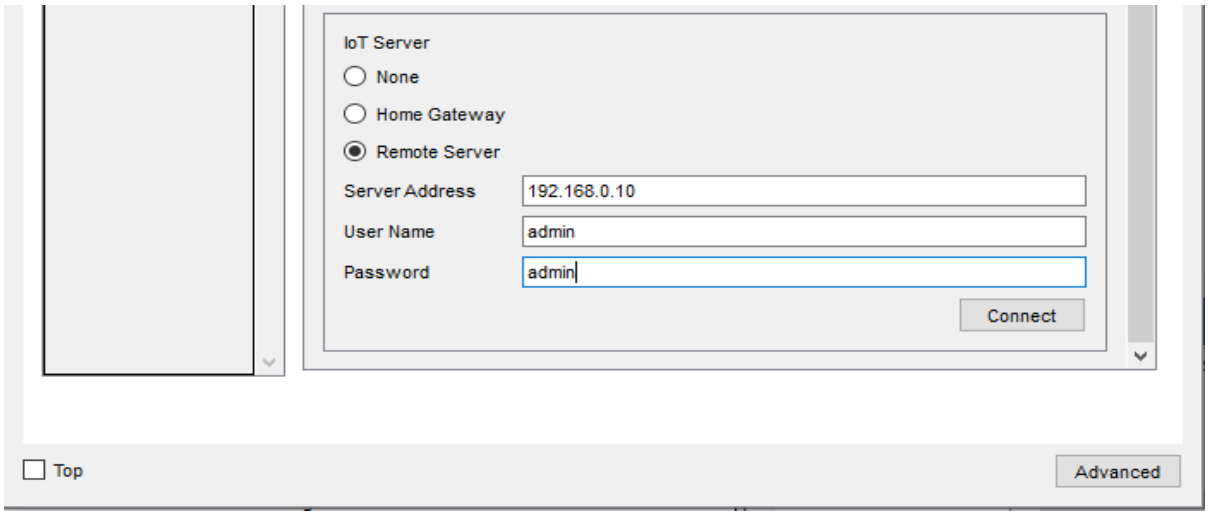
Configuration for Door

Goto->config->change the display name as door

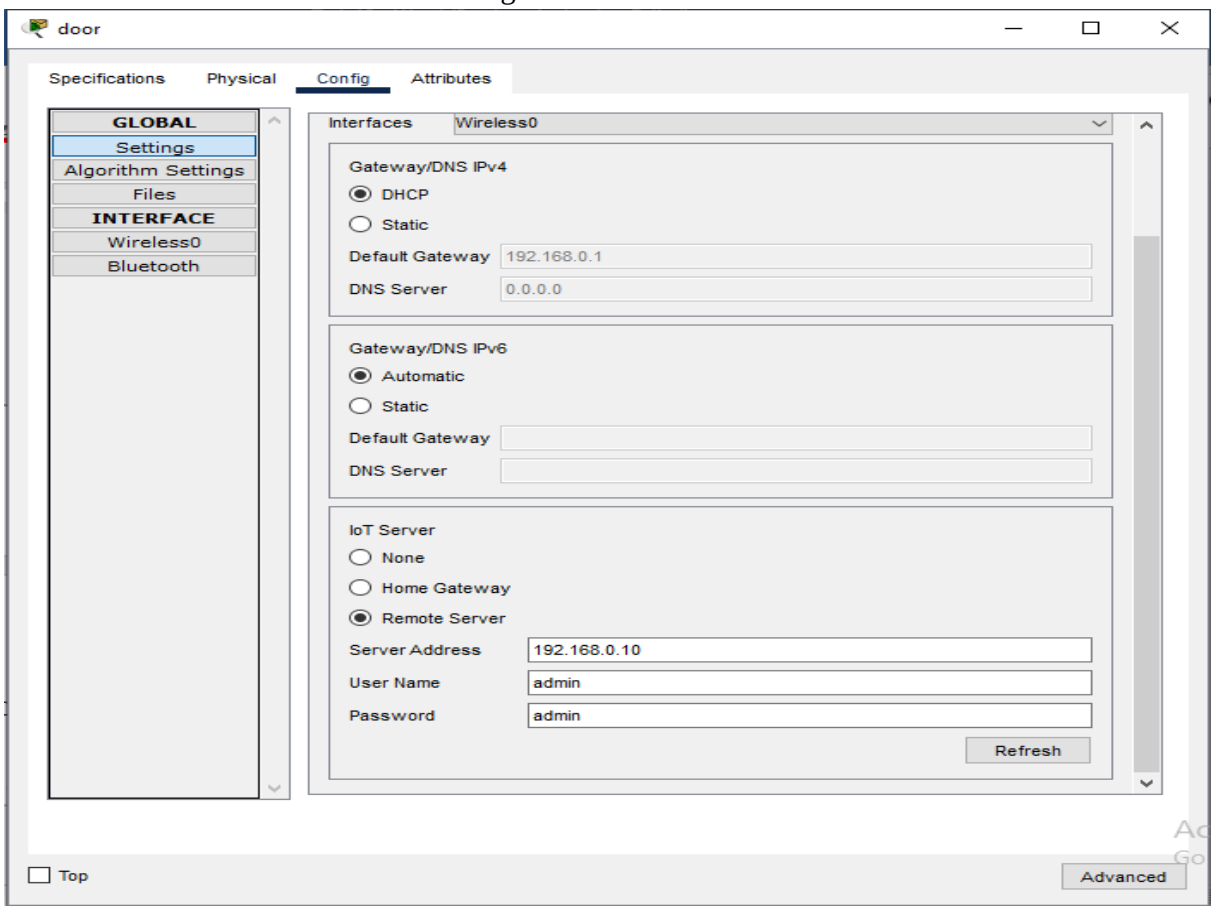
Goto->settings->follow the screenshot



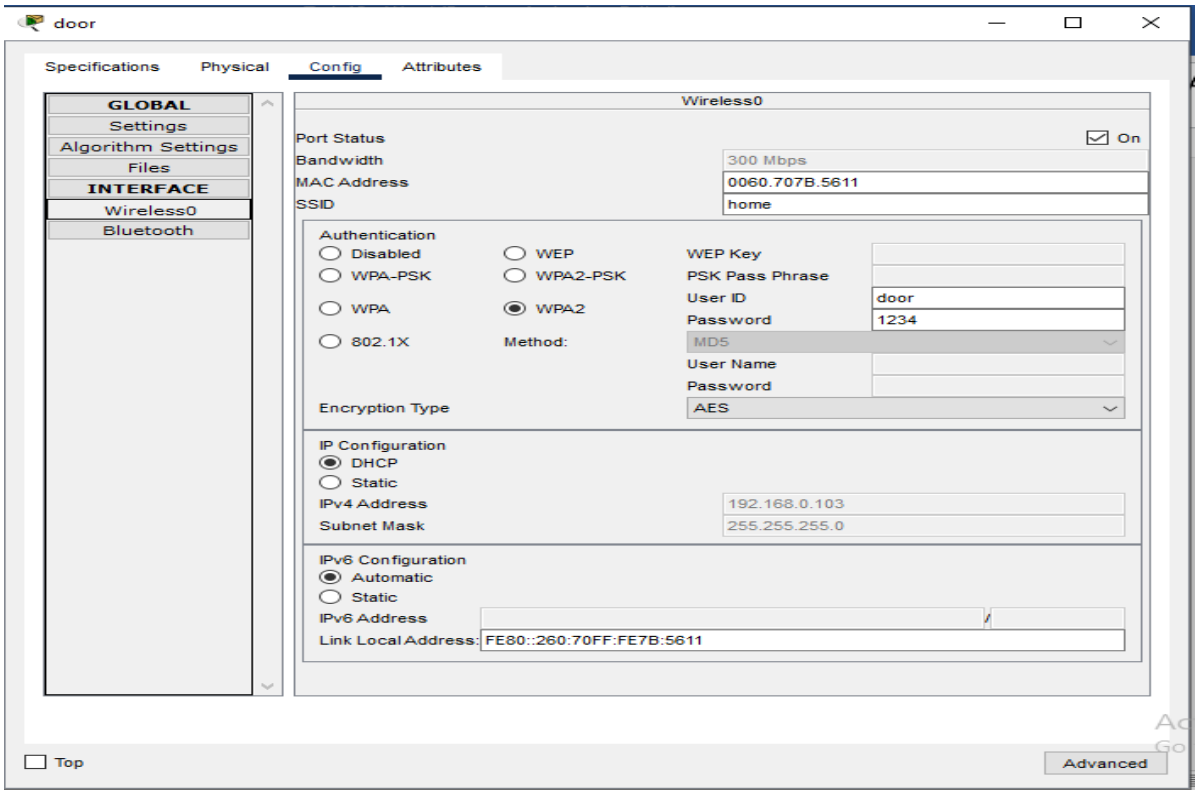
Click on connect



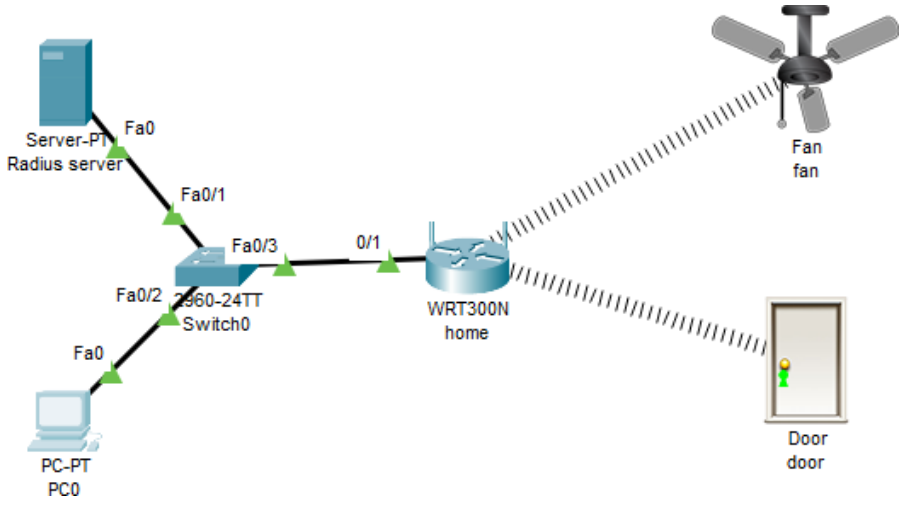
Below is the screenshot after connecting



In config->wireless0 do below configuration



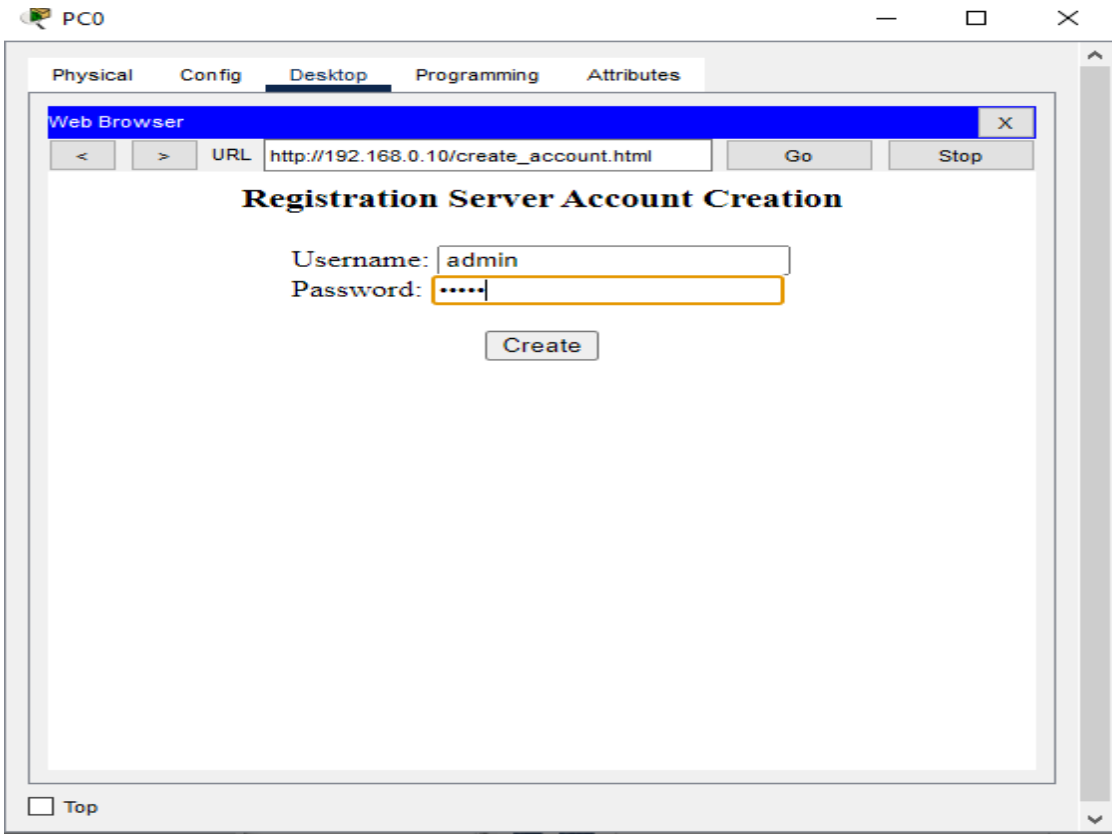
We will get link line between router and devices if both got connected successfully as shown in the below screenshot



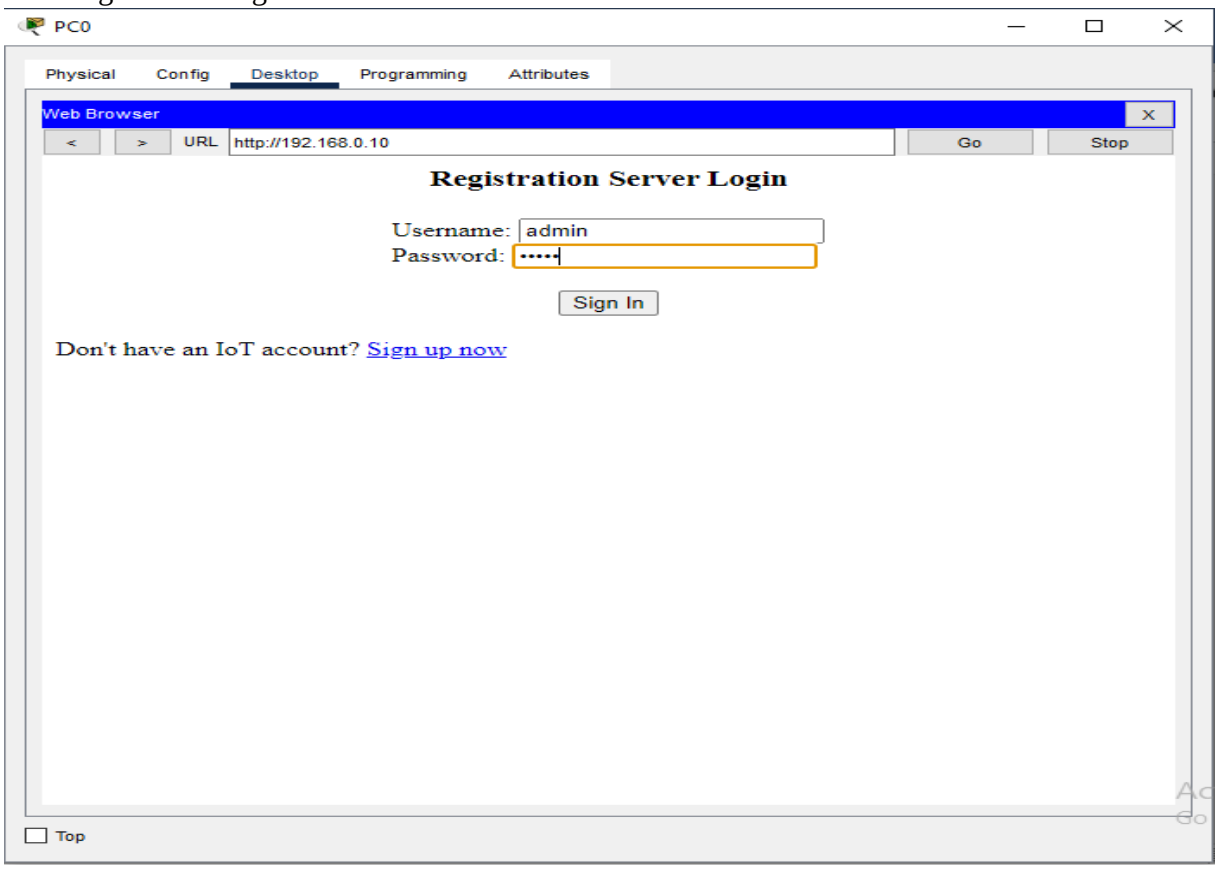
Testing

We on/off the fan and door from the PC

Goto PC->Desktop->webbrowser->give https://192.168.0.10 and register as shown below

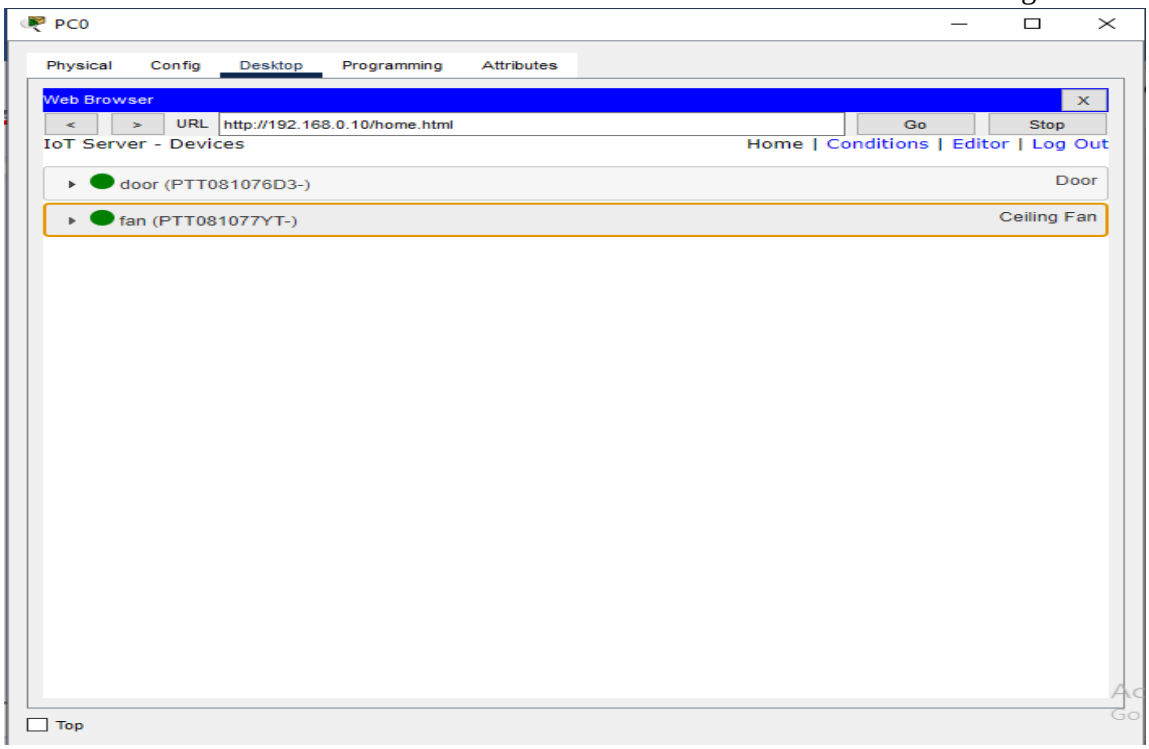


Then signin after registration

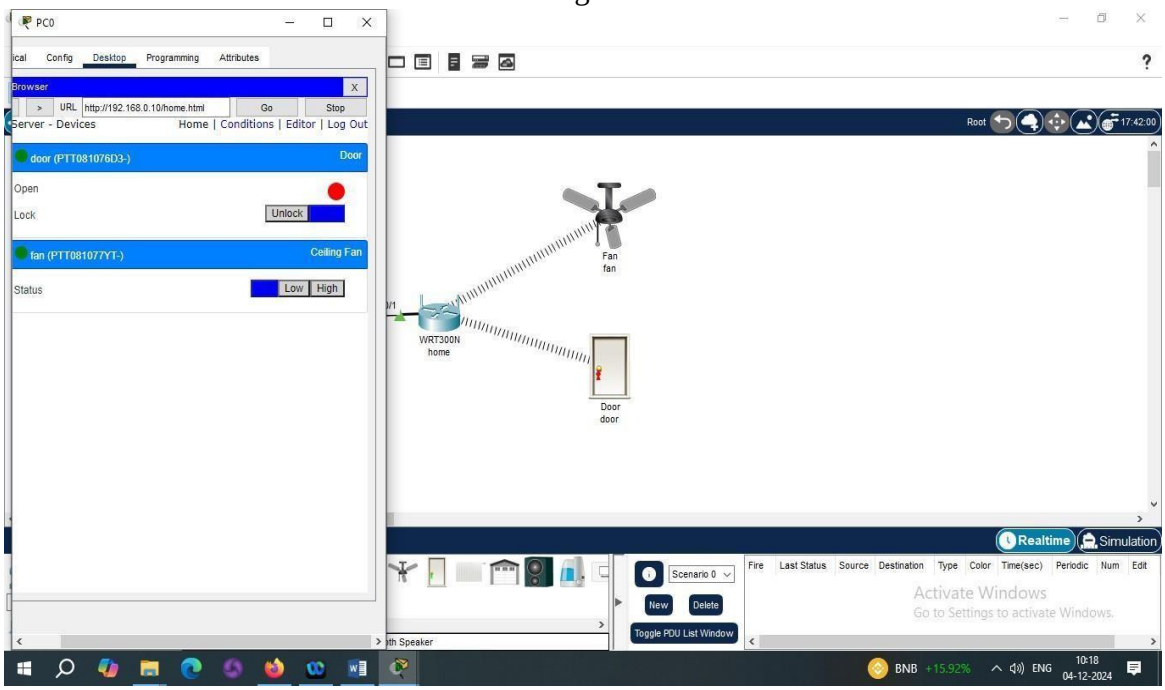


Once signin we will get below screen as it screenshot

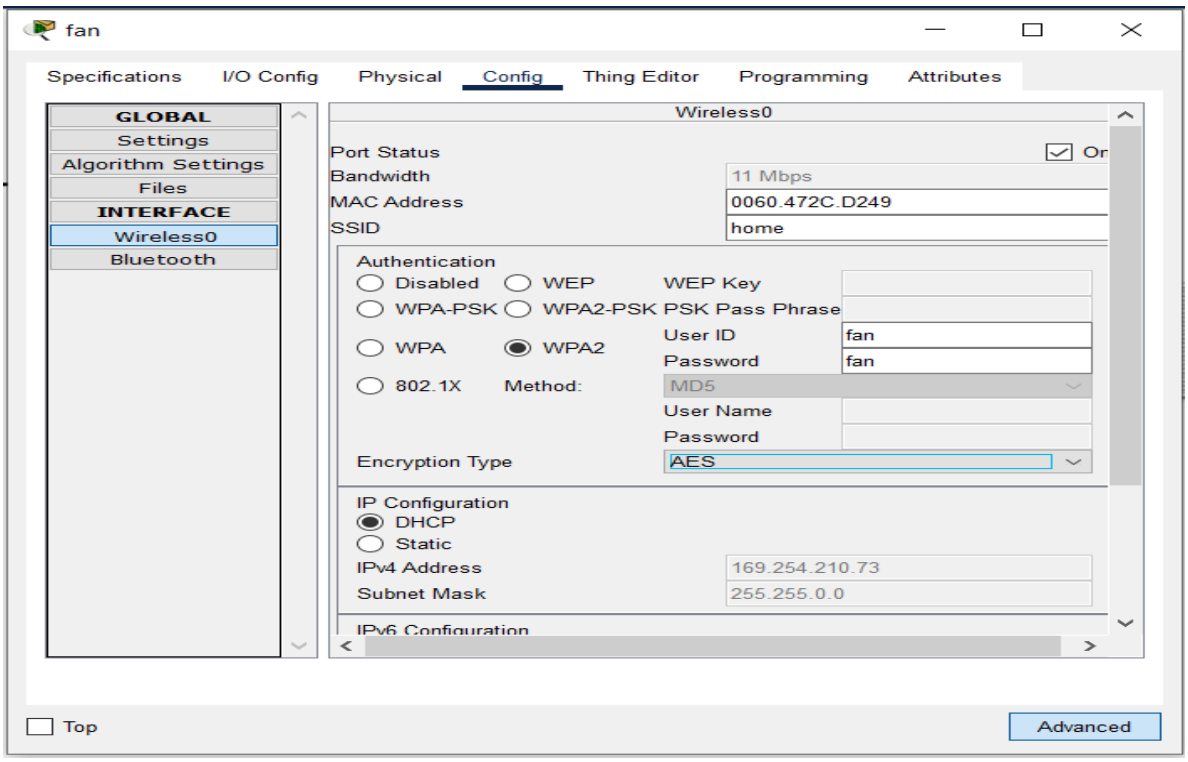
Note: If ur not able to see devices added check if DHCP is enabled in PC->config

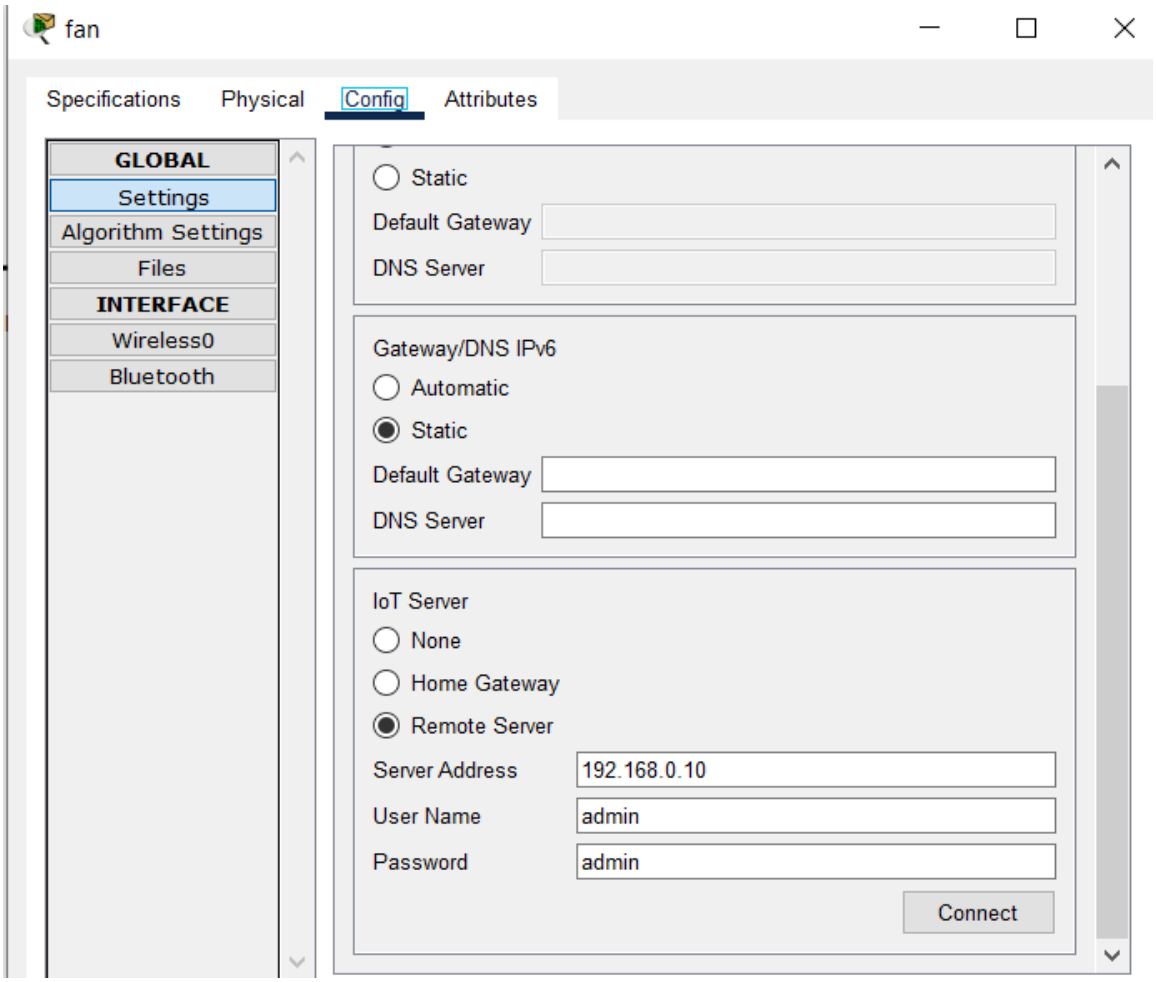


Click on small arrow left of fan and we can see controls for on/off and for door lock/unlock
Door turns red if we click on lock and turns green if we click unlock



For fan if we click on low, vibration is less and if we click on high, vibration will be more as shown in below screenshot


fan
Specifications
I/O Config
Physical
Config
Thing Editor
Programming
Attributes
GLOBAL
Settings
Algorithm Settings
Files
INTERFACE
Wireless0
Bluetooth
Wireless0
Port Status
Bandwidth
11 Mbps
MAC Address
0060.472C.D249
SSID
home
Authentication
Disabled
WEP
WEP Key
WPA-PSK
WPA2-PSK
PSK Pass Phrase
WPA
WPA2
User ID
fan
Password
fan
802.1X
Method:
MD5
User Name
Password
Encryption Type
AES
IP Configuration
DHCP
Static
IPv4 Address
169.254.210.73
Subnet Mask
255.255.0.0
Top
Advanced


fan
Specifications
Physical
Config
Attributes
GLOBAL
Settings
Algorithm Settings
Files
INTERFACE
Wireless0
Bluetooth
Static
Default Gateway
DNS Server
Gateway/DNS IPv6
Automatic
Static
Default Gateway
DNS Server
IoT Server
None
Home Gateway
Remote Server
Server Address
192.168.0.10
User Name
admin
Password
admin
Connect